Improving Function-Hiding Functional Commitments

Anonymous Submission

Abstract. Function-hiding functional commitment schemes allow one party to commit to a private function f and later prove $f(x) = y$ for public x and y without revealing additional information about the function. We construct efficient function-hiding functional commitment schemes for arithmetic circuits of bounded size that achieve proof sizes below 1.6 kB—over an order of magnitude smaller than previous constructions—while simultaneously reducing proving and verification times. We achieve these results by introducing a novel information-theoretic interactive proof system called *Polynomial Interactive Oracle Proofs with Randomized Indexer* (rPHPs). By compiling rPHPs with commit-and-prove zkSNARKs, we are able to leverage relaxed zero-knowledge notions for our building blocks. This approach eliminates the overhead of strict privacy requirements of prior work, directly translating into improved efficiency in both communication and computation.

Keywords: zero-knowledge · succinct arguments · commit-and-prove

1 Introduction

Commitment schemes play a fundamental role in cryptographic protocols, both from a theoretical and a practical point of view. Informally, they allow one party, the sender, to commit to a message m and later open the commitment to reveal the message. The classical *binding* property guarantees that the sender cannot open the commitment to two different messages, while the *hiding* property ensures that the commitment reveals no information about m until it is opened.

Over the years, a variety of richer commitment primitives have been introduced to support more expressive use cases. Notable examples include *vector commitments* [CF13], where the committed message is a vector $m = (m_1, \ldots, m_n)$ that can be selectively opened at chosen positions, and *polynomial commitments* [KZG10], where the committed message is a polynomial $p(X) \in \mathbb{F}[X]$ that can be opened at arbitrary points $x \in \mathbb{F}$ to prove claims of the form $p(x) = y$.

Function-hiding functional commitments (FHFCs), introduced by Boneh, Nguyen, and Ozdemir [BNO21], further generalize these notions by allowing the sender to commit to an entire function $f : \mathcal{D}_{\mathsf{in}} \to \mathcal{D}_{\mathsf{out}}$, represented by an arithmetic circuit of bounded size. Once a function is committed, the sender can later prove, for many different inputs x, that $f(x) = y$, while keeping the description of f hidden.

This ability to reuse a single commitment across many evaluations is the key feature that enables the applications envisioned in [BNO21]. Examples include private access-control systems, where a server commits once to a hidden policy and later proves, for many requests, that the same policy is satisfied; verifiable APIs and smart contracts whose logic is proprietary but invoked repeatedly; privacy-preserving machine learning services, where a model owner commits to a model and proves correct inference on many inputs; and digital rights management mechanisms in which usage rules must remain secret across multiple evaluations. In all these settings, hiding the input or the output alone is insufficient—the function itself must remain confidential *across many executions*.

Building function-hiding functional commitments. A central challenge in constructing function-hiding functional commitments is supporting this repeated reuse securely. After committing once to a secret function f, the sender should be able to produce multiple proofs of statements of the form $f(x) = y$, possibly on adaptively chosen inputs, while all proofs remain consistent with the same underlying committed function.

Proving correctness of such evaluations naturally suggests the use of *commit-and-prove zkSNARKs* (CP-SNARKs) [CFQ19]. This leads to modeling function-hiding functional commitments as zkSNARKs for relations of the form

$$\hat{\mathcal{R}}_{\mathsf{FHFC}} = \{((\boldsymbol{c}, x, y), \mathsf{R}) \mid \mathsf{R} \in \mathcal{D}_{\mathsf{func}} \ \wedge \ (x, y) \in \mathcal{L}(\mathsf{R}) \ \wedge \ \mathsf{Commit}(\mathsf{R}) = \boldsymbol{c}\},$$

where $\mathcal{D}_{\mathsf{func}}$ denotes the set of relations that define functions, i.e., the relations satisfying that for every input x there is a unique output y such that $(x, y) \in \mathcal{L}(\mathsf{R})$.

To build said zkSNARK for $\hat{\mathcal{R}}_{\mathsf{FHFC}}$, Boneh, Nguyen and Ozdemir [BNO21] follow the known paradigm of constructing arguments by compiling an information-theoretic protocol with a cryptographic primitive. The building blocks are an *Algebraic Holographic Proof* (AHP) [CHM+20], which demonstrates $(x, y) \in \mathcal{L}(\mathsf{R})$, and a new primitive called the *Proof of Function Relation* (PFR), which is the protocol that certifies $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}$. These components are then compiled using a hiding and binding polynomial commitment scheme into a zkSNARK for $\hat{\mathcal{R}}_{\mathsf{FHFC}}$. In order to preserve zero-knowledge, the AHP must additionally satisfy a non-standard security notion that they call *index-privacy*, which ensures that proofs do not reveal any information about the relation R.

The main problem: high zero-knowledge requirements leading to subpar efficiency. One drawback of the compiler from [BNO21] are the high zero-knowledge requirements imposed on both the AHP and the polynomial commitment, which have a noticeable impact on efficiency. At the information theoretic level, the techniques used to achieve index-privacy can result in AHPs which double the amount of polynomials the prover needs to send compared to a standard zero-knowledge construction. Moreover, their compiler cannot use the standard KZG polynomial commitment [KZG10], as it does not satisfy the required privacy guarantees, and instead relies on more expensive fully hiding variants.

A similar efficiency issue was addressed by Campanelli, Faonio, Fiore, Querol and Rodríguez [CFF+21], who showed how to build universal zkSNARKs with relaxed zero-knowledge requirements for both the polynomial commitment and the information-theoretic primitive, known in their work as *Polynomial Holographic Interactive Oracle Proofs* (PHPs). Namely, they demonstrated it is enough for the polynomial commitment to satisfy a relaxed privacy notion called *somewhat hiding* (which standard KZG already fulfills), and that the PHP prover can afford to leak some evaluations of the witness-dependent polynomials without compromising zero-knowledge.

At first sight, one could therefore attempt to translate the compiler from [BNO21] into the framework of [CFF+21] in order to improve its efficiency. While correct, we deem that this approach is unsatisfactory. Recall that in a PHP there is an *indexer* that takes as input a relation R and deterministically outputs some polynomials encoding it, which the verifier is given oracle access to. In the setting of [CFF+21], the relation R is public, so they do not need to prevent any leakage from the evaluations of those polynomials. In the function-hiding setting, however, the relation is secret and reused across many evaluations. Revealing even a single evaluation of any of the polynomials output by the indexer would then break zero-knowledge, as they deterministically depend on R. Therefore, enforcing index-privacy in this setting requires techniques similar to those of [BNO21], largely negating the efficiency gains of the framework from [CFF+21].

Our contributions. Our main contribution is a novel compiler for building function-hiding functional commitments that overcomes the aforementioned efficiency limitations. The key technical ingredient is a generalization of PHPs, which we call *Polynomial Interactive Oracle Proofs with Randomized Indexer* (rPHPs). Unlike standard PHPs, where the indexer is deterministic, rPHPs allow the indexer to take as input some randomness when encoding the relation. Crucially, this randomization breaks the deterministic link between the relation and the polynomials that encode it, allowing us to define a relaxed notion of index-privacy in which we are permitted to reveal some evaluations of the polynomials output by the indexer. The primary advantage of this relaxed definition is that it is significantly easier to achieve, rendering index-private rPHPs nearly as efficient as standard zero-knowledge PHPs. Furthermore, following [CFF+21], this also allows us to compile them using a somewhat-hiding polynomial commitment scheme, such as standard KZG.

We demonstrate the power of this framework through two concrete instantiations that build on the popular Marlin [CHM+20] and Plonk [GWC19] constructions.

- **A secure and optimized Marlin-based scheme.** We identify that the AHP presented in [BNO21] was incorrectly claimed to be index-private (a gap also pointed out in [DXNT24]). We provide a fixed construction where our rPHP for Marlin requires sending only two additional polynomials over the standard protocol. Thanks to the use of the KZG commitment, our PFR is essentially an adaptation of the lookup from [CFF+24], yielding a highly efficient construction.

- **A scheme based on a generalization of Plonk.** We introduce the *Generalized Plonk Constraint System*, which allows the wiring permutation to be an arbitrary function. To support this, we modify the construction of [GWC19] (which is secure only for permutations) using techniques from [GPS25]. This generalization significantly simplifies the PFR, making it almost identical to the one used in our Marlin-based scheme.

- **Efficiency.** In both cases, the prover runtime is quasilinear in the size of the relation while the verifier is logarithmic and requires only 2 pairings. At the 128-bit security level, our Marlin-based construction has a total proof size of 1.504 kB (0.752 kB for the PFR and 0.752 kB for the evaluation proof) and our Plonk-based construction 1.568 kB (0.672 kB for the PFR and 0.896 kB for the evaluation proof). In contrast, [BNO21] estimate the proof size of their Marlin-based construction to be approximately 21 kB. Consequently, our work represents a conservative 14× reduction in total communication. Even considering only the recurring evaluation cost (ignoring the PFR which only needs to be computed once), our scheme is 2.6× more compact than the (insecure) protocol from [BNO21].

Other related work. Function-hiding functional commitments should not be confused with the related notion of *functional commitments* introduced by Libert, Ramana and Yung [LRY16]. Here, the sender commits to a secret message x and can later reveal $f(x)$ for a public function f. As pointed out in [BNO21], theoretically, both concepts can be shown to be equivalent by the use of a universal function evaluator $U(f, x) = f(x)$. However, this transformation can impose severe inefficiencies in practice, so it is more convenient to see them as distinct notions. Furthermore, in contrast to most works in this area which prioritize falsifiable assumptions, [BNO21] employs non-falsifiable assumptions to obtain succinct, non-interactive proofs.

2 Preliminaries

For $n \in \mathbb{N}$, $[n]$ denotes the set $\{1, \ldots, n\}$. If $m \in \mathbb{N}$ and $m > n$, $[n, m]$ denotes the set $\{n, n+1, \ldots m-1, m\}$. For a set S, $|S|$ denotes its cardinality and $s \leftarrow S$ represents that s is an element sampled uniformly at random from S.

We denote by $\mathbb{F}$ a finite field, by $\mathbb{F}[X]$ the ring of polynomials in the variable X over $\mathbb{F}$, and by $\mathbb{F}^{\leq d}[X]$, the space of polynomials in $\mathbb{F}[X]$ of degree less or equal to d.

2.1 Bilinear Groups

A *bilinear group* is a tuple $\mathsf{gk} = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, \mathcal{P}_1, \mathcal{P}_2)$, where $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are groups of prime order q, $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is a non-degenerate bilinear map, known as the *pairing*, and $\mathcal{P}_1, \mathcal{P}_2$ are generators of $\mathbb{G}_1, \mathbb{G}_2$ respectively. Elements in $\mathbb{G}_\gamma$ are denoted implicitly as $[a]_\gamma = a\mathcal{P}_\gamma$, where $\gamma \in \{1, 2, T\}$ and $\mathcal{P}_T = e(\mathcal{P}_1, \mathcal{P}_2)$. Moreover, membership in $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ can be efficiently decided, group operations and the pairing are efficiently computable, and there is no efficient algorithm to compute an isomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$.

2.2 Cryptographic Assumptions

We will analyze the security of our constructions in the *Algebraic Group Model* (AGM) of Fuchsbauer, Kiltz and Loss [FKL18]. This is an idealized model in which every adversary $\mathcal{A}$ is restricted to be *algebraic*, in the sense that every group element output by $\mathcal{A}$ is accompanied by a representation of that group element in terms of all the group elements that $\mathcal{A}$ has seen so far.

Definition 1 ((q_1, q_2)-DLOG assumption)**.** For fixed $q_1, q_2 \in \mathbb{N}$, the (q_1, q_2)-DLOG assumption holds if for every polynomial time adversary, the following probability is negligible in λ:

$$\Pr\left[\mathsf{gk} \leftarrow \mathcal{G}(1^\lambda),\ x \leftarrow \mathbb{F}^*,\ x' \leftarrow \mathcal{A}\left(\mathsf{gk}, \{[x^i]_1\}_{i=1}^{q_1}, \{[x^i]_2\}_{i=1}^{q_2}\right) \mid x = x'\right].$$

2.3 Relations

A *universal relation* (or *indexed relation*) $\mathcal{R}$ is a set of triples $(\mathsf{R}, \mathsf{x}, \mathsf{w})$ where $\mathsf{R} \in \mathcal{D}_{\mathsf{rel}}$ is a relation, $\mathsf{x} \in \mathcal{D}_{\mathsf{ins}}$ is the *instance*, $\mathsf{w} \in \mathcal{D}_{\mathsf{wit}}$ the *witness*, and the domains $\mathcal{D}_{\mathsf{ins}}$ and $\mathcal{D}_{\mathsf{wit}}$ may depend on R. Given $\mathcal{R}$, the corresponding *universal language* is the set $\mathcal{L}(\mathcal{R}) = \{(\mathsf{R}, \mathsf{x}) \mid \exists \mathsf{w} \text{ such that } (\mathsf{R}, \mathsf{x}, \mathsf{w}) \in \mathcal{R}\}$. We also write $\mathcal{R}(\mathsf{R}, \mathsf{x}, \mathsf{w}) = 1$ (respectively $\mathsf{R}(\mathsf{x}, \mathsf{w}) = 1$) to denote $(\mathsf{R}, \mathsf{x}, \mathsf{w}) \in \mathcal{R}$ (respectively $(\mathsf{x}, \mathsf{w}) \in \mathsf{R}$). Our universal relations $\mathcal{R}$ may be implicitly parametrized by a set of public parameters pp; when we want to make the public parameters explicit, we will write $\mathcal{R}(\mathsf{pp})$. We may also consider universal relations $\mathcal{R}$ that contain a single R. In this case we will directly write $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$ and ignore R.

2.4 Vanishing Polynomials and their Derivative. Lagrange Basis Polynomials. Univariate Sumcheck

Let $S \subseteq \mathbb{F}$ be any subset of cardinality n and let $s \in S$. The s-th Lagrange basis polynomial associated to S is denoted by $\lambda_s^S(X) \in \mathbb{F}[X]$ and we denote by $z_S(X) = \prod_{s \in S}(X - s)$ the vanishing polynomial over S. We define the formal derivative of the vanishing polynomial as $u_S(X, Y) = \frac{z_S(X) - z_S(Y)}{X - Y}$, which is a polynomial of individual degree $|S| - 1$. When S is a multiplicative subgroup then $z_S(X) = X^n - 1$, $u_S(X, Y) = \frac{X^n - Y^n}{X - Y}$ and $\lambda_s^S(X) = \frac{s(X^n - 1)}{n(X - s)}$, which can be evaluated in $O(\log n)$ field operations. We assume multiplicative subgroups come with a bijection $\phi_S : S \to [n]$ that canonically orders the elements of S. We will

slightly abuse the notation and may use either elements of S or its image under ϕ_S to order lists. For example, for a matrix $M \in \mathbb{F}^{n\times n}$, we may write $M_{s,s'}$ to denote $M_{\phi_S(s),\phi_S(s')}$. For $t \in [n]$, we will also write $z_{S[\leq t]}(X)$ for the vanishing polynomial over the elements of S such that $\phi_S(s) \leq t$.

Lemma 1 (Univariate sumcheck)**.** *Let $p(X) \in \mathbb{F}^{\leq d}[X]$ and $\mathbb{H} \subseteq \mathbb{F}$ a multiplicative subgroup of order n. Then*

$$\sum_{\omega\in\mathbb{H}} p(\omega) = \sigma \iff \exists\, q(X), r(X) \mid p(X) = q(X)z_{\mathbb{H}}(X) + Xr(X) + \frac{\sigma}{n} \quad \text{with } \deg(r) < n-1.$$

2.5 Commitment Schemes

Following [CFF+21], we use the notion of *type-based commitments*, introduced by Escala and Groth [EG14]. Intuitively, a type-based commitment scheme supports multiple commitment "types", each of which may satisfy different security guarantees. In our setting, this flexibility will allow us to require fully hiding for commitments to the relation R, while allowing weaker (somewhat hiding) guarantees for other commitments.

Definition 2 (Type-Based Commitment Scheme)**.** A *type-based commitment scheme* for a set of types $\mathcal{T}$ with message space $\mathcal{M}$ is a tuple of PPT algorithms $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{VerCom})$ that works as follows:

- $\mathsf{Setup}(1^\lambda) \to \mathsf{ck}$: Given the security parameter, outputs a commitment key ck.
- $\mathsf{Commit}(\mathsf{ck}, \mathsf{type}, m) \to (c, o)$: Takes the commitment key ck, a type type, a message $m \in \mathcal{M}$, and outputs a commitment c and an opening o. We assume c contains information about its type, which we denote $\mathsf{type}(c)$.
- $\mathsf{VerCom}(\mathsf{ck}, \mathsf{type}, c, m, o) \to b$: Takes the commitment key ck, a commitment c, an opening o and message $m \in \mathcal{M}$, and it accepts ($b = 1$) or rejects ($b = 0$). By default it outputs 0 if $\mathsf{type}(c) \neq \mathsf{type}$.

We assume that CS satisfies the standard correctness notion. It must also be type-*typed binding* and may be type-*typed trapdoor hiding*, as defined next.

- type-*typed binding.* Let $\mathsf{type} \in \mathcal{T}$, CS is type-*typed binding* if for every PPT adversary $\mathcal{A}$ the following probability is negligible in the security parameter:

$$\Pr\left[\begin{array}{l} \mathsf{VerCom}(\mathsf{ck}, \mathsf{type}, c, m, o) = 1 \\ \mathsf{VerCom}(\mathsf{ck}, \mathsf{type}, c, m', o') = 1 \\ m \neq m' \end{array} \;\middle|\; \begin{array}{l} \mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda) \\ (c, m, o, m', o') \leftarrow \mathcal{A}(\mathsf{ck}, \mathsf{type}) \end{array}\right]$$

  We say that CS is *binding* if it is type-typed binding for any $\mathsf{type} \in \mathcal{T}$.

- type-*typed trapdoor-hiding.* There exist three algorithms $(\mathsf{ck}, \mathsf{td}) \leftarrow \mathcal{S}_{\mathsf{ck}}(1^\lambda)$, $(c, st) \leftarrow \mathsf{TdCom}(\mathsf{td}, \mathsf{type})$ and $o \leftarrow \mathsf{TdOpen}(\mathsf{td}, st, \mathsf{type}, c, m)$ such that: the distribution of the commitment key returned by $\mathcal{S}_{\mathsf{ck}}$ is statistically close to the one of the key returned by Setup; for any $m \in \mathcal{M}$, $(c, o) \approx (c', o')$ where $(c, o) \leftarrow \mathsf{Commit}(\mathsf{ck}, m)$, $(c', st) \leftarrow \mathsf{TdCom}(\mathsf{td}, \mathsf{type})$ and $o' \leftarrow \mathsf{TdOpen}(\mathsf{td}, st, \mathsf{type}, c', m)$.

  In this work, we will simply refer to trapdoor-hiding commitments as *hiding* (or *fully hiding*) commitments.

We will deviate from the original definition of polynomial commitment scheme [KZG10], which includes an algorithm to prove evaluations of the polynomial, and instead use the one from [CFF+21], which just defines them as commitment schemes with message space a set of polynomials. In this framework, evaluations are proven using commit-and-prove zkSNARKs.

Definition 3 (Polynomial Commitment Scheme)**.** A *polynomial commitment scheme* (PCS) is a commitment scheme with message space $\mathcal{M} = \mathbb{F}^{\leq d}[X]$ for some $d \in \mathbb{N}$. We assume without loss of generality that d is given as an input of Setup.

We recall the notion of *somewhat hiding polynomial commitment schemes* [CFF+21], which is a relaxation of the standard hiding property.

Definition 4 (Somewhat Hiding PCS)**.** Let CS be a type-based commitment scheme for a class of polynomials $\mathbb{F}_{\leq d}[X]$. We say that CS is *somewhat hiding* for the type type if there exist three algorithms $(\mathsf{ck}, \mathsf{td} = (\mathsf{td}', s)) \leftarrow \mathcal{S}_{\mathsf{ck}}(s)$, where $s \in \mathbb{F}$, $(c, st) \leftarrow \mathsf{TdCom}(\mathsf{td}, \gamma)$ and $o \leftarrow \mathsf{TdOpen}(\mathsf{td}, st, c, f)$ such that: the distribution of the commitment key returned by $\mathcal{S}_{\mathsf{ck}}$ with a uniformly random $s \leftarrow \mathbb{F}$ as input is identical to the one of the key returned by Setup, and for any $f \in \mathbb{F}_{\leq d}[X]$, $(c, o) \approx (c', o')$, where $(c, o) \leftarrow \mathsf{Commit}(\mathsf{ck}, \mathsf{type}, f)$, $(c', st) \leftarrow \mathsf{TdCom}(\mathsf{td}, f(s))$ and $o' \leftarrow \mathsf{TdOpen}(\mathsf{td}, st, c', f)$.

Finally, we recall the KZG polynomial commitment scheme [KZG10]. It is known that KZG is binding under the $(d+1, 2)$-DLOG assumption and it was proven in [CFF+21] that it is somewhat hiding.

Definition 5 (KZG)**.** Let $\mathsf{gk} = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, \mathcal{P}_1, \mathcal{P}_2)$ be a bilinear group. The KZG polynomial commitment scheme for the message space $\mathbb{F}^{\leq d}[X]$ is made up of the following algorithms.

- $\mathsf{Setup}(1^\lambda, d)$: on input $(1^\lambda, d)$, where d is the degree bound, sample $\tau \leftarrow \mathbb{F}$ and output
$$\mathsf{ck} = \Big(\{[\tau^i]_1\}_{i \in [0,d]}, [1, \tau]_2\}\Big).$$
- $\mathsf{Commit}(\mathsf{ck}, p)$: on input the commitment key ck, and a polynomial $p(X)$ of degree at most d, outputs a commitment $c = [p(\tau)]_1$. The opening o is empty and therefore omitted.
- $\mathsf{VerCom}(\mathsf{ck}, c, p)$ outputs 1 if $c = [p(\tau)]_1$.

2.6 CP-SNARKs

Commit-and-Prove SNARKs (CP-SNARKs) are zkSNARKs whose relations verify predicates over commitments [CFQ19]. Given a commitment scheme CS, we consider relations $\hat{\mathcal{R}}$ whose instances are of the form $\hat{\mathsf{x}} = ((c_j)_{j \in [\ell]}, \mathsf{x})$, where we can unambiguously parse the witness as $\hat{\mathsf{w}} = ((w_j)_{j \in [\ell]}, (o_j)_{j \in [\ell]})$ for some $\ell \in \mathbb{N}$. We assume w_j to be in the domain of the commitment scheme CS, o_j to be the corresponding opening and that the public parameters pp include the commitments key ck of CS. Moreover, there must exists a polynomial time relation $\mathcal{R}$ such that for $\mathsf{w} = (w_j)_{j \in [\ell]}$, we have:

$$\hat{\mathcal{R}}(\hat{\mathsf{x}}, \hat{\mathsf{w}}) = 1 \iff \mathcal{R}(\mathsf{x}, \mathsf{w}) = 1 \ \wedge \ \forall j \in [\ell], \ \mathsf{VerCom}(\mathsf{ck}, c_j, w_j, o_j) = 1.$$

For a commitment key ck, we will identify $\hat{\mathcal{R}}$ with the pair $(\mathcal{R}, \mathsf{ck})$. A CP-SNARK for $\mathcal{R}$ over a commitment scheme CS is a zkSNARK for the associated relation $\hat{\mathcal{R}}$. More in detail, we consider a tuple of algorithms $\mathsf{CP} = (\mathsf{KeyGen}, \mathsf{Prove}, \mathsf{Verify})$ where:

- $\mathsf{KeyGen}(\mathsf{ck}) \to (\mathsf{srs}, \mathsf{td_k})$ is an algorithm that takes as input a commitment key ck for CS and it outputs $\mathsf{srs} := (\mathsf{ek}, \mathsf{vk})$, where ek is the evaluation key, vk is the verification key. We assume that the public parameters for the relation $\mathcal{R} = \mathcal{R}(\mathsf{pp})$ (which include the commitment key ck) are also contained in srs; $\mathsf{td_k}$ is a trapdoor needed for the simulation.

- $\mathsf{Prove}(\mathsf{ek}, \hat{\mathsf{x}}, \hat{\mathsf{w}}) \to \pi$ takes an evaluation key ek, a statement $\hat{\mathsf{x}}$ and a witness $\hat{\mathsf{w}}$ such that $\hat{\mathcal{R}}(\hat{\mathsf{x}}, \hat{\mathsf{w}})$ holds, and returns a proof π.
- $\mathsf{Verify}(\mathsf{vk}, \hat{\mathsf{x}}, \pi) \to b$ takes a verification key, a statement $\hat{\mathsf{x}}$, and a proof π, and either accepts ($b = 1$) or rejects ($b = 0$).

Definition 6 (Succinctness)**.** *Succinctness* holds if the size of the proof π is $\mathsf{poly}(\lambda + \log|\hat{\mathsf{w}}|)$ and Verify runs in time $\mathsf{poly}(\lambda + |\hat{\mathsf{x}}| + \log|\hat{\mathsf{w}}|)$

Definition 7 (Completeness)**.** A CP-SNARK is *complete* if for any $(\hat{\mathsf{x}}, \hat{\mathsf{w}}) \in \hat{\mathcal{R}}$,

$$\Pr\left[\begin{array}{l} \mathsf{ck} \leftarrow \mathsf{CS.Setup}(1^\lambda) \\ (\mathsf{srs}, \mathsf{td_k}) \leftarrow \mathsf{CP.KeyGen}(\mathsf{ck}) \\ \pi \leftarrow \mathsf{CP.Prove}(\mathsf{ek}, \hat{\mathsf{x}}, \hat{\mathsf{w}}) \end{array} \middle| \mathsf{CP.Verify}(\mathsf{vk}, \hat{\mathsf{x}}, \pi) = 1\right] = 1$$

Definition 8 (Knowledge Soundness)**.** We say CP is *knowledge sound* if for every PPT adversary $\mathcal{A}$, there exists an efficient extractor $\mathcal{E}$ such that $\Pr[\mathsf{Game}_{\mathcal{A},\mathcal{E}}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$, where

$$\begin{array}{l} \mathsf{Game}_{\mathcal{A},\mathcal{E}}(1^\lambda) \to b' \\ \hline \mathsf{ck} \leftarrow \mathsf{CS.Setup}(1^\lambda)\,;\ \ (\mathsf{srs} := (\mathsf{ek}, \mathsf{vk}), \mathsf{td_k}) \leftarrow \mathsf{KeyGen}(\mathsf{ck})\,;\ \ (\hat{\mathsf{x}}, \pi) \leftarrow \mathcal{A}(\hat{\mathcal{R}}, \mathsf{srs}) \\ \hat{\mathsf{w}} \leftarrow \mathcal{E}(\mathsf{srs})\,;\ \ b \leftarrow \mathsf{Verify}(\mathsf{vk}, \hat{\mathsf{x}}, \pi)\,;\ \ b_E \leftarrow \hat{\mathcal{R}}(\hat{\mathsf{x}}, \hat{\mathsf{w}})\,;\ \ \textbf{return}\ b' = b \wedge \neg b_E \end{array}$$

Definition 9 (Trapdoor-commitment zero-knowledge)**.** We say CP is *trapdoor-commitment zero-knowledge* if there exists a simulator $\mathcal{S}$ such that for all adversaries $\mathcal{A}$, $(\mathsf{ck}, \mathsf{td}) \in \mathcal{S}_{\mathsf{ck}}(1^\lambda)$ and for all $(\hat{\mathsf{x}}, \hat{\mathsf{w}}) \in \hat{\mathcal{R}}$,

$$\Pr\left[\begin{array}{l} (\mathsf{srs}, \mathsf{td_k}) \leftarrow \mathsf{CP.KeyGen}(\mathsf{ck}) \\ \pi \leftarrow \mathsf{CP.Prove}(\mathsf{ek}, \hat{\mathsf{x}}, \hat{\mathsf{w}}) \end{array} \middle| \mathcal{A}(\mathsf{srs}, \mathsf{td_k}, \mathsf{td}, \hat{\mathcal{R}}, \hat{\mathsf{x}}, \hat{\mathsf{w}}, \pi) = 1\right] \approx$$

$$\Pr\left[\begin{array}{l} (\mathsf{srs}, \mathsf{td_k}) \leftarrow \mathsf{CP.KeyGen}(\mathsf{ck}) \\ \pi \leftarrow \mathcal{S}(\mathsf{td_k}, \mathsf{td}, \hat{\mathcal{R}}, \hat{\mathsf{x}}) \end{array} \middle| \mathcal{A}(\mathsf{srs}, \mathsf{td_k}, \mathsf{td}, \hat{\mathcal{R}}, \hat{\mathsf{x}}, \hat{\mathsf{w}}, \pi) = 1\right]$$

Unless otherwise specified, we assume all the CP-SNARKs in this work to satisfy the previous definitions. For our compiler, some CP-SNARKs will need to satisfy the relaxed notion of *leaky zero-knowledge* [CFF+21]. Intuitively, it says that a proof may leak a bounded number of evaluations of some committed polynomials.

Definition 10 ((b, C)-bounded list)**.** We say that a list $\mathcal{L} = \{(i_1, y_1), \dots\}$ is (b, C)-*bounded* where $\mathsf{b} \in \mathbb{N}^n$, for some $n \in \mathbb{N}$, if C is a PT algorithm and $\forall i \in [n]$, we have $|\{(i, y) \mid (i, y) \in \mathcal{L}\}| \leq \mathsf{b}_i$, and $\forall (i, y) \in \mathcal{L}$, $\mathsf{C}(i, y) = 1$.

Definition 11 (Leaky zero-knowledge)**.** We say that a CP-SNARK CP for a relation over committed polynomials is (b, C)-*leaky zero-knowledge* if there exists a simulator $\mathcal{S} = (\mathcal{S}_{\mathsf{leak}}, \mathcal{S}_{\mathsf{prv}})$ such that for all adversaries $\mathcal{A}$, the following two properties hold.

- *Proof indistinguishability.* For all $(\mathsf{ck}, \mathsf{td}) \in \mathcal{S}_{\mathsf{ck}}(1^\lambda)$, for all $(\hat{\mathsf{x}}, \hat{\mathsf{w}}) \in \hat{\mathcal{R}}$, for any $\mathcal{L} \leftarrow \mathcal{S}_{\mathsf{leak}}(1^\lambda, \hat{\mathsf{x}})$, let $Leak := \{(i, p_i(y))\}_{(i,y)\in\mathcal{L}}$:

$$\Pr\left[\begin{array}{l} (\mathsf{srs}, \mathsf{td_k}) \leftarrow \mathsf{CP.KeyGen}(\mathsf{ck}) \\ \pi \leftarrow \mathsf{CP.Prove}(\mathsf{srs}, \hat{\mathsf{x}}, \hat{\mathsf{w}}) \end{array} \middle| \mathcal{A}(\mathsf{srs}, \mathsf{td_k}, \mathsf{td}, \hat{\mathcal{R}}, \hat{\mathsf{x}}, \hat{\mathsf{w}}, \pi) = 1\right] \approx$$

$$\Pr\left[\begin{array}{l} (\mathsf{srs}, \mathsf{td_k}) \leftarrow \mathsf{CP.KeyGen}(\mathsf{ck}) \\ \pi \leftarrow \mathcal{S}_{\mathsf{prv}}(\mathsf{td_k}, \mathsf{td}, \hat{\mathsf{x}}, Leak) \end{array} \middle| \mathcal{A}(\mathsf{srs}, \mathsf{td_k}, \mathsf{td}, \hat{\mathcal{R}}, \hat{\mathsf{x}}, \hat{\mathsf{w}}, \pi) = 1\right]$$

- *Bounded leakage.* For any $\hat{\mathsf{x}}$ and any $\mathcal{L} \leftarrow \mathcal{S}_{\mathsf{leak}}(1^\lambda, \hat{\mathsf{x}})$, the list $\mathcal{L}$ is (b, C)-bounded with overwhelming probability over the security parameter.

3 PHPs with Randomized Indexer

In this section, we introduce *Polynomial Holographic Interactive Oracle Proofs with Randomized Indexer* (rPHPs), a generalization of *Polynomial Holographic Interactive Oracle Proofs* (PHPs) [CFF+21]. Recall that PHPs work in two phases. In the *offline phase*, an *indexer*, also known as the *relation encoder*, takes as input a relation R and deterministically outputs some polynomials encoding it. In the *online phase*, the verifier is given oracle access to the polynomials output by the indexer and interacts with the prover. During this interaction, the prover may send polynomials (to which the verifier is given oracle access) and messages (which are explicit field elements), while the verifier replies with some messages. At the end of the interaction, the verifier executes a decision phase in which it outputs a collection of algebraic checks involving the information exchanged with the prover and the polynomials output by the indexer.

Informally, the (b, C)-zero-knowledge notion from [CFF+21] says that it suffices for a simulator to simulate the prover's explicit messages together with a bounded number of evaluations of the oracle polynomials involved in the protocol. Crucially, the simulator is not required to simulate any of the polynomials themselves. In the setting of [CFF+21], the simulator is given the relation R as input and can therefore trivially simulate the evaluations of the polynomials encoding the relation. In contrast, in our function-hiding setting, the simulator must not receive R, as the goal is to hide the relation. Consequently, we cannot afford to leak any evaluations of the index polynomials when they are deterministically derived from R, since such evaluations would in general be impossible to simulate without knowledge of the relation R.

As the name suggests, rPHPs generalize the concept of PHPs by allowing the indexer to take as input some randomness when encoding the relation. Critically, this randomization enables certain evaluations of the index polynomials to be independent of R, and therefore simulatable without access to the relation. This allows us to define a relaxed notion of index-privacy that is sufficient for our applications. Apart from this modification, the definition of rPHPs is essentially identical to that of standard PHPs.

Definition 12 (PHP with Randomized Indexer)**.** A *Polynomial Holographic Interactive Oracle Proof with Randomized Indexer* (rPHP) for a universal relation $\mathcal{R}$ is a tuple $\mathsf{rPHP} = (\mathsf{rnd}, \mathsf{n}, \mathsf{m}, \mathsf{d}, \mathsf{n_e}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ where $\mathsf{rnd}, \mathsf{n}, \mathsf{m}, \mathsf{d}, \mathsf{n_e} : \{0,1\}^* \to \mathbb{N}$ are polynomial-time computable functions, and $\mathcal{I}, \mathcal{P}, \mathcal{V}$ are algorithms that work as follows:

- **Offline phase:** The *indexer* (or *relation encoder*) $\mathcal{I}$ takes as input a relation R, some randomness r from a predefined set Rand and returns $\mathsf{n}(0)$ polynomials $\{p_{0,j}(X)\}_{j=1}^{\mathsf{n}(0)}$ encoding the relation R.

- **Online phase:** The prover $\mathcal{P}(\mathsf{R}, \mathsf{r}, \mathsf{x}, \mathsf{w})$ and the verifier $\mathcal{V}^{\mathcal{I}(\mathsf{R},\mathsf{r})}(\mathsf{x})$ are executed for $\mathsf{rnd}(|\mathsf{R}|)$ rounds. The prover has a tuple $(\mathsf{R}, \mathsf{x}, \mathsf{w}) \in \mathcal{R}$ and the indexer randomness r, and the verifier has an instance x and oracle access to the polynomials encoding R. In the i-th round, $\mathcal{V}$ sends a message ρ_i to the prover, and $\mathcal{P}$ replies with $\mathsf{m}(i)$ messages $\{\pi_{i,j} \in \mathbb{F}\}_{j \in \mathsf{m}(i)}$, and $\mathsf{n}(i)$ oracle polynomials $\{p_{i,j} \in \mathbb{F}[X]\}_{j \in \mathsf{n}(i)}$, such that $\deg(p_{i,j}) < \mathsf{d}(|\mathsf{R}|, i, j)$.

- **Decision phase:** After the $\mathsf{rnd}(|\mathsf{R}|)$-th round, the verifier outputs two sets of algebraic checks of the following type:

  - Degree checks: to check a bound on the degree of the polynomials sent by the prover. More in detail, let $\mathsf{n_p} = \sum_{k=1}^{\mathsf{rnd}(|\mathsf{R}|)} \mathsf{n}(k)$ and let $(p_1, \ldots, p_{\mathsf{n_p}})$ be the polynomials sent by $\mathcal{P}$. The verifier specifies a vector of integers $\boldsymbol{d} \in \mathbb{N}^{\mathsf{n_p}}$, which satisfies the following condition

$$\forall k \in [\mathsf{n_p}],\ \deg(p_k) \leq d_k.$$

– Polynomial checks: to check that certain polynomial identities hold between the oracle polynomials and the prover messages. More in detail, let $\mathsf{n}^* = \sum_{k=0}^{\mathsf{rnd}(|\mathsf{R}|)} \mathsf{n}(k)$ and $\mathsf{m}^* = \sum_{k=1}^{\mathsf{rnd}(|\mathsf{R}|)} \mathsf{m}(k)$, and denote by $(p_1, \dots p_{\mathsf{n}^*})$ and $(\pi_1, \dots, \pi_{\mathsf{m}^*})$ all the oracle polynomials (including the $\mathsf{n}(0)$ from the encoder) and all the messages sent by the prover. The verifier can specify a list of $\mathsf{n_e}$ tuples, each of the form $(G, v_1, \dots, v_{\mathsf{n}^*})$ where $G \in \mathbb{F}[X, X_1, \dots, X_{\mathsf{n}^*}, Y_1, \dots Y_{\mathsf{m}^*}]$ and every $v_k \in \mathbb{F}[X]$. Then, a tuple is satisfied if and only if $F(X) \equiv 0$, where

$$F(X) = G\Big(X, \{p_k\big(v_k(X)\big)\}_{k\in[\mathsf{n}^*]}, \{\pi_k\}_{k\in[\mathsf{m}^*]}\Big).$$

The verifier accepts if and only if all the checks are satisfied.

Definition 13 (Completeness)**.** We say rPHP is *complete* if for any triple $(\mathsf{R}, \mathsf{x}, \mathsf{w}) \in \mathcal{R}$ and any randomness $\mathsf{r} \in \mathsf{Rand}$, the checks returned by $\mathcal{V}^{\mathcal{I}(\mathsf{R},\mathsf{r})}$ after interacting with the honest prover $\mathcal{P}(\mathsf{R}, \mathsf{r}, \mathsf{x}, \mathsf{w})$ are satisfied with probability 1.

Definition 14 (Soundness)**.** We say rPHP is ϵ-*sound* if for every relation-instance pair $(\mathsf{R}, \mathsf{x}) \notin \mathcal{L}(\mathcal{R})$, every polynomial time prover $\mathcal{P}^*$ and every randomness $\mathsf{r} \in \mathsf{Rand}$, we have

$$\Pr\left[\langle \mathcal{P}^*, \mathcal{V}^{\mathcal{I}(\mathsf{R},\mathsf{r})}(\mathsf{x})\rangle = 1\right] \leq \epsilon$$

Definition 15 (Index-privacy)**.** We say rPHP is ϵ-*index-private* if there exists a PPT simulator $\mathcal{S}$ such that for every triple $(\mathsf{R}, \mathsf{x}, \mathsf{w}) \in \mathcal{R}$, and every algorithm $\mathcal{V}^*$, the following random variables are within ϵ statistical distance

$$\left\{\Big(\mathsf{View}(\mathcal{P}(\mathsf{R}, \mathsf{r}, \mathsf{x}, \mathsf{w}), \mathcal{V}^*)\Big) \;\middle|\; \mathsf{r} \leftarrow \mathsf{Rand}\right\} \approx_\epsilon \mathsf{View}(\mathcal{S}^{\mathcal{V}^*}(\mathsf{x}))$$

where $\mathsf{View}(\mathcal{P}(\mathsf{R}, \mathsf{r}, \mathsf{x}, \mathsf{w}), \mathcal{V}^*)$ consists of $\mathcal{V}^*$'s randomness, $\mathcal{P}$'s messages $\pi_1, \dots, \pi_{\mathsf{m}^*}$ (which do not include the oracles) and $\mathcal{V}^*$'s list of checks, while $\mathsf{View}(\mathcal{S}^{\mathcal{V}^*}(\mathsf{x}))$ consists of $\mathcal{V}^*$'s randomness followed by $\mathcal{S}$'s output, obtained after having straightline access to $\mathcal{V}^*$, and $\mathcal{V}^*$'s list of checks. Notice that the simulator does not receive the relation R as input.

The following definition captures the fact that zero-knowledge should hold even when the verifier has access to a bounded amount of evaluations of the polynomials that contain information about the witness w or the relation R.

Definition 16 ((b, C)-index-privacy)**.** We say rPHP is (b, C)-*index-private*, where $\mathsf{b} \in \mathbb{N}^{\mathsf{n}^*}$ and n^* is the number of polynomials output by the relation encoder and the prover, if for every triple $(\mathsf{R}, \mathsf{x}, \mathsf{w}) \in \mathcal{R}$ and every (b, C)-bounded list $\mathcal{L}$ (see Definition 10), the following random variables are within ϵ statistical distance:

$$\left\{\Big(\mathsf{View}(\mathcal{P}(\mathsf{R}, \mathsf{r}, \mathsf{x}, \mathsf{w}), \mathcal{V}^{\mathcal{I}(\mathsf{R},\mathsf{r})}(\mathsf{x})), (p_i(y))_{(i,y)\in\mathcal{L}}\Big) \;\middle|\; \mathsf{r} \leftarrow \mathsf{Rand}\right\} \approx_\epsilon \mathcal{S}(\mathsf{x}, \mathcal{V}(\mathsf{x}), \mathcal{L})$$

where $p_1, \dots p_{\mathsf{n}^*}$ are the polynomials returned by $\mathcal{I}(\mathsf{R}, \mathsf{r})$ and the prover $\mathcal{P}$. Notice again that the simulator does not receive the relation R as input.

Moreover, we say that rPHP is *honest-verifier index-private with query bound* $\mathsf{b} \in \mathbb{N}^{\mathsf{n}^*}$ if there exists a PT algorithm C such that rPHP is (b, C)-index-private and for all $i \in [\mathsf{n}^*]$ we have $\Pr[\mathsf{C}(i, y) = 0] \in \mathsf{negl}(\lambda)$ where y is uniformly sampled from $\mathbb{F}$.

As in [CFF+21], we will assume our rPHPs to be *public coin*, which means that each verifier message ρ_i for $i = 1, \dots, \mathsf{rnd}(|\mathsf{R}|)$ is a random element over a prescribed set, and so is an additional value $\rho_{\mathsf{rnd}(|\mathsf{R}|)+1}$ possibly used by the verifier to generate the final checks. Moreover, we will also assume our rPHPs to be *non-adaptive*, which means that the verifier's checks can be fully determined from its inputs and randomness, and thus, are independent of the prover's messages. This means that the verifier $\mathcal{V}(\mathsf{x})$ can be written as the combination of

two prover-independent algorithms: a probabilistic sampler $S_{\mathcal{V}} \to \boldsymbol{\rho} := (\rho_1, \ldots, \rho_{\mathsf{rnd}(|\mathsf{R}|)+1})$ and a deterministic algorithm $D_{\mathcal{V}}(\mathsf{x}, \boldsymbol{\rho}) \to (\boldsymbol{d} \cup \{(G_j, \boldsymbol{v}_j)\}_{j \in [\mathsf{n_e}]})$.

Finally, exactly as in [CFF+21], we can define the relation $\mathcal{R}_{\mathsf{php}}$ that models the rPHP verifier's decision phase, which essentially consists of some polynomial equations and degree checks. We refer to [CFF+21] for the details.

4 Function-Hiding Functional Commitments

In this section, we introduce the formal definition of *function-hiding functional commitment schemes*. After that, we recall the notion of *functional set* [BNO21].

Definition 17 (Function-hiding functional commitment)**.** A *function-hiding functional commitment* (FHFC) for a family of functions $\mathcal{F}$ is a tuple $(\mathsf{CS}, \mathsf{CP}_{\mathsf{FC}})$ where CS is a hiding and binding commitment scheme with message space $\mathcal{M} \supseteq \mathcal{F}$, and $\mathsf{CP}_{\mathsf{FC}}$ is a zkSNARK for the relation

$$\hat{\mathcal{R}}_{\mathsf{FHFC}} = \{((\boldsymbol{c}, x, y), (f, \boldsymbol{o})) \mid f \in \mathcal{F} \ \wedge \ f(x) = y \ \wedge \ \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, f, \boldsymbol{o}) = 1\}.$$

We are giving a slightly more restrictive definition compared to the one from [BNO21]. Namely, we are requiring that the evaluation protocol is non-interactive and succinct, which are properties motivated by the applications of this primitive.

The fact that most proof systems are built for relations and not functions naturally leads to the following definition.

Definition 18 (Functional set)**.** Let $\mathcal{R}$ be a universal relation with instance domain $\mathcal{D}_{\mathsf{ins}} = \mathcal{D}_{\mathsf{in}} \times \mathcal{D}_{\mathsf{out}}$ and relation domain $\mathcal{D}_{\mathsf{rel}}$. We say that $\mathcal{D}_{\mathsf{func}} \subseteq \mathcal{D}_{\mathsf{rel}}$ is a *functional set* if for all $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}$ and for all $x \in \mathcal{D}_{\mathsf{in}}$, there exists a unique $y \in \mathcal{D}_{\mathsf{out}}$ such that $(x, y) \in \mathcal{L}(\mathsf{R})$. Furthermore, $\mathcal{D}_{\mathsf{func}}$ must be equipped with a PT algorithm $\mathsf{Extend}(\mathsf{R}, x) \to (y, \mathsf{w})$ such that $((x, y), \mathsf{w}) \in \mathsf{R}$, for all $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}$ and $x \in \mathcal{D}_{\mathsf{in}}$.

Given a functional set $\mathcal{D}_{\mathsf{func}}$ for a universal relation $\mathcal{R}$, for any $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}$ we can define a function $\varphi_{\mathsf{R}} : \mathcal{D}_{\mathsf{in}} \to \mathcal{D}_{\mathsf{out}}$ as $\varphi_{\mathsf{R}}(x) = y$, where $y \in \mathcal{D}_{\mathsf{out}}$ is such that $\mathsf{Extend}(\mathsf{R}, x) \to (y, \mathsf{w})$. It is well defined since the output y is unique. Therefore, the relation $\hat{\mathcal{R}}_{\mathsf{FHFC}}$ from Definition 17 adapted to functional sets is:

$$\hat{\mathcal{R}}_{\mathsf{FHFC}} = \{((\boldsymbol{c}, x, y), (\mathsf{R}, \boldsymbol{o})) \mid \mathsf{R} \in \mathcal{D}_{\mathsf{func}} \ \wedge \ (x, y) \in \mathcal{L}(\mathsf{R}) \ \wedge \ \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}, \boldsymbol{o}) = 1\}. \quad (1)$$

5 Building Function-Hiding Functional Commitments

As the relation $\hat{\mathcal{R}}_{\mathsf{FHFC}}$ we defined in (1) is given in a commit-and-prove manner, a natural approach for building a FHFC would be to design a PHP for the underlying relation $\mathcal{R}_{\mathsf{FHFC}} = \{((x, y), \mathsf{R}) \mid \mathsf{R} \in \mathcal{D}_{\mathsf{func}} \ \wedge \ (x, y) \in \mathcal{L}(\mathsf{R})\}$ and invoke the commit-and-prove compiler from [CFF+21]. While correct, we argue that this approach is not satisfactory. By building a PHP for $\mathcal{R}_{\mathsf{FHFC}}$, we cannot detach proving that R is in the functional set from proving that (x, y) are in the language defined by a relation. This has efficiency implications: we must prove $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}$ on every execution of the protocol, whereas we would like this step to be only proven once.

In this section, we present a modular compiler for function-hiding functional commitments that lowers the zero-knowledge requirements relative to the one from [BNO21]. Inspired by the compilers from [CFF+21], our building blocks will be a PHP with randomized indexer rPHP and a commitment scheme CS equipped with CP-SNARKs $\mathsf{CP}_{\mathsf{opn}}$, $\mathsf{CP}_{\mathsf{php}}$, $\mathsf{CP}_{\mathsf{PFR}}$ and $\mathsf{CP}_{\mathsf{link}}$ that we define next.

Definition 19 (Compiling commitment scheme)**.** We say CS is a *compiling commitment scheme* if it is a type-based commitment scheme for a class of types $\mathcal{T} = \{\mathtt{swh}, \mathtt{hid}\}$ such that commitments of the type $\mathtt{swh}$ are for messages in $\mathbb{F}^{\leq d}[X]$ for some $d \in \mathbb{N}$ and are somewhat hiding, and commitments of the type $\mathtt{hid}$ are for messages in $\mathcal{D}_{\mathsf{func}}$ and are fully hiding.

First, we define exactly as in [CFF+21] the *opening relation* $\mathcal{R}_{\mathsf{opn}} = \{(\bot, \boldsymbol{p}) \mid \boldsymbol{p} \in \mathbb{F}^k[X],\ k \geq 1\}$. A CP-SNARK for this relation, which we will denote by $\mathsf{CP}_{\mathsf{opn}}$, is a *proof of knowledge* of the openings of the commitments to the polynomials that form the vector $\boldsymbol{p}$, hence the name.

We will also need $\mathsf{CP}_{\mathsf{php}}$, a CP-SNARK for $\mathcal{R}_{\mathsf{php}}$, the relation that models the decision phase of an rPHP. This proof certifies that, the polynomial equations and degree checks of the decision phase of an rPHP hold, which implies that the rPHP verifier would accept.

We slightly modify the definition of the *Proof of Function Relation* (PFR) from [BNO21]. In their construction, the PFR simultaneously certifies that a relation R belongs to the functional set $\mathcal{D}_{\mathsf{func}}$ and that certain polynomials are the output of a (deterministic) indexer on input R. In contrast, our PFR is responsible *only* for certifying that the committed relation R defines a function, namely that $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}$. The consistency between R and the polynomials used in the proof system is enforced separately through a dedicated *linking relation* (Definition 21). This separation is crucial in our setting: as we explain below (see Remark 1), the PFR is a one-time proof associated to a commitment to R, whereas the linking proof may be executed repeatedly in evaluations.

Definition 20 (Proof of Function Relation)**.** Let $\mathcal{D}_{\mathsf{func}}$ be a functional set for a universal relation $\mathcal{R}$. We define the *function relation* for a functional set $\mathcal{D}_{\mathsf{func}}$ as

$$\mathcal{R}_{\mathsf{func}} = \{(\bot, \mathsf{R}) \mid \mathsf{R} \in \mathcal{D}_{\mathsf{func}}\}.$$

A *Proof of Function Relation* (PFR) is a CP-SNARK for $\mathcal{R}_{\mathsf{func}}$, denoted by $\mathsf{CP}_{\mathsf{PFR}}$.

Next, we define $\mathsf{CP}_{\mathsf{link}}$, a CP-SNARK for the *linking relation*, which enforces that a vector of polynomials $\boldsymbol{p}$ is the output of the randomized indexer of an rPHP applied to a relation R with randomness r.

Definition 21 (Linking relation)**.** Let rPHP be an rPHP. We define the *linking relation* as

$$\mathcal{R}_{\mathsf{link}} = \{(\bot, (\mathsf{R}, \boldsymbol{p}, \mathsf{r})) \mid \mathcal{I}(\mathsf{R}, \mathsf{r}) = \boldsymbol{p}\}$$

Let us give a high-level description of our function-hiding functional commitment construction. The full protocol is in Figure 1.

The prover starts by showing that the committed relation R is in the functional set with $\mathsf{CP}_{\mathsf{PFR}}$. Then, it will sample randomness $\mathsf{r} \leftarrow \mathsf{Rand}$, run the randomized indexer, commit to a vector of polynomials $\boldsymbol{p}_0$, prove it knows their openings, and use $\mathsf{CP}_{\mathsf{link}}$ to show that $\boldsymbol{p}_0 = \mathcal{I}(\mathsf{R}, \mathsf{r})$. From here, the prover will commit to the polynomials from the rPHP prover, prove it knows their openings, and propagate its messages. In the last round, it uses $\mathsf{CP}_{\mathsf{php}}$ to demonstrate that the rPHP verifier would accept.

Until the last round, the verifier simply propagates the messages from the rPHP verifier. At the last round, it obtains the checks from the decision phase of the rPHP verifier and checks all of the proofs sent by the prover.

Finally, to remove the interaction one can apply the Fiat-Shamir transform.

Remark 1. Note that our definition of (b, C)-index-privacy (see Definition 16) says that we can only reveal a bounded amount of evaluations of the polynomials output by the randomized relation encoder. Since, in general, those evaluations could be different across several executions of the protocol[1], we could end up exceeding the given bound if reusing

[1] As we will see in the following sections, in some concrete instantiations this does not need to be the case, leading to more efficient constructions.

$\mathsf{KeyGen}(1^\lambda, \mathsf{ck}) \to (\mathsf{ek}, \mathsf{vk})$

$(\mathsf{ek}_{\mathsf{opn}}, \mathsf{vk}_{\mathsf{opn}}) \leftarrow \mathsf{KeyGen}_{\mathsf{opn}}(\mathsf{ck})$
$(\mathsf{ek}_{\mathsf{php}}, \mathsf{vk}_{\mathsf{php}}) \leftarrow \mathsf{KeyGen}_{\mathsf{php}}(\mathsf{ck})$
$(\mathsf{ek}_{\mathsf{link}}, \mathsf{vk}_{\mathsf{link}}) \leftarrow \mathsf{KeyGen}_{\mathsf{link}}(\mathsf{ck})$
$(\mathsf{ek}_{\mathsf{PFR}}, \mathsf{vk}_{\mathsf{PFR}}) \leftarrow \mathsf{KeyGen}_{\mathsf{PFR}}(\mathsf{ck})$
Output $\mathsf{ek} := (\mathsf{ck}, \mathsf{ek}_{\mathsf{php}}, \mathsf{ek}_{\mathsf{opn}}, \mathsf{ek}_{\mathsf{link}}, \mathsf{ek}_{\mathsf{PFR}})$ and $\mathsf{vk} := (\mathsf{vk}_{\mathsf{php}}, \mathsf{vk}_{\mathsf{opn}}, \mathsf{vk}_{\mathsf{link}}, \mathsf{vk}_{\mathsf{PFR}})$

$\mathbb{P}(\mathsf{ek}, (\boldsymbol{c}, x, y, \mathsf{R}, \boldsymbol{o}, \bar{\rho}_1 \ldots, \bar{\rho}_i) \to \bar{\pi}_i$

Round $i = 0$: // Send the PFR and new index polynomials; prove the link

Compute $\pi_{\mathsf{PFR}} \leftarrow \mathsf{Prove}_{\mathsf{PFR}}(\mathsf{ek}_{\mathsf{PFR}}, \boldsymbol{c}, \mathsf{R}, \boldsymbol{o})$ and extend witness $(y, \mathsf{w}) \leftarrow \mathsf{Extend}(\mathsf{R}, x)$
Sample $\mathsf{r} \leftarrow \mathsf{Rand}$ and compute $\boldsymbol{p}_0 \leftarrow \mathcal{I}(\mathsf{R}, \mathsf{r})$
Compute $(\boldsymbol{c}_0, \boldsymbol{o}_0) \leftarrow \mathsf{Commit}(\mathsf{ck}, \mathtt{swh}, \boldsymbol{p}_0)$ and $\pi_{\mathsf{opn},0} \leftarrow \mathsf{Prove}_{\mathsf{opn}}(\mathsf{ek}_{\mathsf{opn}}, \boldsymbol{c}_0, \boldsymbol{o_0})$
Compute $\pi_{\mathsf{link}} \leftarrow \mathsf{Prove}_{\mathsf{link}}(\mathsf{ek}_{\mathsf{link}}, \boldsymbol{c}, \boldsymbol{c}_0, \mathsf{R}, \boldsymbol{p}_0, \boldsymbol{o}, \boldsymbol{o}_0, \mathsf{r})$
Send $\bar{\pi}_0 := (\pi_{\mathsf{PFR}}, \boldsymbol{c}_0, \pi_{\mathsf{opn},0}, \pi_{\mathsf{link}})$

Let $\mathsf{rnd} := \mathsf{rnd}(|\mathsf{R}|)$
Round $1 \leq i \leq \mathsf{rnd}$: // Send the polynomials and messages from the rPHP prover

Run $(\boldsymbol{p}_i, \boldsymbol{\pi}_i) \leftarrow \mathcal{P}(\mathsf{R}, \mathsf{r}, x, y, \mathsf{w}, \bar{\rho}_1, \ldots, \bar{\rho}_i)$
Compute $(\boldsymbol{c}_i, \boldsymbol{o}_i) \leftarrow \mathsf{Commit}(\mathsf{ck}, \mathtt{swh}, \boldsymbol{p}_i)$ and $\pi_{\mathsf{opn},i} \leftarrow \mathsf{Prove}_{\mathsf{opn}}(\mathsf{ek}_{\mathsf{opn}}, \boldsymbol{c}_i, \boldsymbol{o}_i)$
Send $\bar{\pi}_i := (\boldsymbol{c}_i, \boldsymbol{\pi}_i, \pi_{\mathsf{opn},i})$

Round $i = \mathsf{rnd} + 1$: // Get checks from the rPHP verifier and prove it would accept

Retrieve checks $((d_j)_{j \in [\mathsf{n_p}]}, (G_j, \boldsymbol{v}_j)_{j \in [\mathsf{n_e}]}) \leftarrow D\mathcal{V}(x, y, \rho_1, \ldots, \rho_{\mathsf{rnd}+1})$
Construct instance $\hat{\mathsf{x}}_{\mathsf{php}} := ((d_j)_{j \in [\mathsf{n_p}]}, (G'_j, \boldsymbol{v}_j)_{j \in [\mathsf{n_e}]})$, where G'_k partially evaluates G_k:
$G'_k(X, (X_j)_{j \in [\mathsf{n}^*]}) := G_k(X, (X_j)_{j \in [\mathsf{n}^*]}, (\boldsymbol{\pi}_1 || \ldots || \boldsymbol{\pi}_{\mathsf{rnd}}))$
Construct witness $\hat{\mathsf{w}}_{\mathsf{php}} := ((\boldsymbol{p}_0 || \boldsymbol{p}_1 || \ldots || \boldsymbol{p}_{\mathsf{rnd}}), (\boldsymbol{o}_0 || \boldsymbol{o}_1 || \ldots || \boldsymbol{o}_{\mathsf{rnd}}))$
Compute $\pi_{\mathsf{php}} \leftarrow \mathsf{Prove}_{\mathsf{php}}(\mathsf{ek}_{\mathsf{php}}, \hat{\mathsf{x}}_{\mathsf{php}}, \boldsymbol{c}_0 || \boldsymbol{c}_1 || \ldots || \boldsymbol{c}_{\mathsf{rnd}}), \hat{\mathsf{w}}_{\mathsf{php}})$
Send $\bar{\pi}_{\mathsf{rnd}+1} := \pi_{\mathsf{php}}$

$\mathbb{V}(\mathsf{vk}, (\boldsymbol{c}, x, y), \bar{\pi}_0, \bar{\pi}_1, \ldots, \bar{\pi}_i) \to \bar{\rho}_i$

Round $1 \leq i \leq \mathsf{rnd}$: // Send the messages from the rPHP verifier

$\rho_i \leftarrow \mathcal{V}(x, y, \boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_{i-1})$
$\bar{\rho}_i := \rho_i$

Round $i = \mathsf{rnd} + 1$: // Perform final checks

Reconstruct $\hat{\mathsf{x}}_{\mathsf{php}}$ via $D\mathcal{V}$ and the partial evaluation of G_k (as the prover) and compute
$b_{\mathsf{PFR}} \leftarrow \mathsf{Verify}_{\mathsf{PFR}}(\mathsf{vk}_{\mathsf{PFR}}, \boldsymbol{c}, \pi_{\mathsf{PFR}})$
$b_{\mathsf{link}} \leftarrow \mathsf{Verify}_{\mathsf{link}}(\mathsf{vk}_{\mathsf{link}}, \boldsymbol{c}, \boldsymbol{c}_0, \pi_{\mathsf{link}})$
$b_{\mathsf{php}} \leftarrow \mathsf{Verify}_{\mathsf{php}}(\mathsf{vk}_{\mathsf{php}}, \hat{\mathsf{x}}_{\mathsf{php}}, (\boldsymbol{c}_0 || \ldots || \boldsymbol{c}_{\mathsf{rnd}}), \pi_{\mathsf{php}})$
$b^i \leftarrow \mathsf{Verify}_{\mathsf{opn}}(\mathsf{vk}_{\mathsf{opn}}, \boldsymbol{c}_i, \pi_{\mathsf{opn},i})$ for all $i \in [0, \mathsf{rnd}]$
Accept if and only if all checks are true

Figure 1: Compiler from rPHPs to function-hiding functional commitments.

those polynomials. This is the reason why, to start the protocol, the prover needs to sample fresh randomness $\mathsf{r} \leftarrow \mathsf{Rand}$, send commitments to the new index polynomials $\boldsymbol{p} = \mathcal{I}(\mathsf{R}, \mathsf{r})$, and prove their link with the relation R. Notice however that this is not the case for the PFR, which only needs to be computed once and is not a recurring cost in the protocol.

Theorem 1. *Let* $\mathsf{rPHP} = (\mathsf{rnd}, \mathsf{n}, \mathsf{m}, \mathsf{d}, \mathsf{n_e}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ *be a non-adaptive public coin sound rPHP for a universal relation* $\mathcal{R}$ *and let* $\mathcal{D}_{\mathsf{func}}$ *be a functional set for* $\mathcal{R}$*. Let* CS *be a compiling commitment scheme equipped with CP-SNARKs* $\mathsf{CP_{opn}}$ *for* $\mathcal{R}_{\mathsf{opn}}$*,* $\mathsf{CP_{php}}$ *for* $\mathcal{R}_{\mathsf{php}}$*,* $\mathsf{CP_{link}}$ *for* $\mathcal{R}_{\mathsf{link}}$ *and* $\mathsf{CP_{PFR}}$ *for* $\mathcal{R}_{\mathsf{func}}$*. Suppose that for a checker* C*,* rPHP *(respectively* $\mathsf{CP_{php}}$*) is* $(\mathsf{b} + \mathbf{1}, \mathsf{C})$[2]*-bounded honest-verifier index-private (resp. trapdoor-commitment* (b, C)*-leaky zero-knowledge) and* $\mathsf{CP_{opn}}, \mathsf{CP_{link}}$ *and* $\mathsf{CP_{PFR}}$ *are trapdoor-commitment zero-knowledge. Then, the protocol from Figure 1 is a trapdoor-commitment zero-knowledge succinct interactive*[3] *argument of knowledge for the relation*

$$\hat{\mathcal{R}}_{\mathsf{FHFC}} = \{((\boldsymbol{c}, x, y), (\mathsf{R}, \boldsymbol{o})) \mid \mathsf{R} \in \mathcal{D}_{\mathsf{func}} \ \wedge \ (x, y) \in \mathcal{L}(\mathsf{R}) \ \wedge \ \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}, \boldsymbol{o}) = 1\}.$$

Notice the similarities between our construction and the compilers from [CFF+21]. A key difference is that, in our case, the witness is R, whereas for them it is w, the witness of the relation R. This implies that the knowledge-soundness proof will need to be modified. However, this does not affect the proof of zero-knowledge: the only changes are that our simulator must additionally simulate the commitments to the polynomials output by the randomized relation encoder (which we can, thanks to the $+\mathbf{1}$ in the index-privacy bound of the rPHP), invoke the simulators for the CP-SNARKs $\mathsf{CP_{link}}$ and $\mathsf{CP_{PFR}}$, and use our notion of (b, C)-index-privacy for the rPHP instead of the regular (b, C)-zero-knowledge. For this reason, we will omit the proof of zero-knowledge and refer to [CFF+20]. Our formal proof of knowledge-soundness is in Appendix A.2, next we give some intuition.

Let $\mathcal{A}$ be a malicious adversary that makes the verifier accept. To extract the relation R we can simply use the extractor $\mathcal{E}_{\mathsf{PFR}}$ for the PFR. Since the verifier accepts the proof π_{PFR}, we have that R is in the functional set and the commitment $\boldsymbol{c}$ opens to R thanks to the knowledge-soundness of $\mathsf{CP_{PFR}}$. As the verifier also accepts the proof π_{link}, we deduce that the commitment $\boldsymbol{c}_0$ opens to a vector of polynomials $\boldsymbol{p}_0$ that is the output of $\mathcal{I}(\mathsf{R}, \mathsf{r})$ for some $\mathsf{r} \in \mathsf{Rand}$ thanks to the knowledge-soundness of $\mathsf{CP_{link}}$. Moreover, we can also extract from the proof π_{php} the polynomials that make the rPHP verifier which has oracle access to $\mathcal{I}(\mathsf{R}, \mathsf{r})$ accept. Thanks to the binding property of the commitment scheme, all of the values extracted so far have to be consistent with each other (that is, the extracted relation from π_{PFR} has to be equal to the one extracted from π_{link}, same for all the polynomials involved in the protocol). From here, we can define a malicious rPHP prover that makes the rPHP verifier accept, and thanks to the soundness of rPHP, we must have $(x, y) \in \mathcal{L}(\mathsf{R})$.

6 FHFCs from Marlin

In this section, we present our Marlin-based function-hiding functional commitment scheme [CHM+20]. We begin by introducing a small modification to the constraint system, which will let us define a simpler functional set. Afterwards, we present our rPHP for this constraint system. Finally, we describe our commitment scheme and its CP-SNARKs to instantiate the compiler from Theorem 1.

[2] We denote by $\mathbf{1} \in \mathbb{F}^{\mathsf{n}^*}$ the vector that is equal to 1 on each coordinate.

[3] The formal definition of interactive arguments is almost identical to that of non-interactive ones. For more details, see Appendix A.2.

6.1 Restricted Output-Final R1CS and its Functional Set

We start by recalling the variant of R1CS given in [BNO21], which makes explicit that the instance (x, y) is an input-output pair, and the output coordinates are at the end of the vector $z = (x, w, y)$. For a matrix M, we denote by $||M||$ the number of its non-zero elements.

Definition 22 (Output-final R1CS)**.** Let $n, m, t, s \in \mathbb{N}$ such that $s + t \leq n$, $m = O(n)$. The *output-final R1CS universal relation* $\mathcal{R}_{R1CS-f}(n, m, t, s)$ is defined as

$$\left\{ \left(\begin{array}{l} (A, B, C) \in (\mathbb{F}^{n\times n})^3 \\ ((x, y), w) \in (\mathbb{F}^t \times \mathbb{F}^s) \times \mathbb{F}^{n-t-s} \end{array} \right) \middle| \begin{array}{l} Az \circ Bz = Cz \\ z = (x, w, y) \\ \max\{||A||, ||B||, ||C||\} \leq m \end{array} \right\}.$$

Before giving a functional set for $\mathcal{R}_{R1CS-f}$ we need to recall the following notions introduced in [BNO21].

Definition 23. Let $M \in \mathbb{F}^{n\times n}$ be a matrix and $t \leq n$.

- We say that M is *t-strictly lower triangular* if M is strictly lower triangular and its first t rows are zero.
- We say that M is *t-diagonal* if M is diagonal, the first t entries along the diagonal are zero, and the last $n - t$ entries are nonzero.

Theorem 2 (Functional set for $\mathcal{R}_{R1CS-f}$, [BNO21])**.** *The set of triples $(A, B, C) \in (\mathbb{F}^{n\times n})^3$ such that A, B are t-strictly lower triangular and C is t-diagonal is a functional set for $\mathcal{R}_{R1CS-f}(n, t, s)$.*

We observe that, since C is t-diagonal, the R1CS equation can be written as $Az \circ Bz = c \circ z$, where $c \in \mathbb{F}^n$ is the diagonal of C. The next lemma shows that we can fix the vector c to be $c_i = 0$ for $i \in [t]$ and $c_i = 1$ for $i \in [t + 1, n]$ and thus, we can restrict ourselves to a simpler constraint system while retaining the same expressiveness.

Lemma 2. *Suppose $A, B \in \mathbb{F}^{n\times n}$ are t-strictly lower triangular and let $c = (0, \ldots, 0, 1 \ldots, 1) \in \mathbb{F}^n$. Then, there exists a strictly t-lower triangular matrix $A' \in \mathbb{F}^{n\times n}$ such that $Az \circ Bz = c \circ z$ if and only if $A'z \circ Bz = c \circ z$ for any $z \in \mathbb{F}^n$.*

Proof. For each $i \geq t + 1$, we have that $\langle A_i, z\rangle\langle B_i, z\rangle = c_i z_i$, where A_i is the i-th row of A and B_i the i-th row of B. Since $c_i \neq 0$, we can also write this as $z_i = c_i^{-1}\langle A_i, z\rangle\langle B_i, z\rangle = \langle c_i^{-1}A_i, z\rangle\langle B_i, z\rangle$. Letting $A'_i = c_i^{-1}A_i$, and A' be the matrix with the first t rows equal to 0 and the rest equal to A'_i, we have that $A'z \circ Bz = c \circ z$. It is evident that A' is strictly t-lower triangular if and only if A is strictly t-lower triangular. □

We formalize this new constraint system in the following definition. Moreover, we will change the sparsity parameter m. Looking ahead, this is because we will use a joint encoding of the matrices A and B instead of a separate encoding.

Definition 24 (Restricted output-final R1CS)**.** Let $n, m, t, s \in \mathbb{N}$ such that $s + t \leq n$, $m = O(n)$. Let $c = (0, \ldots, 0, 1, \ldots, 1) \in \mathbb{F}^n$ be the vector that is zero in its first t positions and 1 in the rest. The *restricted output-final R1CS* universal relation $\mathcal{R}_{R1CS-rf}(n, m, t, s)$ is defined as

$$\left\{ \left(\begin{array}{l} (A, B) \in (\mathbb{F}^{n\times n})^2 \\ ((x, y), w) \in (\mathbb{F}^t \times \mathbb{F}^s) \times \mathbb{F}^{n-t-s} \end{array} \right) \middle| \begin{array}{l} Az \circ Bz = c \circ z \\ z = (x, w, y) \\ ||A|| + ||B|| \leq m \end{array} \right\}.$$

Lemma 2 immediately implies that we can define the following, simpler functional set for $\mathcal{R}_{R1CS-rf}(n, m, t, s)$:

$$\mathcal{D}_{\mathsf{func}}(n, m, t, s) = \left\{ (A, B) \in (\mathbb{F}^{n\times n})^2 \middle| \begin{array}{l} A, B \text{ are } t\text{-strictly lower triangular} \\ ||A|| + ||B|| \leq m \end{array} \right\}. \quad (2)$$

6.2 Our rPHP for Restricted Output-Final R1CS

In this section we describe our rPHP for the restricted output-final R1CS relation. We begin by introducing the randomized encoder $\mathcal{I}$ and then we present the necessary changes to make the Marlin PHP[4] [CHM+20] work with our constraint system and to achieve (b, C)-index-privacy.

6.2.1 Offline Phase of our rPHP

Essentially, our randomized encoder will work as the deterministic one from [CHM+20], except it will add blinding terms to the polynomials that leave their values over a subgroup unchanged. Moreover, to achieve better efficiency, we will incorporate optimizations from followup works [COS20, CFF+21] and the reference implementation [ac22]. So, we start by recalling the deterministic joint encoding of matrices $(A, B) \in \mathcal{D}_{\mathsf{func}}(n, m, t, s)$.

Let $S = \{(r, c) \in [n] \times [n] \mid A_{r,c} \neq 0 \ \vee B_{r,c} \neq 0\}$ be the set of indices where either A or B are nonzero. Let $\mathbb{K} = \langle \kappa \rangle$ be the minimal-size multiplicative subgroup of $\mathbb{F}$ such that $|\mathbb{K}| \geq |S|$. Since $|S| \leq ||A|| + ||B|| \leq m$, we can assume without loss of generality that $|\mathbb{K}| = m$. Let $\mathbb{H} = \langle \omega \rangle$ be another subgroup of order n. We assume that $|\mathbb{H}|/|\mathbb{F}|$ and $|\mathbb{K}|/|\mathbb{F}|$ are negligible in the security parameter. For $(r_i, c_i) \in S$ and $M \in \{A, B\}$, define

$$\widehat{\mathsf{row}}(X) = \sum_{i=1}^{m} \omega^{r_i} \lambda_i^{\mathbb{K}}(X), \qquad \widehat{\mathsf{col}}(X) = \sum_{i=1}^{m} \omega^{c_i} \lambda_i^{\mathbb{K}}(X)$$

$$\widehat{\mathsf{val}}_M(X) = \sum_{i=1}^{m} \frac{M_{\widehat{\mathsf{row}}(\kappa^i), \widehat{\mathsf{col}}(\kappa^i)}}{u_{\mathbb{H}}(\mathsf{row}(\kappa^i), \mathsf{row}(\kappa^i)) u_{\mathbb{H}}(\mathsf{col}(\kappa^i), \mathsf{col}(\kappa^i))} \lambda_i^{\mathbb{K}}(X).$$

The polynomial $\widehat{\mathsf{val}}_M$ is chosen this way so that the low degree extension of $M \in \{A, B\}$ can be written as

$$\hat{M}(X, Y) = \sum_{i=1}^{m} \frac{z_{\mathbb{H}}(X)}{X - \widehat{\mathsf{row}}(\kappa^i)} \frac{z_{\mathbb{H}}(Y)}{Y - \widehat{\mathsf{col}}(\kappa^i)} \widehat{\mathsf{val}}_M(\kappa^i) = \sum_{i=1}^{m} \mathsf{val}_M(\kappa^i) \lambda^{\mathbb{H}}_{\mathsf{row}(\kappa^i)}(X) \lambda^{\mathbb{H}}_{\mathsf{col}(\kappa^i)}(Y).$$

It was shown in [COS20] that, instead of using the encoding described above, a more efficient PHP can be achieved by using the encoding of the pair[5] (A^*, B^*), where $M^*_{a,b} = u_{\mathbb{H}}(b, b) M_{b,a}$ for $a, b \in \mathbb{H}$ and $u_{\mathbb{H}}(X, Y)$ is the formal derivative of the vanishing polynomial over $\mathbb{H}$. We will also incorporate these ideas into our rPHP.

The randomized encoder will simply output masked versions of the polynomials encoding the low degree extensions of the matrices A^* and B^* we just described. The degree of the blinding factor depends on the number of evaluations that have to be revealed in the rPHP. To achieve better efficiency, we also use the polynomial rowcol, which is already present in the reference implementation of Marlin [ac22] and in some variants of [CFF+21]. With these optimizations in place, we specify the randomized indexer $\mathcal{I}(\mathsf{R}, \mathsf{r})$ as follows.

The randomness space. Let $\mathsf{b}_{\mathsf{row}}, \mathsf{b}_{\mathsf{col}}, \mathsf{b}_{\mathsf{rowcol}}, \mathsf{b}_{\mathsf{val}_{A^*}}, \mathsf{b}_{\mathsf{val}_{B^*}}, \mathsf{b}_{h_2} \in \mathbb{N}$ and define the randomness space as

$$\begin{aligned}\mathsf{Rand} := \mathbb{F}^{\leq \mathsf{b}_{\mathsf{row}} + \mathsf{b}_{h_2} - 1}[X] \times \mathbb{F}^{\leq \mathsf{b}_{\mathsf{col}} + \mathsf{b}_{h_2} - 1}[X] \times \mathbb{F}^{\leq \mathsf{b}_{\mathsf{rowcol}} + \mathsf{b}_{h_2} - 1}[X] \\ \times \mathbb{F}^{\leq \mathsf{b}_{\mathsf{val}_{A^*}} + \mathsf{b}_{h_2} - 1}[X] \times \mathbb{F}^{\leq \mathsf{b}_{\mathsf{val}_{B^*}} + \mathsf{b}_{h_2} - 1}[X].\end{aligned}$$

For $\mathsf{r} \in \mathsf{Rand}$, write $\mathsf{r} = (r_{\mathsf{row}}(X), r_{\mathsf{col}}(X), r_{\mathsf{rowcol}}(X), r_{\mathsf{val}_{A^*}}(X), r_{\mathsf{val}_{B^*}}(X))$.

[4] Technically it is an AHP but for the sake of simplicity, from this point we identify the two concepts and just refer to them as PHPs.

[5] They also encode the matrix C and use a separate matrix encoding, but their ideas still apply here.

The randomized indexer. On input a relation $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}(n, m, t, s)$ and randomness $\mathsf{r} \leftarrow \mathsf{Rand}$, the randomized indexer $\mathcal{I}(\mathsf{R}, \mathsf{r})$ outputs the polynomials $\mathsf{row}(X)$, $\mathsf{col}(X)$, $\mathsf{rowcol}(X)$, $\mathsf{val}_{A^*}(X)$ and $\mathsf{val}_{B^*}(X)$ defined as

$$\mathsf{row}(X) := \widehat{\mathsf{row}}(X) + r_{\mathsf{row}}(X) z_{\mathbb{K}}(X) \qquad \mathsf{col}(X) := \widehat{\mathsf{col}}(X) + r_{\mathsf{col}}(X) z_{\mathbb{K}}(X)$$

$$\mathsf{val}_{A^*}(X) := \widehat{\mathsf{val}}_{A^*}(X) + r_{\mathsf{val}_{A^*}}(X) z_{\mathbb{K}}(X) \quad \mathsf{val}_{B^*}(X) := \widehat{\mathsf{val}}_{B^*}(X) + r_{\mathsf{val}_{B^*}}(X) z_{\mathbb{K}}(X)$$

$$\mathsf{rowcol}(X) := \sum_{i=1}^{m} \mathsf{row}(\kappa^i)\mathsf{col}(\kappa^i)\lambda_i^{\mathbb{K}}(X) + r_{\mathsf{rowcol}}(X) z_{\mathbb{K}}(X),$$

where $\widehat{\mathsf{row}}, \widehat{\mathsf{col}}, \widehat{\mathsf{val}}_{A^*}, \widehat{\mathsf{val}}_{B^*}$ are the polynomials that encode the pair of matrices (A^*, B^*) described above.

Remark 2. Note that the polynomial $c(X) = 1 - \sum_{i=1}^{t} \lambda_i^{\mathbb{H}}(X)$, which is the low degree extension of the vector $c = (0, \ldots, 0, 1 \ldots, 1)$, is not output by the encoder, as it can be computed efficiently by the verifier.

6.2.2 Online Phase of our rPHP

We first give an overview of the online phase of the regular Marlin's PHP, which will aid in understanding the changes we need to perform. We refer to [CHM+20, COS20] for the details.

Summary of the Marlin PHP. To prove $Az \circ Bz = Cz$, the prover first sends $\hat{z}_A(X), \hat{z}_B$ and $\hat{z}_C(X)$, which are randomized low degree extensions of the vectors $z_A = Az$, $z_B = Bz$ and $z_C = Cz$, respectively. To show $z_A \circ z_B = z_C$, it is enough to perform a zero check over $\mathbb{H}$ for $\hat{z}_A(X)\hat{z}_B(X) - \hat{z}_C(X)$, that is, to send the polynomial $q(X)$ such that $\hat{z}_A(X)\hat{z}_B(X) - \hat{z}_C(X) = q(X)z_{\mathbb{H}}(X)$. It remains to prove that indeed, $z_M = Mz$ for $M \in \{A, B, C\}$, which is known as the *lincheck* protocol. To simplify our presentation, we explain it in the case where there is only one matrix involved.

To prove the linear constraint $z_M = Mz$, the prover samples a vector linearly independent polynomials $\vec{u}(X) = (u_1(X), \ldots, u_n(X)) \in \mathbb{F}^n[X]$ and observes that correctness of $z_M = Mz$ is equivalent to $\sum_j u_j(X) z_{M,j} = \sum_j (\vec{u}(X)M)_j z_j$. By the Schwartz–Zippel lemma, it suffices to verify this identity at a random challenge $\alpha \in \mathbb{F}$ chosen by the verifier, which is done via the sumcheck protocol.

To preserve zero-knowledge, the prover randomizes the sumcheck by sampling $s_1(X)$ such that $\sum_j s_1(\omega^j) = 0$. The check is expressed in terms of the bivariate low-degree extensions $u(X, Y)$ and $r_M(X, Y)$ of the vectors of polynomials $\vec{u}(X)$ and $\vec{u}(X)M$, respectively. By Lemma 1, the lincheck reduces to proving that

$$s_1(X) + u(\alpha, X)\hat{z}_M(X) - r_M(\alpha, X)\hat{z}(X) = h_1(X)z_{\mathbb{H}}(X) + Xg_1(X),$$

for some $h_1, g_1 \in \mathbb{F}[X]$ with $\deg(g_1) < |\mathbb{H}| - 1$. The verifier checks this identity at a second random point $\beta \in \mathbb{F}$, corresponding to evaluating $u(X, Y)$ and $r_M(X, Y)$ at (α, β).

By carefully choosing the vector of polynomials $\vec{u}(X)$, its low degree extension turns out to be $u(X, Y) = u_{\mathbb{H}}(X, Y)$, the formal derivative of the vanishing polynomial, which can be evaluated in logarithmic time by the verifier. For the second term, it is shown in [COS20] that $r_M(X, Y) = \hat{M}^*(Y, X)$, where $\hat{M}^*(X, Y)$ is the low-degree extension of the sparse matrix M^*,

$$\hat{M}^*(Y, X) = \sum_{i=1}^{m} \mathsf{val}_{M^*}(\kappa^i)\, \lambda^{\mathbb{H}}_{\mathsf{row}(\kappa^i)}(Y)\, \lambda^{\mathbb{H}}_{\mathsf{col}(\kappa^i)}(X).$$

Since the verifier cannot compute $r_M(\alpha, \beta)$ efficiently, the prover must send a value γ and a polynomial $t(X)$ and then show that $r_M(\alpha, X) = t(X)$ and $t(\beta) = \gamma$. This reduces to

a sumcheck over a polynomial $f(X)$ satisfying $f(\kappa^i) = \mathsf{val}_{M^*}(\kappa^i)\frac{z_{\mathbb{H}}(\beta)}{\beta-\mathsf{row}(\kappa^i)}\frac{z_{\mathbb{H}}(\alpha)}{\alpha-\mathsf{col}(\kappa^i)}$, More concretely, the prover will show that

$$f(X) = Xg_2(X) + \gamma/|\mathbb{K}|$$
$$f(X)(\alpha\beta - \alpha\mathsf{row}(X) - \beta\mathsf{col}(X) + \mathsf{rowcol}(X)) - z_{\mathbb{H}}(\alpha)z_{\mathbb{H}}(\beta)\mathsf{val}_{M^*}(X) = h_2(X)z_{\mathbb{K}}(X)$$

for some $h_2, g_2 \in \mathbb{F}[X]$ with $\deg(g) < |\mathbb{K}| - 1$. The first equation demonstrates that f sums to γ over $\mathbb{K}$, while the second shows that f indeed takes the claimed values over $\mathbb{K}$.

We proceed to describe the changes we need to conduct on the standard Marlin PHP. There are modifications related to the constraint system, which are minimal and straightforward, and modifications related to the index zero-knowledge, which are a bit more involved. There are no modifications regarding the randomized encoding (beyond raising the degree of some polynomials) as it is essentially the same as the deterministic one. The full protocol is depicted in Figure 2.

Constraint system changes. We simply need to substitute the lincheck $z_C = Cz$, with a Hadamard check $z_C = c \circ z$, which can be proven more efficiently: it reduces to showing that there exists a polynomial $q(X)$ such that $\hat{z}_C(X) - c(X)\hat{z}(X) = q(X)z_{\mathbb{H}}(X)$. To optimize our protocol, we never send $\hat{z}_C(X)$ and we replace it with $\hat{z}_A(X)\hat{z}_B(X)$ as described in [CHM+20]. Moreover, we do not need to send $q(X)$, as this zero check will be batched with other zero checks in the rPHP. As all of the linchecks are batched, certain expressions that originally involved the three matrices A, B, C will now only involve A and B.

In the standard Marlin PHP, the polynomial $\hat{z}(X)$ that encodes $z = (x, w)$ is defined as $\hat{z}(X) = \hat{w}(X)z_{\mathbb{H}[\leq t]}(X) + \hat{x}(X)$. When the verifier queries $\hat{z}(X)$ at a point $\alpha \in \mathbb{F}$, the prover answers with $\hat{w}(\alpha)$ and the verifier must evaluate itself $\hat{x}(\alpha)$ and $z_{\mathbb{H}[\leq t]}(\alpha)$ in order to enforce consistency with the input. In our case, as now $z = (x, w, y)$, we define $\hat{z}(X) = \hat{w}(X)z_{[\mathbb{H}\leq t]}(X)z_{[\mathbb{H}\geq n-s+1]}(X) + \hat{x}(X) + \hat{y}(X)$. Likewise, when $\hat{z}(\alpha)$ is queried, the prover responds with $\hat{w}(\alpha)$ and then the verifier must evaluate $\hat{x}(\alpha)$, $\hat{y}(\alpha)$, $z_{[\mathbb{H}\leq t]}(\alpha)$ and $z_{[\mathbb{H}\geq n-s+1]}(\alpha)$ to enforce consistency with the input and the output.

Index-privacy changes. The first part of the protocol remains unchanged, as checking $z_A \circ z_B = z_C$ does not involve the polynomials output by the indexer. However, the lincheck protocol is not index-private. We highlight the two points where this property is broken. First, the prover sends the polynomial $t(X) = r_M(\alpha, X)$ and its evaluation $t(\beta)$, both of which reveal information about the relation. Then, the last sumcheck of the protocol involves index polynomials and it is not masked. While the protocol from [BNO21] makes the last sumcheck zero-knowledge, it does not prevent the leakage from the first step[6].

Recall that in the original protocol, $t(X) = r_M(\alpha, X)$ and $r_M(X, Y) = \hat{M}^*(Y, X)$. The idea is to instead define $t(X)$ and $r_M(\alpha, X)$ using a randomized low degree extension of the matrix M^* so that no information about the matrix is leaked. Since the prover only reveals $\hat{M}^*(\beta, \alpha)$ but never sends the polynomial $\hat{M}^*(Y, X)$, it is enough to mask it using a random field element, i.e., sample $\sigma \leftarrow \mathbb{F}$ and define

$$r_M(X, Y) := \hat{M}^*(Y, X) + \sigma z_{\mathbb{H}}(Y) = \sum_{i=1}^{m} \mathsf{val}_{M^*}(\kappa^i)\lambda^{\mathbb{H}}_{\mathsf{row}(\kappa^i)}(Y)\lambda^{\mathbb{H}}_{\mathsf{col}(\kappa^i)}(X) + \sigma z_{\mathbb{H}}(Y).$$

[6]Technically, they work with the PHP from [CHM+20] without some of the optimizations we have described. In this variant, the polynomial $t(X)$ is defined a bit differently and the prover never sends it to the verifier, but they still need to send the evaluation $t(\beta)$, breaking index-privacy.

The same cannot be said for $t(X)$, as the prover must send the polynomial and later reveal the evaluation $t(\beta)$. So, a polynomial of at least degree 1 (we specify the concrete degree in Figure 2) must be used to mask $t(X)$. More concretely, the prover will sample a random polynomial $\rho_t(X)$ and send

$$t(X) := \sum_{i=1}^{m} \mathsf{val}_{M^*}(\kappa^i)\lambda^{\mathbb{H}}_{\mathsf{row}(\kappa^i)}(X)\lambda^{\mathbb{H}}_{\mathsf{col}(\kappa^i)}(\alpha) + \rho_t(X)z_{\mathbb{H}}(X).$$

Notice that now $t(X) \neq r_M(\alpha, X)$. However, they still take the same values over $\mathbb{H}$. Thus, in order not to break soundness we must perform a zero check over $\mathbb{H}$ to demonstrate that $t(X)$ is well defined. Namely, prover will send $q(X)$ such that $t(X) - r_M(\alpha, X) = q(X)z_{\mathbb{H}}(X)$, and this equation will be tested at a random β sent by the verifier. The final step is to convince the verifier that

$$\gamma = r_M(\alpha, \beta) = \sum_{i=1}^{m} \mathsf{val}_{M^*}(\kappa^i)\lambda^{\mathbb{H}}_{\mathsf{row}(\kappa^i)}(\beta)\lambda^{\mathbb{H}}_{\mathsf{col}(\kappa^i)}(\alpha) + \sigma z_{\mathbb{H}}(\beta).$$

The idea is to make the prover send in the first round of the protocol a random polynomial $s_2(X)$ conditioned on $\sigma = \sum_{i=1}^{m} s_2(\kappa^i)$, which will be used to blind the final sumcheck. Then, if we let $f(X)$ be a masked polynomial that takes the values $f(\kappa^i) = \frac{z_{\mathbb{H}}(\beta)}{(\beta - \mathsf{row}(\kappa^i))}\frac{z_{\mathbb{H}}(\alpha)}{(\alpha - \mathsf{col}(\kappa^i))}$ for $i \in [m]$, we have

$$\gamma = \sum_{i=1}^{m} f(\kappa^i) + s_2(\kappa^i)z_{\mathbb{H}}(\beta)$$

which can be proven using the sumcheck protocol. The last step is to show that f takes the claimed values just like in regular Marlin.

Theorem 3. *The rPHP from Figure 2 is complete, sound and index-private. Furthermore, it is honest-verifier index-private with query bound* $\mathsf{b} \in \mathbb{N}^{16}$ *given by*

$$\mathsf{b} = (\mathsf{b}_{\mathsf{row}}, \mathsf{b}_{\mathsf{col}}, \mathsf{b}_{\mathsf{rowcol}}, \mathsf{b}_{\mathsf{val}_{A^*}}, \mathsf{b}_{\mathsf{val}_{B^*}}, \mathsf{b}_{\hat{w}}, \mathsf{b}_{\hat{z}_A}, \mathsf{b}_{\hat{z}_B}, \mathsf{b}_{s_1}, \mathsf{b}_{s_2}, \mathsf{b}_t, \mathsf{b}_{g_1}, \mathsf{b}_{h_1}, \mathsf{b}_f, \mathsf{b}_{g_2}, \mathsf{b}_{h_2}).$$

Proof. Taking into account the changes we described at the beginning of this section, the completeness and soundness proofs are very similar to the original [CHM+20, COS20] and therefore omitted. Our index-privacy proof resembles the one from [CFF+20] (Theorem 2 therein), but we need to generalize it to take into account the polynomials output by the randomized indexer.

The simulator gets access to the random tape of the verifier, and only gets as input the instance of the relation (x, y). The only (non-oracle) message sent by the prover is the value γ. In the honest execution of the protocol,

$$\gamma = \Phi(\alpha, \beta) = \sum_{M \in \{A,B\}} \eta_M \hat{M}^*(\beta, \alpha) + \sigma z_{\mathbb{H}}(\beta),$$

where $\sigma \leftarrow \mathbb{F}$ is a uniformly random field element. Therefore, γ is also a uniformly random field element. Thus, the simulator will simply output $\widetilde{\gamma} \leftarrow \mathbb{F}$.

We move on to prove the (b, C)-index-privacy. We define $\mathsf{C}(i, \delta)$ as the algorithm that outputs 1 if $i \in [16]$ and $\delta \notin \mathbb{K}$. As $|\mathbb{K}|/|\mathbb{F}|$ is negligible, if $\delta \leftarrow \mathbb{F}$, then $\mathsf{C}(i, \delta) = 1$ with overwhelming probability.

For any $(i, \delta) \in \mathcal{L}$ with $i \in [16] \setminus \{13, 16\}$ (that is, for every query on each polynomial except h_1 and h_2), the simulator samples uniformly random values $p_{i,\delta} \leftarrow \mathbb{F}$ and declares them as $p_i(\delta)$. Next, it defines

$$\hat{z}_\delta = \hat{w}_\delta z_{[\mathbb{H} \leq t]}(\delta) z_{[\mathbb{H} \geq n-s+1]}(\delta) + \hat{x}(\delta) + \hat{y}(\delta)$$

$$h_{1,\delta} = \frac{s_{1,\delta} + u_{\mathbb{H}}(\alpha, \delta)\left(\sum_{M \in \{A,B\}} \eta_M \hat{z}_{M,\delta}\right) - t_\delta \hat{z}_\delta + \delta g_{1,\delta} + \eta_C(\hat{z}_{A,\delta}\hat{z}_{B,\delta} - c(\delta)\hat{z}_\delta) + \eta(t_\delta - \widetilde{\gamma})}{z_{\mathbb{H}}(\delta)}$$

$\mathcal{P}(\mathbb{F}, \mathbb{H}, \mathbb{K}, A, B, x, y, w, \mathsf{r})$ $\qquad$ $\mathcal{V}^{\mathsf{row},\mathsf{col},\mathsf{rowcol},\mathsf{val}_{A^*},\mathsf{val}_{B^*}}(\mathbb{F}, \mathbb{H}, \mathbb{K}, x, y)$

$z := (x, w, y), \quad z_A := Az, \quad z_B := Bz, \quad z_C = (0, w, y)$
Sample randomized low degree extensions $\hat{w}(X) \leftarrow \mathbb{F}^{\leq |w| + \mathsf{b}_{\hat{w}} + \mathsf{b}_{h_1} - 1}[X]$,
$\hat{z}_A(X) \leftarrow \mathbb{F}^{\leq |\mathbb{H}| + \mathsf{b}_{\hat{z}_A} + \mathsf{b}_{h_1} - 1}[X]$ and $\hat{z}_B(X) \leftarrow \mathbb{F}^{\leq |\mathbb{H}| + \mathsf{b}_{\hat{z}_B} + \mathsf{b}_{h_1} - 1}[X]$
Sample $q_{s_1}(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_{s_1} + \mathsf{b}_{h_1} - 1}[X]$, $r_{s_1}(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_{g_1} + \mathsf{b}_{h_1} - 1}[X]$
Define $s_1(X) := q_{s_1}(X) z_{\mathbb{H}}(X) + X r_{s_1}(X)$
Sample $q_{s_2}(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_{s_2} + \mathsf{b}_{h_2} - 1}[X]$, $r_{s_2}(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_{g_2} + \mathsf{b}_{h_2} - 1}[X]$ and $\sigma \leftarrow \mathbb{F}$
Define $s_2(X) := q_{s_2}(X) z_{\mathbb{K}}(X) + X r_{s_2}(X) + \sigma / |\mathbb{K}|$

$$\xrightarrow{\quad \hat{w}(X), \hat{z}_A(X), \hat{z}_B(X), s_1(X), s_2(X) \quad}$$

$\eta, \eta_A, \eta_B, \eta_C \leftarrow \mathbb{F}$
$\alpha \leftarrow \mathbb{F} \setminus \mathbb{H}$

$$\xleftarrow{\quad \eta, \eta_A, \eta_B, \eta_C, \alpha \in \mathbb{F} \quad}$$

$\Phi(X, Y) := \sum_{M \in \{A,B\}} \eta_M \hat{M}^*(Y, X) + \sigma z_{\mathbb{H}}(Y)$
Sample $\rho_t(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_t + \mathsf{b}_{h_1} - 1}[X]$ and compute $t(X) := \sum_{M \in \{A,B\}} \eta_M \widehat{M}^*(\alpha, X) + \rho_t(X) z_{\mathbb{H}}(X)$

sumcheck for $s_1(X) + u_{\mathbb{H}}(\alpha, X)(\sum_{M \in \{A,B\}} \eta_M \hat{z}_M(X)) - t(X)\hat{z}(X)$ **over** $\mathbb{H}$
and zero check for $\hat{z}_C(X) - c(X)\hat{z}(X)$ **over** $\mathbb{H}$
and zero check for $t(X) - \Phi(\alpha, X)$ **over** $\mathbb{H}$

Find $g_1(X), h_1(X)$ such that:
$s_1(X) + u_{\mathbb{H}}(\alpha, X) \left(\sum_{M \in \{A,B\}} \eta_M \hat{z}_M(X) \right) - t(X)\hat{z}(X) + X g_1(X) + \eta_C(\hat{z}_A(X)\hat{z}_B(X) - c(X)\hat{z}(X)) + \eta(t(X) - \Phi(\alpha, X)) = h_1(X) z_{\mathbb{H}}(X)$

$$\xrightarrow{\quad t(X), g_1(X), h_1(X) \quad}$$

$\beta \leftarrow \mathbb{F} \setminus \mathbb{H}$

$$\xleftarrow{\quad \beta \in \mathbb{F} \quad}$$

sumcheck for $s_2(X) z_{\mathbb{H}}(\beta) + \frac{z_{\mathbb{H}}(\alpha) z_{\mathbb{H}}(\beta)}{(\beta - \mathsf{row}(X))(\alpha - \mathsf{col}(X))} \left(\sum_{M \in \{A,B\}} \eta_M \mathsf{val}_{M^*}(X) \right)$ **over** $\mathbb{K}$

Let $\gamma := \Phi(\alpha, \beta)$. Sample $\rho_f(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_f + \mathsf{b}_{h_2} - 1}[X]$ and let
$f(X) := \sum_{i=1}^{m} \frac{z_{\mathbb{H}}(\beta) z_{\mathbb{H}}(\alpha)(\eta_A \mathsf{val}_{A^*}(\kappa^i) + \eta_B \mathsf{val}_{B^*}(\kappa^i))}{(\beta - \mathsf{row}(\kappa^i))(\alpha - \mathsf{col}(\kappa^i))} \lambda_i^{\mathbb{K}}(X) + \rho_f(X) z_{\mathbb{K}}(X)$

$$\xrightarrow{\quad f(X), \gamma \quad}$$

$\zeta \leftarrow \mathbb{F}$

$$\xleftarrow{\quad \zeta \in \mathbb{F} \quad}$$

Find $g_2(X)$ and $h_2(X)$ such that:
$h_2(X) z_{\mathbb{K}}(X) = (a(X) - b(X) f(X)) + \zeta(s_2(X) z_{\mathbb{H}}(\beta) + f(X) - X g_2(X) - \gamma / |\mathbb{K}|)$
where $a(X) := z_{\mathbb{H}}(\beta) z_{\mathbb{H}}(\alpha)(\eta_A \mathsf{val}_{A^*}(X) + \eta_B \mathsf{val}_{B^*}(X))$
$b(X) := \alpha\beta - \alpha \mathsf{row}(X) - \beta \mathsf{col}(X) + \mathsf{rowcol}(X)$

$$\xrightarrow{\quad g_2(X), h_2(X) \quad}$$

Verifier's checks

$$\deg(\hat{w}, \hat{z}_A, \hat{z}_B, s_1, s_2, t, h_1, h_2, f) \overset{?}{\leq} \mathsf{D}, \quad \deg(g_1) \overset{?}{\leq} |\mathbb{H}| - 2, \quad \deg(g_2) \overset{?}{\leq} |\mathbb{K}| - 2$$

$$s_1(\beta) + u_{\mathbb{H}}(\alpha, \beta) \Big(\sum_{M \in \{A,B\}} \eta_M \hat{z}_M(\beta) \Big) - t(\beta)\hat{z}(\beta) + \beta g_1(\beta)$$
$$+ \eta_C(\hat{z}_A(\beta)\hat{z}_B(\beta) - c(\beta)\hat{z}(\beta)) + \eta(t(\beta) - \gamma) \overset{?}{=} h_1(\beta) z_{\mathbb{H}}(\beta)$$

$$f(X)\big(\alpha\beta - \alpha \mathsf{row}(X) - \beta \mathsf{col}(X) + \mathsf{rowcol}(X)\big) - z_{\mathbb{H}}(\beta) z_{\mathbb{H}}(\alpha)(\eta_A \mathsf{val}_{A^*}(X) + \eta_B \mathsf{val}_{B^*}(X))$$
$$- \zeta(s_2(X) z_{\mathbb{H}}(\beta) + f(X) - X g_2(X) - \gamma / |\mathbb{K}|) \overset{?}{=} h_2(X) z_{\mathbb{K}}(X)$$

Figure 2: Marlin's PHP with randomized indexer. We denote by D the maximum degree of all the polynomials involved in the protocol.

and declares $h_1(\delta)$ as $h_{1,\delta}$. For $h_2(\delta)$, it first defines

$$a_\delta = z_{\mathbb{H}}(\beta) z_{\mathbb{H}}(\alpha)(\eta_A \mathsf{val}_{A^*,\delta} + \eta_B \mathsf{val}_{B^*,\delta}), \quad b_\delta = \alpha\beta - \alpha \mathsf{row}_\delta - \beta \mathsf{col}_\delta + \mathsf{rowcol}_\delta$$

$$h_{2,\delta} = \frac{b_\delta f_\delta - a_\delta + \zeta(s_{2,\delta} z_{\mathbb{H}}(\beta) + f_\delta - \delta g_{2,\delta} - \widetilde{\gamma}/|\mathbb{K}|)}{z_{\mathbb{K}}(\delta)}$$

and finally declares $h_2(\delta)$ as $h_{2,\delta}$. The simulator keeps a table of the computed values above to answer the oracle queries consistently.

We proceed to argue that the distribution of the simulator's output is identical to the one of

$$\left\{ \left(\mathsf{View}(\mathcal{P}(\mathsf{R}, \mathsf{r}, \mathsf{x}, \mathsf{w}), \mathcal{V}^{\mathcal{I}(\mathsf{R},\mathsf{r})}(\mathsf{x})), (p_i(y))_{(i,y)\in\mathcal{L}} \right) \;\middle|\; \mathsf{r} \leftarrow \mathsf{Rand} \right\}.$$

Given that $\mathsf{row}(X)$ has been masked with a random polynomial of degree $\mathsf{b}_{\mathsf{row}} + \mathsf{b}_{h_1} - 1$, we have that (at least) $\mathsf{b}_{\mathsf{row}}$ evaluations of $\mathsf{row}(X)$ are uniformly random in $\mathbb{F}$, so the simulated answers $\{\mathsf{row}_\delta\}$ are identically distributed to the ones in an honest execution of the protocol. We can use the same reasoning for $\{p_{i,\delta}\}_{(i,\delta)\in\mathcal{L}}$ for $i \in \{2, 3, 4, 6, 7, 8, 11, 14\}$, that is, for the rest of the index polynomials and for $\hat{w}, \hat{z}_A, \hat{z}_B, t$ and f, with their respective query bound.

We analyze $s_1(X)$, $g_1(X)$ and $h_1(X)$. Recall that $s_1(X) = q_{s_1}(X) z_{\mathbb{H}}(X) + X r_{s_1}(X)$. Define

$$\varphi(X) = u_{\mathbb{H}}(\alpha, X) \left(\sum_{M\in\{A,B\}} \eta_M \hat{z}_M(X) \right) - t(X)\hat{z}(X),$$

which can be decomposed as

$$\varphi(X) = h_\varphi(X) z_{\mathbb{H}}(X) + X r_\varphi(X)$$

for some unique $h_\varphi(X)$ and $r_\varphi(X)$. In particular, $s_1(X) + \varphi(X) = h(X) z_{\mathbb{H}}(X) + X g_1(X)$, where $g_1(X) = r_{s_1}(X) + r_\varphi(X)$, by the uniqueness of the previous decomposition. Since $r_1(X)$ is a random polynomial of degree $\mathsf{b}_{g_1} + \mathsf{b}_{h_1} - 1$, we have that b_{g_1} evaluations of $g_1(X)$ are uniformly random in $\mathbb{F}$, which is exactly how the simulator declared $\{g_{1,\delta}\}$. Furthermore, since $q_{s_1}(X)$ is a random polynomial of degree $\mathsf{b}_{s_1} + \mathsf{b}_{h_1} - 1$, then b_{s_1} evaluations of $s_1(X)$ are uniformly random in $\mathbb{F}$ even conditioned on $r_{s_1}(X)$, so the simulated answers $\{s_{1,\delta}\}$ are identically distributed. The values $h_{1,\delta}$ are chosen to make the verifier accept, as the polynomial $h_1(X)$ is determined by the equation

$$h_1(X) = \frac{s_1(X) + \varphi(X) + X g_1(X) + \eta_C(\hat{z}_A(X)\hat{z}_B(X) - c(X)\hat{z}(X)) + \eta(t(X) - \Phi(\alpha, X))}{z_{\mathbb{H}}(X)}.$$

As all of the polynomials involved in the expression above support b_{h_1} more evaluations, we have that the simulated answers $\{h_{1,\delta}\}$ are identically distributed to the evaluations $\{h_1(\delta)\}$ in the real protocol. To see this, recall that the honest definitions of $t(X)$ and $\Phi(\alpha, X)$ imply that $t(X) - \Phi(\alpha, X) = (\rho_t(X) - \sigma) z_{\mathbb{H}}(X)$, where $\rho_t(X)$ is a random polynomial of degree $\mathsf{b}_t + \mathsf{b}_{h_1} - 1$ and σ is a random field element.

The analysis on s_2, h_2 and g_2 is analogous. We can argue again that in the real protocol,

$$\begin{aligned} s_2(X) z_{\mathbb{H}}(\beta) + f(X) = (q_{s_2}(X) z_{\mathbb{H}}(\beta) + q_f(X)) z_{\mathbb{K}}(X) + (r_{s_2}(X) z_{\mathbb{H}}(\beta) + r_f(X)) X \\ + (\sigma z_{\mathbb{H}}(\beta) + \sigma_f)/|\mathbb{K}|), \end{aligned}$$

where now $g_2(X) = r_{s_2}(X) z_{\mathbb{H}}(\beta) + r_f(X)$ and since $r_{s_2}(X)$ is a random polynomial of degree $\mathsf{b}_{g_2} + \mathsf{b}_{h_2} - 1$, the evaluations $\{g_2(\delta)\}$ are also uniformly random in $\mathbb{F}$, which is how we defined $\{g_{2,\delta}\}$. Furthermore, since $q_{s_2}(X)$ is a random polynomial of degree $\mathsf{b}_{s_2} + \mathsf{b}_{h_2} - 1$, up to b_{s_2} evaluations of $s_2(X)$ are uniformly random in $\mathbb{F}$ even conditioned on $r_{s_2}(X)$ and

σ, so the simulated answers $\{s_{2,\delta}\}$ are identically distributed. The values $\{h_{2,\delta}\}$ are chosen to make the verifier accept, as the polynomial $h_2(X)$ is determined by the equation

$$h_2(X) = \frac{b(X)f(X) - a(X) + \zeta(s_2(X)z_{\mathbb{H}}(\beta) + f(X) - Xg_2(X) - \gamma/|\mathbb{K}|)}{z_{\mathbb{K}}(X)}$$

As all of the polynomials involved in the expression above support b_{h_2} more evaluations, we have that the simulated answers $\{h_{2,\delta}\}$ are identically distributed to the evaluations $\{h_2(\delta)\}$ in the real protocol. □

We make the same remark as in [CFF+20] (Remark 1 therein) to optimize the degree of the polynomials.

Remark 3 (On degree optimizations). The proof of this Theorem shows that the evaluations $h_1(\delta)$ and $h_2(\delta)$ can be computed using the evaluations $\{p_i(\delta)\}_{i\in[16]\setminus\{13,16\}}$, that is, every polynomial involved in the protocol except $h_1(X)$ and $h_2(X)$. Therefore, additional information about $\{p_i(X)\}_{i\in[16]\setminus\{13,16\}}$ is only revealed if $h_1(X)$ or $h_2(X)$ are evaluated in a point in which the $\{p_i(X)\}_{i\in[16]\setminus\{13,16\}}$ were not already evaluated. If this is not the case, we can lower the degree of the polynomials by b_{h_1} or b_{h_2}. More concretely, we can have $\deg(p_i(X)) = |\mathbb{H}| + \mathsf{b}_i - 1$ for $i \in \{6,7,8,9,12\}$, that is, for $\hat{w}, \hat{z}_A, \hat{z}_B, s_1$ and g_1, and $\deg(p_i(X)) = |\mathbb{K}| + \mathsf{b}_i - 1$ for $i \in [5] \cup \{10,11,14,15\}$, that is for $\mathsf{row}, \mathsf{col}, \mathsf{rowcol}, \mathsf{val}_{A^*}, \mathsf{val}_{B^*}, s_2, t, f$ and g_2.

6.3 Instantiating our Compiler: FHFCs from Marlin

In this section, we present the last ingredients we need to instantiate Theorem 1, namely, the compiling commitment scheme CS and its CP-SNARKs.

The compiling commitment scheme. Recall that the compiling commitment scheme CS (see Definition 19) must contain two types $\{\mathtt{swh},\mathtt{hid}\}$, the first being for polynomials, which must be somewhat hiding, and the second for the relation, which must be fully hiding. For the type $\mathtt{swh}$, we will proceed as in [CFF+21] and use the KZG commitment, which is somewhat hiding.

If $v \in \mathbb{F}^m$ is a vector of length $m = |\mathbb{K}|$, we can also use KZG to produce a fully hiding commitment to v assuming a commitment key $\mathsf{ck} = \big(\{[\tau^i]_1\}_{i\in[0,d]}, [1,\tau]_2\}\big)$, with $d \geq m$. Namely, we let $p(X) \in \mathbb{F}^{\leq d}[X]$ be a randomized low degree extension of v, i.e., $p(X) = \sum_{i=1}^{m} v_i \lambda_i^{\mathbb{K}}(X) + \rho(X) z_{\mathbb{K}}(X)$ for a random $\rho(X) \leftarrow \mathbb{F}^{\leq d-m}[X]$. The commitment is then $c = [p(\tau)]_1$ and the opening $o = \rho(X)$.

Observe that our relation $\mathsf{R} = (A,B) \in \mathcal{D}_{\mathsf{func}}$ can be identified with vectors v_{row}, v_{col}, v_{rowcol}, $v_{\mathsf{val}_{A^*}}$ and $v_{\mathsf{val}_{B^*}}$ in $\mathbb{F}^m$. Furthermore, the randomized indexer $\mathcal{I}(\mathsf{R},\mathsf{r})$ outputs randomized low degree extensions of those vectors (see Section 6.2.1). Thus, to commit to the relation R, we will send KZG commitments to the polynomials output by $\mathcal{I}(\mathsf{R},\mathsf{r})$ for $\mathsf{r} \leftarrow \mathsf{Rand}$.

It can be argued that the polynomial $\mathsf{rowcol}(X)$ should not be included in the commitment to R, as its values over the subgroup $\mathbb{K}$ are determined by $\mathsf{row}(X)$ and $\mathsf{col}(X)$. We explain in Remark 5 how having $\mathsf{rowcol}(X)$ be part of the commitment to the relation allows us to optimize the compiled protocol.

Therefore, our commitment key will be $\mathsf{ck} := \big(\{[\tau^i]_1\}_{i\in[0,\mathsf{D}]}, [1,\tau]_2\}\big)$, where we recall that $\mathsf{D} \geq m$ is the maximum degree of any polynomial involved in the rPHP.

The CP-SNARKs $\mathsf{CP}_{\mathsf{opn}}$ and $\mathsf{CP}_{\mathsf{php}}$. We will use the CP-SNARKs $\mathsf{CP}_{\mathsf{opn}}$ and $\mathsf{CP}_{\mathsf{php}}$ from [CFF+21]. As we will analyze our protocol in the Algebraic Group Model, the proof of opening for the KZG commitment is the empty string, so from now on we will ignore it.

Remark 4 (Further degree optimizations). Our compiler requires that $\mathsf{CP}_{\mathsf{php}}$ is (b, C)-leaky zero-knowledge for a $(\mathsf{b}+\mathbf{1})$-index-private rPHP. Thanks to the homomorphic property of the KZG commitment and the use of linearization tricks, the first verification equation in our rPHP (see Figure 2) can be proven by only revealing $t(\beta)$ and $\hat{z}_B(\beta)$, and the second equation by only revealing $f(\beta')$, where $\beta' \leftarrow \mathbb{F}$. Therefore, our concrete $\mathsf{b} \in \mathbb{F}^{16}$ is given by

$$\begin{aligned}\mathsf{b} = (&\mathsf{b}_{\mathsf{row}} = 0, \mathsf{b}_{\mathsf{col}} = 0, \mathsf{b}_{\mathsf{rowcol}} = 0, \mathsf{b}_{\mathsf{val}_{A^*}} = 0, \mathsf{b}_{\mathsf{val}_{B^*}} = 0, \mathsf{b}_{\hat{w}} = 0, \mathsf{b}_{\hat{z}_A} = 0,\\ &\mathsf{b}_{\hat{z}_B} = 1, \mathsf{b}_{s_1} = 0, \mathsf{b}_{s_2} = 0, \mathsf{b}_t = 1, \mathsf{b}_{g_1} = 0, \mathsf{b}_{h_1} = 0, \mathsf{b}_f = 1, \mathsf{b}_{g_2} = 0, \mathsf{b}_{h_2} = 0).\end{aligned}$$

Since all of the polynomials are evaluated at the trapdoor τ, we can use the optimization from Remark 3 to decrease the degree of the polynomials involved in the protocol and still meet the $\mathsf{b}+\mathbf{1}$ bound for the rPHP with the concrete b from above. In particular, since all of the index polynomials are only evaluated once, we set the randomness space to be $\mathsf{Rand} = \mathbb{F}^5$. Moreover, the maximum degree of the protocol is $\mathsf{D} = |\mathbb{K}| + 1 = m + 1$.

The CP-SNARKs $\mathsf{CP}_{\mathsf{PFR}}$ and $\mathsf{CP}_{\mathsf{link}}$. Recall that the PFR must check that the committed relation $\mathsf{R} = (A, B)$ is in the functional set $\mathcal{D}_{\mathsf{func}}(n, m, t, s)$ defined in (2). In other words, it must check that A and B are t-strictly lower triangular matrices (see Definition 23). Notice that the polynomials $\mathsf{row}(X)$ and $\mathsf{col}(X)$ we have used to commit to R encode the row and column of the non-zero values of A^* and B^*. Given that the matrix M^* is defined by $M^*_{a,b} = u_{\mathbb{H}}(b,b)M_{b,a}$ for $a, b \in \mathbb{H}$, since $u_{\mathbb{H}}(b,b) \neq 0$ for each $b \in \mathbb{H}$, we have that M is t-strictly lower triangular if and only if the first t columns of M^* are equal to zero and M^* is strictly upper triangular. This condition is equivalent to having $\mathsf{row}(\mathbb{K}) \subseteq \{1, \omega, \ldots, \omega^{n-1}\}$, $\mathsf{col}(\mathbb{K}) \subseteq \{\omega^t, \ldots, \omega^{n-1}\}$ and $\log_\omega(\mathsf{row}(\kappa^i)) < \log_\omega(\mathsf{col}(\kappa^i))$ for $i \in [m]$. Instead of building a relation that only checks those conditions, we will augment it to also check $\mathsf{rowcol}(\kappa^i) = \mathsf{row}(\kappa^i)\mathsf{col}(\kappa^i)$ for $i \in [m]$. In more detail, we define

$$\hat{\mathcal{R}}_{\mathsf{func}} = \left\{ \left(\begin{array}{c} (c_{\mathsf{row}}, c_{\mathsf{col}}, c_{\mathsf{rowcol}}, c_{\mathsf{val}_{A^*}}, c_{\mathsf{val}_{B^*}}), \\ (\mathsf{row}, \mathsf{col}, \mathsf{rowcol}, \mathsf{val}_{A^*}, \mathsf{val}_{B^*}) \end{array} \right) \middle| \begin{array}{l} \mathsf{row}(\mathbb{K}) \subseteq \{1, \omega, \ldots, \omega^{n-1}\} \\ \mathsf{col}(\mathbb{K}) \subseteq \{\omega^t, \ldots, \omega^{n-1}\} \\ \log_\omega(\mathsf{row}(\kappa^i)) < \log_\omega(\mathsf{col}(\kappa^i)),\ i \in [m] \\ \mathsf{rowcol}(\kappa^i) = \mathsf{row}(\kappa^i)\mathsf{col}(\kappa^i),\ i \in [m] \\ c_{\mathsf{row}} = [\mathsf{row}(\tau)]_1,\ c_{\mathsf{col}} = [\mathsf{col}(\tau)]_1 \\ c_{\mathsf{rowcol}} = [\mathsf{rowcol}(\tau)]_1 \\ c_{\mathsf{val}_B} = [\mathsf{val}_B(\tau)]_1,\ c_{\mathsf{val}_A} = [\mathsf{val}_A(\tau)]_1 \end{array} \right\}.$$

Remark 5 (Optimizing the compiled protocol.). Recall that in the first round of the compiler from Figure 1, the prover samples fresh randomness $\mathsf{r} \leftarrow \mathsf{Rand}$, sends commitments to the polynomials output by $\mathcal{I}(\mathsf{R}, \mathsf{r})$ and, also sends the proofs π_{PFR} and π_{link}. Notice that an accepting proof π_{PFR} for the relation $\hat{\mathcal{R}}_{\mathsf{func}}$ we have just defined implies that the commitment $\boldsymbol{c}$ to the relation R also opens to the polynomials output by the randomized indexer, i.e., it also proves the linking relation (this is the reason we added the commitment to rowcol to the commitment of R and the condition $\mathsf{rowcol}(\kappa^i) = \mathsf{row}(\kappa^i)\mathsf{col}(\kappa^i)$ to the relation). Therefore, the prover can skip sending the new commitments to the fresh output of $\mathcal{I}(\mathsf{R}, \mathsf{r})$ and instead use the commitment to R. Moreover, the prover can also avoid sending the proof π_{link} for the linking relation.

The reason why zero-knowledge is not broken is because the index polynomials are only evaluated at the trapdoor τ (see Remark 4), which is the same across different executions of the protocol. Therefore, it is secure to reuse the same commitments without leaking additional information.

In order to build the CP-SNARK for $\hat{\mathcal{R}}_{\mathsf{func}}$, we first translate the condition $\log_\omega(\mathsf{row}(\kappa^i)) < \log_\omega(\mathsf{col}(\kappa^i))$ into other terms that are easier to check. Like in [BNO21], we assume that there exists $\Delta \in \mathbb{F}$ such that $\Delta^2 = \omega$, and thus, generates a subgroup $\mathbb{D}$ of order $2n$.

Lemma 3. *Let* $\mathsf{row}, \mathsf{col}, R, C, \varphi \in \mathbb{F}[X]$ *such that* $R^2(\kappa^i) = \mathsf{row}(\kappa^i), C^2(\kappa^i) = \mathsf{col}(\kappa^i)$ *and* $\varphi(\kappa^i) = \frac{C(\kappa^i)}{\Delta R(\kappa^i)}$. *Furthermore, suppose that* $R(\mathbb{K}), C(\mathbb{K}), \varphi(\mathbb{K}) \subseteq \{1, \Delta, \ldots, \Delta^{n-1}\}$. *Then* $\mathsf{row}(\mathbb{K}), \mathsf{col}(\mathbb{K}) \subseteq \mathbb{H}$ *and* $\log_\omega(\mathsf{row}(\kappa^i)) < \log_\omega(\mathsf{col}(\kappa^i))$.

Proof. Since $R(\kappa^i), C(\kappa^i) \in \{1, \Delta, \ldots, \Delta^{n-1}\}$, write $R(\kappa^i) = \Delta^{r_i}$, $C(\kappa^i) = \Delta^{c_i}$, for some $r_i, c_i \in [0, n-1]$. Then $\mathsf{row}(\kappa^i) = R(\kappa^i)^2 = (\Delta^{r_i})^2 = \omega^{r_i}$, so $\mathsf{row}(\mathbb{K}) \subseteq \mathbb{H}$. Same for $\mathsf{col}(\mathbb{K}) \subseteq \mathbb{H}$. Also, $\varphi(\kappa^i) \in \{1, \Delta, \ldots, \Delta^{n-1}\}$, so $\varphi(\kappa^i) = \Delta^{\alpha_i}$ with $\alpha_i \in [0, n-1]$. In other words, $\Delta^{c_i} = C(\kappa^i) = \Delta R(\kappa^i)\varphi(\kappa^i) = \Delta\Delta^{r_i}\Delta^{\alpha_i} = \Delta^{r_i+\alpha_i+1}$, that is, $c_i = r_i + \alpha_i + 1 \bmod 2n$ because Δ generates a group of order $2n$. Since $r_i, c_i, \alpha_i \in [0, n-1]$, we have that

$$1 \leq r_i + \alpha_i + 1 \leq n - 1 + n - 1 + 1 = 2n - 1,$$

so no reduction modulo $2n$ occurs and the identity $c_i = r_i + \alpha_i + 1$ holds as natural numbers. In particular, $c_i \geq r_i + 1$, so $c_i > r_i$. □

Notice that if we check the hypothesis of Lemma 3, we would have already covered most conditions needed for the PFR, the ones missing would be $\mathsf{rowcol}(\kappa^i) = \mathsf{row}(\kappa^i)\mathsf{col}(\kappa^i)$ and $\mathsf{col}(\mathbb{K}) \subseteq \{\omega^t, \ldots, \omega^{n-1}\}$. The former is simply a zero check over $\mathbb{K}$; for the latter we could do the same as [BNO21] and check it with another lookup $\mathsf{col}(\mathbb{K}) \subseteq \{\omega^t, \ldots, \omega^{n-1}\}$. However, this would require performing lookups over different tables, which cannot be efficiently batched. Instead, we will add the lookup $\frac{C(\mathbb{K})}{\Delta^t} \subseteq \{1, \Delta, \ldots, \Delta^{n-1}\}$, as having both conditions $C(\mathbb{K}), \frac{C(\mathbb{K})}{\Delta^t} \subseteq \{1, \Delta, \ldots, \Delta^{n-1}\}$ implies $\mathsf{col}(\mathbb{K}) \subseteq \{\omega^t, \ldots, \omega^{n-1}\}$.

Therefore, the PFR basically comes down to executing several lookups over the same table. We will use logarithmic derivatives [Hab22] due to their nice batching properties. Since we are working with the KZG commitment, our protocol will reduce to performing minor changes on the state-of-the-art lookup [CFF+24]. For completeness, we present the explicit protocol in Appendix B. We highlight that proof consists of 13 elements in $\mathbb{G}_1$ and 4 elements in $\mathbb{F}$. The prover is quasilinear in $n = |\mathbb{H}|$, while the verifier is logarithmic in n and needs to perform 2 pairings.

Overall efficiency of our Marlin-based Function-Hiding Functional Commitment. The commitment to the relation $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}(n, m, t, s)$ consists of 5 elements in $\mathbb{G}_1$. Thanks to Remark 5, the final proof for our function-hiding functional commitment is $\pi = (\pi_{\mathsf{PFR}}, \pi_{\mathsf{R}})$, where π_{R} is the proof that results from compiling the rPHP from Figure 2. There, the prover sends 11 polynomials and one field element and after compilation, the proof consists of 15 elements in $\mathbb{G}_1$ and 4 elements in $\mathbb{F}$. The additional field elements come from the evaluation of three polynomials; two of the additional group elements are evaluation proofs and the two remaining enforce the degree checks. Actually, we could employ the same trick described in [CFF+24] (we also explain it in Appendix B for the PFR) to enforce the degree checks without sending additional group elements, so the proof consists of 13 elements in $\mathbb{G}_1$ and 4 elements in $\mathbb{F}$. At the 128-bit security level, when instantiating our protocol with the BLS12-381 curve[7], the proof size of π_{R} ends at 0.752 kB, and the proof $\pi = (\pi_{\mathsf{PFR}}, \pi_{\mathsf{R}})$, at 1.504 kB. The prover is quasilinear in n; the verifier is logarithmic in n and needs to perform 4 pairings, which with standard batching tricks can be reduced to just 2 pairings.

7 FHFCs from Plonk

In this section, we present our Function-Hiding Functional Commitments from Plonk [GWC19]. We start by describing the constraint system, which will be a generalization of the one from [GWC19], and its functional set. After that, we introduce our rPHP for this constraint system to finally instantiate our compiler.

[7] In this case, an element in $\mathbb{G}_1$ takes 384 bits and an element in $\mathbb{F}$ takes 256 bits.

7.1 The Generalized Plonk Constraint System

We begin by defining a relation that generalizes the Plonk constraint system [GWC19]. We directly define its output-final version.

Definition 25. Let $n, \ell, s \in \mathbb{N}$ such that $\ell + s \leq n$. Let $q_O = (1, \ldots, 1) \in \mathbb{F}^n$. The *Generalized Plonk constraint system* is the set $\mathcal{R}_{\mathsf{GPlonk}}(n, \ell, s)$ defined as

$$\left\{ (\mathsf{R}, (x, y), w) \;\middle|\; \begin{array}{l} \mathsf{R} = ((q_L, q_R, q_M) \in (\mathbb{F}^n)^3,\ f : [3n] \to [3n]), \\ (x, y) = ((x_1, \ldots, x_\ell), (y_1, \ldots, y_s)) \in \mathbb{F}^\ell \times \mathbb{F}^s, \\ w = (w_1, \ldots, w_{3n}) \in \mathbb{F}^{3n}, \\ \forall i \in [\ell], q_{L_i} = q_{R_i} = q_{M_i} = 0,\ x_i = w_{2n+i} \\ \forall i \in [s], y_i = w_{3n-s+i} \\ \forall i \in [\ell+1, n], q_{L_i} w_i + q_{R_i} w_{n+i} + q_{M_i} w_i w_{n+i} = q_{O_i} w_{2n+i} \\ \forall i \in [3n], w_i = w_{f(i)} \end{array} \right\}$$

We explain the modifications we have performed on the original definition. First of all, we have changed where the witness w encodes the input x. Namely, we have $w_{2n+i} = x_i$ for $i \in [\ell]$, whereas normally it is required that $w_i = x_i$ for $i \in [\ell]$. This change implies that we have to modify the definition of the vectors q_L, q_R, q_M and q_O to enforce consistency with the input. Usually, it is required that $q_{L_i} = 1$, $q_{R_i} = q_{M_i} = q_{O_i} = 0$, so that the equation $q_{L_i} w_i + q_{R_i} w_{n+i} + q_{M_i} w_i w_{n+1} = q_{O_i} w_{2n+i} - x_i$ becomes $w_i = x_i$ for $i \in [\ell]$. By letting $q_{L_i} = 0$ and $q_{O_i} = 1$, the previous equation enforces $w_{2n+i} = x_i$, as we wanted. Finally, the output y are the last s coordinates of w.

The assumption on $q_{O_i} = 1$ for $i \in [\ell + 1, n]$ is not needed. However, it does not limit the expressiveness of the constraint system and, like in the Marlin case, it will help us achieve better efficiency in the final protocol as now the vector q_O only depends on the public parameters.

Yet, the most important change of our definition with respect to the original one is that we are allowing $f : [3n] \to [3n]$ to be an arbitrary map, whereas in [GWC19] it must be a permutation. This has non-trivial implications, as the Plonk zkSNARK proves the equation $w_i = w_{f(i)}$ with a protocol that only works when f is a permutation. Furthermore, our functional set will require that f is not injective, which implies that it cannot be a permutation. Hence, we will need to use different techniques to design our rPHP for $\mathcal{R}_{\mathsf{GPlonk}}(n, \ell, s)$.

7.2 Functional Set for the Generalized Plonk Constraint System

Before describing the functional set for the constraint system we have just introduced, we explain how it relates to arithmetic circuits, as this intuition will be helpful later in the section. First, we introduce the definition of arithmetic circuit we will work with.

Definition 26. An *arithmetic circuit with ℓ inputs, s outputs, and n_g gates* is a sequence $C = \{(l_i, r_i, s_i)\}_{i=1}^{n_g}$ where $l_i, r_i < \ell + i$ and $s_i \in \{+, \times\}$ for each $i \in [n_g]$. For the gate $i \in [n_g]$, we refer to l_i, r_i as the *left and right input wire indices* and $\ell + i$ as the *output wire index*.

Given an input $x \in \mathbb{F}^\ell$, to calculate the output of an arithmetic circuit C, one proceeds as follows. Let $w'_i = x_i$ for $i \in [\ell]$. Then, $w'_{\ell+i} = w'_{l_i} + w'_{r_i}$ if $s_i = +$ or $w'_{\ell+i} = w'_{l_i} w'_{r_i}$ if $s_i = \times$ for $i \in [n_g]$. The output $y = (y_1, \ldots, y_s) = (w'_{n-s+1}, \ldots, w'_n)$ are the last s coordinates of the witness $w' \in \mathbb{F}^{\ell+n_g}$.

Essentially, the Generalized Plonk witness $w \in \mathbb{F}^{3n}$ is a redundant wire assignment. The equation $q_{L_i} w_i + q_{R_i} w_{n+i} + q_{M_i} w_i w_{n+1} = q_{O_i} w_{2n+i}$ ensures the correctness of each gate operation, where $w_{\ell+i}$, $w_{n+\ell+i}$ and $w_{2n+\ell+i}$ are supposed to be the left, right and output wires of the i-th gate, respectively. Therefore $w_{\ell+i}$ should be an input of the circuit,

or an output of a gate that has already been computed (same for $w_{n+\ell+1}$). This should be enforced with the equation $w_i = w_{f(i)}$ for $i \in [3n]$. To make sure that this is the case, we will need that the function f satisfies the following properties.

Definition 27. Let $f : [3n] \to [3n]$. We say f is *function inducing* for ℓ inputs if

1. $f(2n+i) = 2n+i \quad \forall i \in [n]$.
2. $f(\ell+i) \in [2n+1, 2n+\ell+i-1]$ for $i \in [n-\ell]$.
3. $f(n+\ell+i) \in [2n+1, 2n+\ell+i-1]$ for $i \in [n-\ell]$.
4. $f(i) = f(n+i) = 2n+i \quad \forall i \in [\ell]$.

Intuitively, the first item enforces that the witness $w \in \mathbb{F}^{3n}$ of the Generalized Plonk has the witness $w' \in \mathbb{F}^n$ of the arithmetic circuit in the last n coordinates, i.e., $w_{2n+i} = w'_i$ for $i \in [n]$, where $n = \ell + n_g$. To make sure that $w_{\ell+i}$ is an input of the circuit or an output of a gate that has already been computed, we impose $f(\ell+i) \in [2n+1, 2n+\ell+i-1]$ for $i \in [n_g]$. The analogous holds for the right inputs and item 3. The values w_i and w_{n+i} for $i \in [\ell]$ are completely redundant and have no relation with the arithmetic circuit. Item 4 fixes their values to be $2n+i$ because this will lead to a more efficient PFR, although they could be arbitrary.

The next theorem shows that Definition 27 is enough to define a functional set for the Generalized Plonk constraint system.

Theorem 4. *The following is a functional set for* $\mathcal{R}_{\mathsf{GPlonk}}(n, \ell, s)$

$$\mathcal{D}_{\mathsf{func}}(n,\ell,s) = \left\{ ((q_L, q_R, q_M) \in (\mathbb{F}^n)^3,\ f : [3n] \to [3n]), \ \middle|\ \begin{array}{l} q_{L_i} = q_{R_i} = q_{M_i} = 0,\ \forall i \in [\ell] \\ f \text{ is function inducing} \end{array} \right\}$$

Proof. Let $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}(n, \ell, s)$. We will prove that for each input $x \in \mathbb{F}^\ell$, there is a unique output $y \in \mathbb{F}^s$ such that $(x, y) \in \mathcal{L}(\mathsf{R})$. We start by computing the witness $w \in \mathbb{F}^{3n}$. This will also show that the algorithm $\mathsf{Extend}(\mathsf{R}, x) \to (y, w)$ is efficient to compute. For $i \in [\ell]$, let

$$w_i := x_i, \quad w_{n+i} := x_i, \quad w_{2n+i} := x_i,$$

which satisfy $w_i = w_{f(i)}$, $w_{n+i} = w_{f(n+i)}$ and $w_{2n+i} = w_{f(2n+i)}$. For $i = \ell+1$, define

$$w_{\ell+1} := w_{f(\ell+1)}, \qquad w_{n+\ell+1} := w_{f(n+\ell+1)}.$$

Since f is function inducing, $f(\ell+1) = 2n+j$ for some $j \in [\ell]$, so $w_{\ell+1}$ is well defined because $w_{f(\ell+1)} = w_{2n+j} = x_j$. The same applies for $w_{n+\ell+1}$. Then, we define

$$w_{2n+\ell+1} := q_{L_{\ell+1}} w_{\ell+1} + q_{R_{\ell+1}} w_{n+\ell+1} + q_{M_{\ell+1}} w_{\ell+1} w_{n+\ell+1},$$

which also satisfies $w_{2n+\ell+1} = w_{f(2n+\ell+1)}$. We will construct the rest of the solution inductively. For $i > 1$, define

$$w_{\ell+i} := w_{f(\ell+i)}, \qquad w_{n+\ell+i} := w_{f(n+\ell+i)},$$

$$w_{2n+\ell+i} := q_{L_{\ell+i}} w_{\ell+i} + q_{R_{\ell+i}} w_{n+\ell+i} + q_{M_{\ell+i}} w_{\ell+i} w_{n+\ell+i}.$$

Again, $w_{\ell+i}$ and $w_{n+\ell+i}$ are well defined thanks to $f(\ell+i), f(n+\ell+i) \in [2n+1, 2n+\ell+i-1]$ and the induction hypothesis. Moreover, we also have $w_{2n+\ell+i} = w_{f(2n+\ell+i)}$. Letting $y \in \mathbb{F}^s$ be the last s coordinates of $w \in \mathbb{F}^{3n}$, we arrive to $((x, y), w) \in \mathsf{R}$.

Let us prove that this $w \in \mathbb{F}^{3n}$ is the unique solution. Suppose that there exists another $w' \in \mathbb{F}^{3n}$, with $w'_{2n+i} = x_i$ for $i \in [\ell]$ such that $w'_i = w'_{f(i)}$ and

$$w'_{2n+\ell+i} = q_{L_{\ell+i}} w'_{\ell+i} + q_{R_{\ell+1}} w'_{n+\ell+i} + q_{M_{\ell+1}} w'_{\ell+i} w'_{n+\ell+i}.$$

In particular, the equation above holds for $i = 1$. Again, we can write $f(\ell + 1) = 2n + j$ for some $j \in [\ell]$. Then, $w'_{\ell+1} = w'_{f(\ell+1)} = w'_{2n+j} = x_j$. But we also know that $w_{\ell+1} = x_j$, so $w'_{\ell+1} = w_{\ell+1}$, and the same argument works for $w'_{n+\ell+1} = w_{n+\ell+1}$. Both conditions imply that $w'_{2n+\ell+1} = w_{2n+\ell+1}$. We can reason inductively as before to deduce $w' = w$.

Since the output $y \in \mathbb{F}^s$ are the last s coordinates of $w \in \mathbb{F}^{3n}$, which is uniquely determined by the input x, so is y.

□

To show that we are actually defining something meaningful, we will prove that every arithmetic circuit can be described by a relation $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}(n, \ell, s)$.

Theorem 5. *Let $C = \{(l_i, r_i, s_i)\}_{i=1}^{n_g}$ be an arithmetic circuit with input $x \in \mathbb{F}^\ell$, output $y \in \mathbb{F}^s$ and n_g gates. Let $n = \ell + n_g$. Then, there exists $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}(n, \ell, s)$ such that $(\mathsf{R}, (x, y)) \in \mathcal{L}(\mathcal{R}_{\mathsf{GPlonk}}(n, \ell, s))$.*

Proof. We start by defining the map $f : [3n] \to [3n]$ that will satisfy the properties of Definition 27. Some of its values are fixed and do not depend on the arithmetic circuit. Namely, we must have $f(2n + i) := 2n + i$ for $i \in [n]$, and $f(i) := 2n + i$, $f(n + i) := 2n + i$ for $i \in [\ell]$. Thus, we only need to define $f(\ell + i)$ and $f(n + \ell + i)$ for $i \in [n_g]$. Let

$$f(\ell + i) := 2n + l_i, \quad f(n + \ell + i) := 2n + r_i, \tag{3}$$

where l_i and r_i are the left and right indices of i-th gate of the arithmetic circuit, respectively. By definition of arithmetic circuit, we have $l_i,\ r_i \in [\ell + i - 1]$ for $i \in [n_g]$. This immediately implies that f satisfies Definition 27.

Now, we define the selector vectors in the standard way. For $i \in [\ell]$, let $q_{L_i} = q_{R_i} = q_{M_i} = 0$. For $i \in [n_g]$ let $q_{L_{i+\ell}} = q_{R_{i+\ell}} = 1$ and $q_{M_{i+\ell}} = 0$ if $s_i = +$ or $q_{L_{i+\ell}} = q_{R_{i+\ell}} = 0$ and $q_{M_{i+\ell}} = 1$ if $s_i = \times$.

Finally, we show how to compute the witness $w \in \mathbb{F}^{3n}$ for this Generalized Plonk relation that has x as input and y as output. Define

$$w_i := x_i, \quad w_{n+i} := x_i, \quad w_{2n+i} := x_i, \quad i \in [\ell]. \tag{4}$$

Let $w' \in \mathbb{F}^n$ be the witness for the arithmetic circuit C and define

$$w_{\ell+i} := w'_{l_i}, \quad w_{n+\ell+i} := w'_{r_i}, \quad w_{2n+\ell+i} := w'_{\ell+i}, \qquad i \in [n_g]. \tag{5}$$

It is immediate to see that w has in the last s coordinates the output y. Moreover, we also have that for $i \in [\ell + 1, n]$,

$$q_{L_i} w_i + q_{R_i} w_{n+i} + q_{M_i} w_i w_{n+i} = w_{2n+i}.$$

All that is left to prove is that $w_i = w_{f(i)}$ for $i \in [3n]$.

As $f(i) = f(n + i) = 2n + i$ if $i \in [\ell]$, and $w_{2n+i} = x_i$ for $i \in [\ell]$, the equation $w_i = w_{f(i)}$ clearly holds in these cases. Since $f(2n + i) = 2n + i$ for $i \in [n]$, we also have $w_{2n+i} = w_{f(2n+i)}$. We argue $w_{\ell+i} = w_{f(\ell+i)}$ for $i \in [n_g]$, the case $w_{n+\ell+i} = w_{f(n+\ell+i)}$ is analogous. Equation (5) says that $w_{\ell+i} = w'_{l_i}$ for $i \in [n_g]$. On the other hand, equations (5) and (4) imply that $w_{2n+i} = w'_i$ for $i \in [n]$. Thanks to (3), $f(\ell + i) = 2n + l_i$. Putting it all together, we arrive to

$$w_{f(\ell+i)} = w_{2n+l_i} = w'_{l_i} = w_{\ell+i}.$$

□

7.3 Our rPHP for the Generalized Plonk Constraint System

In this section, we present our rPHP for the Generalized Plonk constraint system.

7.3.1 Offline Phase of our rPHP

As in the Marlin case, the randomized indexer will essentially mask the regular index polynomials from [GWC19]. We assume the existence of a subgroup $\mathbb{H} = \langle \omega \rangle$ of order n.

The randomness space. Let $\mathsf{b}_L, \mathsf{b}_R, \mathsf{b}_M, \mathsf{b}_{f,A}, \mathsf{b}_{f,B}, \mathsf{b}_q \in \mathbb{N}$ and define

$$\mathsf{Rand} := \mathbb{F}^{\leq \mathsf{b}_L + \mathsf{b}_q - 1}[X] \times \mathbb{F}^{\leq \mathsf{b}_R + \mathsf{b}_q - 1}[X] \times \mathbb{F}^{\leq \mathsf{b}_M + \mathsf{b}_q - 1}[X] \times \mathbb{F}^{\leq \mathsf{b}_{f,A} + \mathsf{b}_q - 1}[X] \times \mathbb{F}^{\leq \mathsf{b}_{f,B} + \mathsf{b}_q - 1}[X].$$

For $\mathsf{r} \in \mathsf{Rand}$, write $\mathsf{r} = (r_L(X), r_R(X), r_M(X), r_{f,A}(X), r_{f,B}(X))$.

The randomized indexer On input a relation $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}(n, \ell, s)$ and randomness $\mathsf{r} \leftarrow \mathsf{Rand}$, the randomized indexer $\mathcal{I}(\mathsf{R}, \mathsf{r})$ outputs the polynomials $q_L(X)$, $q_R(X)$, $q_M(X)$, $S_{f,A}(X)$ and $S_{f,B}(X)$ defined as

$$q_L(X) := \sum_{i=1}^{n} q_{L_i} \lambda_i(X) + r_L(X) z_{\mathbb{H}}(X) \qquad q_R(X) := \sum_{i=1}^{n} q_{R_i} \lambda_i(X) + r_R(X) z_{\mathbb{H}}(X)$$

$$q_M(X) := \sum_{i=1}^{n} q_{M_i} \lambda_i(X) + r_M(X) z_{\mathbb{H}}(X) \qquad S_{f,A}(X) := \sum_{i=1}^{n} \omega^{f(i)} \lambda_i(X) + r_{f,A}(X) z_{\mathbb{H}}(X)$$

$$S_{f,B}(X) := \sum_{i=1}^{n} \omega^{f(n+i)} \lambda_i(X) + r_{f,B}(X) z_{\mathbb{H}}(X)$$

Notice that we are encoding the function $f : [3n] \to [3n]$ with only 2 polynomials. This is because $f(2n+i) = 2n+i$ for $i \in [n]$, so the polynomial encoding this part is the identity which is public and can be efficiently computed. Moreover, since $f([3n]) \subseteq [2n+1, 3n]$, we have $S_{f,A}(\mathbb{H})$ and $S_{f,B}(\mathbb{H}) \subseteq \mathbb{H}$, whereas in [GWC19] they need to take values over different cosets of $\mathbb{H}$.

7.3.2 Online Phase of our rPHP

We proceed to give an overview of the online phase of our rPHP for the Generalized Plonk constraint system. The full protocol is depicted in Figure 3.

The prover starts by splitting the witness $w \in \mathbb{F}^{3n}$ into three different polynomials $A(X)$, $B(X)$ and $C(X)$, which are randomized low degree extensions of $(w_1, \ldots, w_n)$, $(w_{n+1}, \ldots, w_{2n})$ and $(w_{2n+1}, \ldots, w_{3n})$, respectively. To keep the maximum degree of the protocol low, the prover will additionally send an auxiliary polynomial $\mathsf{aux}(X)$, which will be a randomized low degree extension of the vector $(w_1 w_{n+1}, \ldots, w_n w_{2n})$. Then, the gate equations $q_{L_i} w_i + q_{R_i} w_{n+i} + q_{M_i} w_i w_{n+1} = w_{2n+i}$ can be proven with a zero check over $\mathbb{H}$, i.e., by sending the polynomial $q(X)$ such that

$$q_L(X)A(X) + q_R(X)B(X) + q_M(X)\mathsf{aux}(X) = C(X) + \mathsf{PI}(X) + q(X) z_{\mathbb{H}}(X).$$

Here $\mathsf{PI}(X) = -\sum_{i=1}^{\ell} x_i \lambda_i(X)$ is the polynomial that interpolates the input $x \in \mathbb{F}^{\ell}$ and enforces that $w_{2n+i} = x_i$ for $i \in [\ell]$. Moreover, the prover must also enforce consistency with the output. This can be done by running a zero check over $\mathbb{H}$ for the polynomial $z_{\mathbb{H}[\leq n-s]}(X)(C(X) - \mathsf{PO}(X))$, where now $\mathsf{PO}(X) = \sum_{i=1}^{s} y_i \lambda_{n-s+i}(X)$. Finally, to show that $\mathsf{aux}(X)$ has been defined correctly, it must also run a zero check over $\mathbb{H}$ for the polynomial $\mathsf{aux}(X) - A(X)B(X)$.

This first step is almost identical to the original Plonk PHP. However, in order to prove $w_i = w_{f(i)}$ for $i \in [3n]$, we cannot proceed as in [GWC19] because their protocol relies on the fact that f is a permutation, which is not our case. Therefore, we must use different ideas.

We transform $w_i = w_{f(i)}$ into equations involving the polynomials that encode the witness w and the function f. For $i \in [n]$, it becomes $A(\omega^i) = C(S_{f,A}(\omega^i))$, since $f([n]) \subseteq [2n+1, 3n]$. For $i \in [n+1, 2n]$, the equation becomes $B(\omega^i) = C(S_{f,B}(\omega^i))$ since $f([n+1, 2n]) \subseteq [2n+1, 3n]$. For $i \in [2n+1, 3n]$, it becomes $C(\omega^i) = C(\omega^i)$, since $f(i) = i$, so in this case there is nothing to show. Therefore, the problem reduces to proving that a certain polynomial agrees with the composition of two other polynomials over the subgroup $\mathbb{H}$. As pointed out in [GPS25], this problem can be solved by using logarithmic derivatives, in this case, their indexed variant.

Lemma 4 ([GPS25])**.** *Let $\mathbb{F}$ be field of characteristic $p > n$. Let $f(X), g(X), h(X) \in \mathbb{F}[X]$. Then $f(\omega^i) = g(h(\omega^i))$ for $i \in [n]$ if and only if there exists a vector $m \in \mathbb{F}^n$ such that*

$$\sum_{i=1}^{n} \frac{1}{f(\omega^i) + Xh(\omega^i) + Y} = \sum_{i=1}^{n} \frac{m_i}{g(\omega^i) + X\omega^i + Y}.$$

Thus, if the prover sends a polynomial $m_A(X)$ and then shows

$$\sum_{i=1}^{n} \frac{1}{A(\omega^i) + \beta S_{f,1}(\omega^i) + \gamma} = \sum_{i=1}^{n} \frac{m_A(\omega^i)}{C(\omega^i) + \beta\omega^i + \gamma} \tag{6}$$

for some random challenges β, γ sent by the verifier, thanks to Lemma 4, we will have $A(\omega^i) = C(S_{f,A}(\omega^i))$ for all $i \in [n]$. The same can be done for $B(\omega^i) = C(S_{f,B}(\omega^i))$. Equation (6) can be proven using the same techniques we describe in the PFR for Marlin (see Appendix B). Finally, all of the zero checks are batched to improve efficiency. The complete protocol is depicted in Figure 3. Next, we state the theorem that summarizes the security of our rPHP. We omit the proof, as it is very similar to the one of Theorem 3.

Theorem 6. *The rPHP from Figure 3 is complete, sound and index-private. Furthermore, it is honest-verifier index-private with query bound $\mathsf{b} \in \mathbb{F}^{18}$ given by*

$$\mathsf{b} = (\mathsf{b}_L, \mathsf{b}_R, \mathsf{b}_M, \mathsf{b}_{f,A}, \mathsf{b}_{f,B}, \mathsf{b}_A, \mathsf{b}_B, \mathsf{b}_C, \mathsf{b}_{m_A}, \mathsf{b}_{m_B}, \mathsf{b}_{\mathsf{aux}}, \mathsf{b}_s, \mathsf{b}_{F_{A,1}}, \mathsf{b}_{F_{A,2}}, \mathsf{b}_{F_{B,1}}, \mathsf{b}_{F_{B,2}}, \mathsf{b}_r, \mathsf{b}_q)$$

The same optimization from Remark 3 that reduces the degree of the polynomials in the rPHP by b_q also holds in this case.

7.4 Instantiating our Compiler: FHFCs from Plonk

We instantiate the compiler following the same blueprint we explained in the Marlin case. For this reason, we skip some of the details.

The compiling commitment scheme. Recall that the compiling commitment scheme must contain the types $\{\texttt{swh}, \texttt{hid}\}$. For the type $\texttt{swh}$ we use the KZG commitment, and to commit to the relation $\mathsf{R} = (q_L, q_R, q_M, f) \in \mathcal{D}_{\mathsf{func}}$, we send KZG commitments to the polynomials output by $\mathcal{I}(\mathsf{R}, \mathsf{r})$ for $\mathsf{r} \leftarrow \mathsf{Rand}$. Again, the commitment key required by our compiling commitment scheme is $\mathsf{ck} = \big(\{[\tau^i]_1\}_{i \in [0,\mathsf{D}]}, [1, \tau]_2\}\big)$.

The CP-SNARKs $\mathsf{CP}_{\mathsf{opn}}$ and $\mathsf{CP}_{\mathsf{php}}$. Once more, we use the CP-SNARKs $\mathsf{CP}_{\mathsf{opn}}$ and $\mathsf{CP}_{\mathsf{php}}$ from [CFF+21]. Given that we will analyze our protocol in the Algebraic Group Model, the proof of opening is the empty string.

Remark 6 (Further degree optimizations)*.* Our compiler requires that $\mathsf{CP}_{\mathsf{php}}$ is (b, C)-leaky zero-knowledge for a $(\mathsf{b} + \mathbf{1})$-index-private rPHP. To prove the verification equation of the

$\mathcal{P}(\mathbb{F}, \mathbb{H}, q_L, q_R, q_M, f, x, y, w, \mathsf{r})$ $\qquad$ $\mathcal{V}^{q_L, q_R, q_M, S_{f,A}, S_{f,B}}(\mathbb{F}, \mathbb{H}, x, y)$

Sample $\rho_A(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_A + \mathsf{b}_q - 1}[X]$ and define $A(X) := \sum_{i=1}^n w_i \lambda_i(X) + \rho_A(X) z_{\mathbb{H}}(X)$
Sample $\rho_B(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_B + \mathsf{b}_q - 1}[X]$ and define $B(X) := \sum_{i=1}^n w_i \lambda_{n+i}(X) + \rho_B(X) z_{\mathbb{H}}(X)$
Sample $\rho_C(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_C + \mathsf{b}_q - 1}[X]$ and define $C(X) := \sum_{i=1}^n w_{2n+i} \lambda_i(X) + \rho_C(X) z_{\mathbb{H}}(X)$
Sample $q_s(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_s + \mathsf{b}_q - 1}[X],\ r_s(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_r + \mathsf{b}_q}[X]$ and define
$s(X) := q_s(X) z_{\mathbb{H}}(X) + X r_s(X)$
Sample $\rho_{\mathsf{aux}}(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_{\mathsf{aux}} + \mathsf{b}_q - 1}[X]$ and define
$\mathsf{aux}(X) := \sum_{i=1}^n A(\omega^i) B(\omega^i) \lambda_i(X) + \rho_{\mathsf{aux}}(X) z_{\mathbb{H}}(X)$
For $Z = A, B$, compute $v_Z \in \mathbb{F}^n$ such that $\sum_{i=1}^n \frac{1}{Z(\omega^i) + X S_{f,Z}(\omega^i) + Y} = \sum_{i=1}^n \frac{v_{Z,i}}{C(\omega^i) + X\omega^i + Y}$
For $Z = A, B$, sample $\rho_{m_Z}(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_{m_Z} + \mathsf{b}_q - 1}[X]$ and define
$m_Z(X) := \sum_{i=1}^n v_{Z,i} \lambda_i(X) + \rho_{m_Z} z_{\mathbb{H}}(X)$

$$\xrightarrow{\quad A(X), B(X), C(X), m_A(X), m_B(X), \mathsf{aux}(X), s(X) \quad}$$

$$\xleftarrow{\quad \beta, \gamma \in \mathbb{F} \quad} \qquad \beta, \gamma \leftarrow \mathbb{F}$$

For $Z = A, B$, sample $\rho_{F_{Z,j}}(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_{F_{Z,j}} + \mathsf{b}_q - 1}[X]$ and $\rho_{F_{Z,2}}(X) \leftarrow \mathbb{F}^{\leq \mathsf{b}_{F_{Z,2}} + \mathsf{b}_q - 1}[X]$
Define $F_{Z,1}(X) := \sum_{i=1}^n \frac{1}{Z(\omega^i) + \beta S_{f,Z}(\omega^i) + \gamma)} + \rho_{F_{Z,1}}(X) z_{\mathbb{H}}(X)$
Define $F_{Z,2}(X) := \sum_{i=1}^n \frac{m_Z(\omega^i)}{C(\omega^i) + \beta\omega^i + \gamma)} + \rho_{F_{Z,2}}(X) z_{\mathbb{H}}(X)$

$$\xrightarrow{\quad F_{A,1}(X),\ F_{A,2}(X),\ F_{B,1}(X),\ F_{B,2}(X) \quad}$$

$$\xleftarrow{\quad \eta \in \mathbb{F} \quad} \qquad \eta \leftarrow \mathbb{F}$$

Define $\mathsf{gates}(X) := q_L(X)A(X) + q_R(X)B(X) + q_M(X)\mathsf{aux}(X) - C(X) - \mathsf{PI}(X)$
Find $q(X)$ and $r(X)$ such that

$$\begin{aligned}
&\mathsf{gates}(X) + \eta\big(\mathsf{aux}(X) - A(X)B(X)\big) + \eta^2 z_{\mathbb{H}[\leq n-s]}(X)(C(X) - \mathsf{PO}(X)) \\
&+\eta^3\big(F_{A,1}(X)\big(A(X) + \beta S_{f,A}(X) + \gamma\big) - 1\big) + \eta^4\big(F_{A,2}(X)\big(C(X) + \beta X + \gamma\big) - m_A(X)\big) \\
&+\eta^5\big(F_{B,1}(X)\big(B(X) + \beta S_{f,B}(X) + \gamma\big) - 1\big) + \eta^6\big(F_{B,2}(X)\big(C(X) + \beta X + \gamma\big) - m_B(X)\big) \\
&+\eta^7\big(F_{A,1}(X) + F_{A,2}(X) + F_{B,1}(X) + F_{B,2}(X) + Xr(X)\big) + \eta^8 s(X) = q(X) z_{\mathbb{H}}(X)
\end{aligned}$$

$$\xrightarrow{\quad q(X),\ r(X) \quad}$$

Verifier's checks

$$\deg(A, B, C, m_A, m_B, \mathsf{aux}, s, F_{A,1}, F_{A,2}, F_{B,1}, F_{B,2}, q) \overset{?}{\leq} \mathsf{D}, \qquad \deg(r) \overset{?}{\leq} n - 2$$

$$\begin{aligned}
&\mathsf{gates}(X) + \eta\big(\mathsf{aux}(X) - A(X)B(X)\big) + \eta^2 z_{\mathbb{H}[\leq n-s]}(X)(C(X) - \mathsf{PO}(X)) \\
&+ \eta^3\big(F_{A,1}(X)\big(A(X) + \beta S_{f,A}(X) + \gamma\big) - 1\big) + \eta^4\big(F_{A,2}(X)\big(C(X) + \beta X + \gamma\big) - m_A(X)\big) \\
&+ \eta^5\big(F_{B,1}(X)\big(B(X) + \beta S_{f,B}(X) + \gamma\big) - 1\big) + \eta^6\big(F_{B,2}(X)\big(C(X) + \beta X + \gamma\big) - m_B(X)\big) \\
&+ \eta^7\big(F_{A,1}(X) + F_{A,2}(X) + F_{B,1}(X) + F_{B,2}(X) + Xr(X)\big) + \eta^8 s(X) \overset{?}{=} q(X) z_{\mathbb{H}}(X)
\end{aligned}$$

Figure 3: Our rPHP for the Generalized Plonk constraint system. We denote by D the maximum degree of all the polynomials involved in the protocol.

rPHP from Figure 3, we only need to reveal evaluations of $A(X)$, $B(X)$, $\mathsf{aux}(X)$, $F_{A,1}(X)$, $F_{A,2}(X)$, $F_{B,1}(X)$ and $F_{B,2}(X)$. Therefore, our concrete $\mathsf{b} \in \mathbb{N}^{18}$ is given by

$$\begin{aligned}\mathsf{b} = (&\mathsf{b}_L = 0, \mathsf{b}_R = 0, \mathsf{b}_M = 0, \mathsf{b}_{f,A} = 0, \mathsf{b}_{f,B} = 0, \mathsf{b}_A = 1, \mathsf{b}_B = 1, \mathsf{b}_C = 0, \mathsf{b}_{m_A} = 0,\\ &\mathsf{b}_{m_B} = 0, \mathsf{b}_{\mathsf{aux}} = 1, \mathsf{b}_s = 0, \mathsf{b}_{F_{A,1}} = 1, \mathsf{b}_{F_{A,2}} = 1, \mathsf{b}_{F_{B,1}} = 1, \mathsf{b}_{F_{B,2}} = 1, \mathsf{b}_r = 0, \mathsf{b}_q = 0).\end{aligned}$$

Since all of the polynomials are evaluated at the trapdoor τ, we can use the optimization from Remark 3 to decrease the degree of the polynomials involved in the protocol and still meet the $\mathsf{b} + \mathbf{1}$ bound for the rPHP with the concrete b from above. In particular, as all of the index polynomials are only evaluated at the trapdoor τ, we set the randomness space to be $\mathsf{Rand} = \mathbb{F}^5$. Moreover, the maximum degree of the protocol is $\mathsf{D} = n + 1$.

The CP-SNARKs $\mathsf{CP}_{\mathsf{PFR}}$ and $\mathsf{CP}_{\mathsf{link}}$. The function relation can be written as

$$\hat{\mathcal{R}}_{\mathsf{func}}(n, \ell, s) = \left\{ \begin{pmatrix} (c_L, c_R, c_M, c_{f,A}, c_{f,B}), \\ (q_L, q_R, q_M, S_{f,A}, S_{f,B}) \end{pmatrix} \;\middle|\; \begin{array}{l} q_L(\omega^i) = q_M(\omega^i) = q_R(\omega^i) = 0,\ i \in [\ell] \\ \log_\omega(S_{f,A}(\omega^i)) \le i,\ i \in [n] \\ \log_\omega(S_{f,B}(\omega^i)) \le i,\ i \in [n] \\ c_L = [q_L(\tau)]_1,\ c_R = [q_R(\tau)]_1, \\ c_M = [q_M(\tau)]_1,\ c_{f,A} = [S_{f,A}(\tau)]_1, \\ c_{f,B} = [S_{f,B}(\tau)]_1, \end{array} \right\}.$$

We see that building a CP-SNARK for this relation essentially comes down to doing a discrete logarithm comparison, just like in the Marlin case. For this reason, we omit the explicit PFR. The proof consists of 12 elements of $\mathbb{G}_1$ and 3 elements of $\mathbb{F}$ (it is the same as in the Marlin PFR except we are saving the commitment to $\widetilde{\mathsf{row}}(X)$ and its evaluation). The prover is quasilinear in n; the verifier is logarithmic in n and needs to perform 2 pairings.

Remark 7 (Optimizing the compiled protocol). Note that an accepting proof π_{PFR} for the relation $\hat{\mathcal{R}}_{\mathsf{func}}(n, \ell, s)$ implies that the commitment to the relation R also opens to the polynomials output by the randomized indexer. Since we only need to evaluate those polynomials at the trapdoor τ, we can apply the same optimization we described in Remark 5. Specifically, the prover does not need to send commitments to new index polynomials in each execution of the protocol and can skip proving the linking relation.

Overall efficiency of our Plonk-based Function-Hiding Functional Commitment. The commitment to the relation $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}(n, \ell, s)$ consists of 5 elements in $\mathbb{G}_1$. Thanks to the previous remark, the final proof for our protocol is $\pi = (\pi_{\mathsf{PFR}}, \pi_{\mathsf{R}})$, where π_{R} is the proof that results from compiling the rPHP from Figure 3. There, the prover sends 13 polynomials and after compilation, the proof consists of 14 elements in $\mathbb{G}_1$ and 7 field elements. The field elements come from the evaluation of seven polynomials and the additional group elements is the evaluation proof (we are again applying the optimization for the degree checks). Therefore, the proof size of π_{R} at the 128-bit security level, when instantiating the protocol with the BLS12-381 curve, results in 0.896 kB. The final proof $\pi = (\pi_{\mathsf{PFR}}, \pi_{\mathsf{R}})$ ends at 1.568 kB. The prover is quasilinear in n; the verifier is logarithmic in n and needs to perform 4 pairings, which with standard batching tricks can be reduced to just 2 pairings.

Comparison with standard Plonk. Let us briefly forget about the PFR and see what would happen if we were to compile our protocol with an rPHP for the standard Plonk constraint system. It can be shown that the original Plonk PHP is index-private without making any further modifications if we define a randomized encoder similar to the one from Section 7.3.1. Hence, a Plonk proof after compilation results in 9 elements in $\mathbb{G}_1$ and 6 elements in $\mathbb{F}$. However, Plonk requires evaluating the three index polynomials that

encode the permutation in a random challenge sent by the verifier, so we cannot apply the same optimizations from Remark 7. Namely, the prover would need to send three different polynomials encoding the permutation and prove the linking relation on each execution of the protocol. Moreover, in this case, the commitment to the relation would consist of 6 polynomials, whereas in our case it is 5. This shows that, although our rPHP involves sending more polynomials than the one of [GWC19], after compilation, this results in little overhead. The main advantage of our constraint system is that we can design a simpler PFR, which results in a more efficient protocol overall.

References

[ac22] arkworks contributors. `arkworks` zksnark ecosystem, 2022. URL: https://arkworks.rs.

[BNO21] Dan Boneh, Wilson Nguyen, and Alex Ozdemir. Efficient functional commitments: How to commit to private functions. Cryptology ePrint Archive, Report 2021/1342, 2021. URL: https://eprint.iacr.org/2021/1342.

[CF13] Dario Catalano and Dario Fiore. Vector commitments and their applications. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013: 16th International Conference on Theory and Practice of Public Key Cryptography*, volume 7778 of *Lecture Notes in Computer Science*, pages 55–72. Springer, Berlin, Heidelberg, February / March 2013. doi:10.1007/978-3-642-36362-7_5.

[CFF+20] Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. Lunar: a toolbox for more efficient universal and updatable zkSNARKs and commit-and-prove extensions. Cryptology ePrint Archive, Report 2020/1069, 2020. URL: https://eprint.iacr.org/2020/1069.

[CFF+21] Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. Lunar: A toolbox for more efficient universal and updatable zkSNARKs and commit-and-prove extensions. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part III*, volume 13092 of *Lecture Notes in Computer Science*, pages 3–33. Springer, Cham, December 2021. doi:10.1007/978-3-030-92078-4_1.

[CFF+24] Matteo Campanelli, Antonio Faonio, Dario Fiore, Tianyu Li, and Helger Lipmaa. Lookup arguments: Improvements, extensions and applications to zero-knowledge decision trees. In Qiang Tang and Vanessa Teague, editors, *PKC 2024: 27th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 14602 of *Lecture Notes in Computer Science*, pages 337–369. Springer, Cham, April 2024. doi:10.1007/978-3-031-57722-2_11.

[CFQ19] Matteo Campanelli, Dario Fiore, and Anaïs Querol. LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 2075–2092. ACM Press, November 2019. doi:10.1145/3319535.3339820.

[CHM+20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part I*, volume 12105 of *Lecture Notes in*

Computer Science, pages 738–768. Springer, Cham, May 2020. doi:10.1007/978-3-030-45721-1_26.

[COS20] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. Fractal: Post-quantum and transparent recursive proofs from holography. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 769–793. Springer, Cham, May 2020. doi:10.1007/978-3-030-45721-1_27.

[DXNT24] Zijing Di, Lucas Xia, Wilson D. Nguyen, and Nirvan Tyagi. MuxProofs: Succinct arguments for machine computation from vector lookups. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part V*, volume 15488 of *Lecture Notes in Computer Science*, pages 236–265. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0935-2_8.

[EG14] Alex Escala and Jens Groth. Fine-tuning Groth-Sahai proofs. In Hugo Krawczyk, editor, *PKC 2014: 17th International Conference on Theory and Practice of Public Key Cryptography*, volume 8383 of *Lecture Notes in Computer Science*, pages 630–649. Springer, Berlin, Heidelberg, March 2014. doi:10.1007/978-3-642-54631-0_36.

[FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part II*, volume 10992 of *Lecture Notes in Computer Science*, pages 33–62. Springer, Cham, August 2018. doi:10.1007/978-3-319-96881-0_2.

[GPS25] Chaya Ganesh, Sikhar Patranabis, and Nitin Singh. Samaritan: Linear-time prover SNARK from new multilinear polynomial commitments. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology - ASIACRYPT 2025, Part V*, volume 16249 of *Lecture Notes in Computer Science*, pages 456–489. Springer Nature Singapore, December 2025. doi:10.1007/978-981-95-5116-3_15.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. URL: https://eprint.iacr.org/2019/953.

[Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022. URL: https://eprint.iacr.org/2022/1530.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194. Springer, Berlin, Heidelberg, December 2010. doi:10.1007/978-3-642-17373-8_11.

[LRY16] Benoît Libert, Somindu C. Ramanna, and Moti Yung. Functional commitment schemes: From polynomial commitments to pairing-based accumulators from simple assumptions. In Ioannis Chatzigiannakis, Michael Mitzenmacher, Yuval Rabani, and Davide Sangiorgi, editors, *ICALP 2016: 43rd International Colloquium on Automata, Languages and Programming*, volume 55 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 30:1–30:14. Schloss Dagstuhl, July 2016. doi:10.4230/LIPIcs.ICALP.2016.30.

A Additional Material from Section 5

A.1 Interactive Arguments

Definition 28 (Commit-and-Prove Interactive Arguments)**.** A *Commit-and-Prove Interactive Argument* is a tuple $\mathsf{iCP} = (\mathsf{KeyGen}, \mathbb{P}, \mathbb{V})$ of PT algorithms where all the algorithms work as in CP-SNARKs (see Section 2.6) but where $\mathbb{P}$ and $\mathbb{V}$ form an interactive protocol. In particular:

- $\mathbb{P}(\mathsf{ek}, \hat{\mathsf{x}} = (\mathsf{ck}, \mathsf{x}, (\hat{c}_j)_{j\in[\ell]}), \hat{\mathsf{w}} = ((w_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]}), \bar{\rho}_1, \ldots, \bar{\rho}_i)$: the next-message function of $\mathbb{P}$ takes as input the evaluation key ek, the instance $\hat{\mathsf{x}}$, the witness $\hat{\mathsf{w}}$ and the first i-th messages $\bar{\rho}_1, \ldots, \bar{\rho}_i$ from $\mathbb{V}$.
- $\mathbb{V}(\mathsf{vk}, \hat{\mathsf{x}}, \pi_1, \ldots, \pi_i)$: the next-message function of $\mathbb{V}$ takes as input the verification key vk, the instance $\hat{\mathsf{x}}$ and the first i-th messages $\pi_1, \ldots, \pi_i$ from the prover $\mathbb{P}$. At the last round of interaction the verifier outputs a decision bit b.

The security definitions for commit-and-prove interactive arguments are almost identical to the ones for CP-SNARKs. The main difference is that we need to substitute the proof π with a transcript of the protocol tr. We give the explicit definition of knowledge soundness next, as we will use it in the next section; zero-knowledge is analogous.

Definition 29. We say iCP is *knowledge sound* if for every PPT adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$, there exists an efficient extractor $\mathcal{E}$ such that $\Pr[\mathsf{Game}_{\mathcal{A},\mathcal{E}}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$, where

$\mathsf{Game}_{\mathcal{A},\mathcal{E}}(1^\lambda) \to b'$

$\mathsf{ck} \leftarrow \mathsf{CS.Setup}(1^\lambda)$; $\mathsf{srs} := (\mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(\mathsf{ck})$; $(\hat{\mathsf{x}}, st) \leftarrow \mathcal{A}_0(\hat{\mathcal{R}}, \mathsf{srs})$

$(\mathsf{tr}, b) \leftarrow \langle \mathcal{A}_1(st), \mathbb{V}(\mathsf{vk}, \hat{\mathsf{x}})\rangle$; $\hat{\mathsf{w}} \leftarrow \mathcal{E}(\mathsf{srs})$; $b_E \leftarrow \hat{\mathcal{R}}(\hat{\mathsf{x}}, \hat{\mathsf{w}})$; **return** $b' = b \wedge \neg b_E$

A.2 Proof of Theorem 1

Proof. Let $\mathbf{H}_0$ be the knowledge soundness game. We define the extractor of the interactive argument as the extractor of $\mathsf{CP}_{\mathsf{PFR}}$. We define a sequence of hybrids $\mathbf{H}_1, \ldots \mathbf{H}_5$ in Figure 4 and Figure 5. We write $\mathbf{H}_i = 1$ if and only if $b^i_W = 1 \wedge b_E = 0$.

- Consider the hybrid $\mathbf{H}_1$. It is a modification of $\mathbf{H}_0$, where now it is also checked if the extracted $(\mathsf{R}, \boldsymbol{o})$ satisfies $(\boldsymbol{c}, (\mathsf{R}, \boldsymbol{o})) \in \hat{\mathcal{R}}_{\mathsf{func}}$. We have

  $$\Pr[b = 1 \wedge b_E = 0] = \Pr[b = 1 \wedge b_E = 0 \wedge b_{\mathsf{PFR}} = 1] + \Pr[b = 1 \wedge b_E = 0 \wedge b_{\mathsf{PFR}} = 0],$$

  where we are using the fact that for any two events A and B, $\Pr[A] = \Pr[A \wedge B] + \Pr[A \wedge \neg B]$. Since, $\Pr[\mathbf{H}_0 = 1] = \Pr[b = 1 \wedge b_E = 0]$, we deduce

  $$\Pr[\mathbf{H}_0 = 1] \leq \Pr[b = 1 \wedge b_E = 0 \wedge b_{\mathsf{PFR}} = 1] + \Pr[b = 1 \wedge b_{\mathsf{PFR}} = 0].$$

  As $b = 1$, the verifier accepts; more concretely, it accepts the proof π_{PFR}, but $b_{\mathsf{PFR}} = 0$ means $(\boldsymbol{c}, (\mathsf{R}, \boldsymbol{o})) \notin \hat{\mathcal{R}}_{\mathsf{func}}$. Thanks to the knowledge soundness of $\mathsf{CP}_{\mathsf{PFR}}$, we have that $\Pr[b = 1 \wedge b_{\mathsf{PFR}} = 0] \leq \epsilon_{\mathsf{PFR}}$, which implies that

  $$\Pr[\mathbf{H}_0 = 1] \leq \Pr[\mathbf{H}_1 = 1] + \epsilon_{\mathsf{PFR}}.$$

- Consider the hybrid $\mathbf{H}_2$, where we additionally run the extractor for $\mathsf{CP}_{\mathsf{link}}$ and check if the extracted values $(\mathsf{R}', \boldsymbol{o}', \boldsymbol{p}^*, \boldsymbol{o}^*, \mathsf{r})$ satisfy $((\boldsymbol{c}, \boldsymbol{c}_0), (\mathsf{R}', \boldsymbol{o}', \boldsymbol{p}^*, \boldsymbol{o}^*, \mathsf{r})) \in \hat{\mathcal{R}}_{\mathsf{link}}$. Using the same reasoning as above, $\Pr[\mathbf{H}_1 = 1] \leq \Pr[\mathbf{H}_2 = 1] + \Pr[b = 1 \wedge b_{\mathsf{link}} = 0]$. Thanks to the knowledge soundness of $\mathsf{CP}_{\mathsf{link}}$, we deduce that $\Pr[\mathbf{H}_1 = 1] \leq \Pr[\mathbf{H}_2 = 1] + \epsilon_{\mathsf{link}}$.

- Consider the hybrid $\mathbf{H}_3$, where we additionally run the extractor for $\mathsf{CP}_{\mathsf{php}}$ and check if the extracted values $(p'_j, o'_j)_j$ satisfy $(c_j, (p'_j, o'_j))_j \in \hat{\mathcal{R}}_{\mathsf{php}}$. Again, $\Pr[\mathbf{H}_2 = 1] \leq \Pr[\mathbf{H}_3 = 1] + \Pr[b = 1 \wedge b_{\mathsf{php}} = 0]$. Thanks to the knowledge soundness of $\mathsf{CP}_{\mathsf{php}}$, we deduce that $\Pr[\mathbf{H}_2 = 1] \leq \Pr[\mathbf{H}_3 = 1] + \epsilon_{\mathsf{php}}$.

- Consider the hybrid $\mathbf{H}_4$, where we additionally run the extractor for $\mathsf{CP}_{\mathsf{opn}}$ and check if the extracted values $(p_{i,j}, o_{i_j})_{i,j}$ satisfy $(c_{i,j}, (p_{i,j}, o_{i_j}))_{i,j} \in \hat{\mathcal{R}}_{\mathsf{opn}}$. We have $\Pr[\mathbf{H}_3 = 1] \leq \Pr[\mathbf{H}_4 = 1] + \Pr[\exists i \mid b^i_{\mathsf{opn}} = 0] \leq \Pr[\mathbf{H}_4 = 1] + \alpha \max_i \Pr[b = 1 \wedge b^i_{\mathsf{opn}} = 0]$, where $\alpha \in \mathbb{F}$ is the amount of commitments sent in the protocol. Thanks to the knowledge soundness of $\mathsf{CP}_{\mathsf{opn}}$, we deduce that $\Pr[\mathbf{H}_3 = 1] \leq \Pr[\mathbf{H}_4 = 1] + \alpha\epsilon_{\mathsf{opn}}$.

- Consider the hybrid $\mathbf{H}_5$, where we additionally check if all of the extracted values so far are consistent with each other. Again, $\Pr[\mathbf{H}_4 = 1] \leq \Pr[\mathbf{H}_5 = 1] + \Pr[b = 1 \wedge b_{\mathsf{CS}} = 0]$. Since the commitment scheme is binding, we deduce that $\Pr[\mathbf{H}_4 = 1] \leq \Pr[\mathbf{H}_5 = 1] + \epsilon_{\mathsf{CS}}$.

- For any i, j, let $ind(i, j) := \sum_{i' < i} \mathsf{n}^*(i') + j$ and let ind^{-1} be its inverse in the domain $[\mathsf{n}^*]$. The function ind re-indexes the j-th polynomial sent in the i-th round as the $ind(i, j)$-th polynomial sent by the rPHP prover. Similar to [CFF+20], we can define a cheating prover for rPHP as in Figure 5 such that

$$\Pr[\langle \mathcal{P}^*, \mathcal{V}^{\boldsymbol{p}_0}(\mathsf{x}, \mathsf{y})\rangle = 1] = \Pr[b^5_W = 1].$$

  Since $\mathbf{H}_5 = 1$ if and only if $b^5_W = 1 \wedge b_E = 0$, we have that the extracted R satisfies $\mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}, \boldsymbol{o}) = 1$ and $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}$ (in particular R is a relation), so $b_E = 0$ must be because $(x, y) \notin \mathcal{L}(\mathsf{R})$. Moreover, $b^5_W = 1$ also implies $\boldsymbol{p}_0 = \mathcal{I}(\mathsf{R}, \mathsf{r})$. Then, thanks to the soundness of rPHP, we deduce that

$$\Pr[\mathbf{H}_5 = 1] = \Pr[\langle \mathcal{P}^*, \mathcal{V}^{\boldsymbol{p}_0}(x, y)\rangle = 1 \ \wedge \ (x, y) \notin \mathcal{L}(\mathsf{R})] \leq \epsilon_{\mathsf{rPHP}}.$$

Putting it all together, we conclude that $\Pr[\mathbf{H}_0 = 1] \leq \epsilon_{\mathsf{PFR}} + \epsilon_{\mathsf{link}} + \epsilon_{\mathsf{php}} + \alpha\epsilon_{\mathsf{opn}} + \epsilon_{\mathsf{CS}} + \epsilon_{\mathsf{rPHP}}$. □

B Explicit PFR for Marlin

We recall the logarithmic derivative lemma [Hab22], which will be the basis for our protocol.

Lemma 5. *Let m, n be positive integers and let $\mathbb{F}$ be field of characteristic $p > \max(n, m)$. Let $(a_i)_{i=1}^m$, $(b_j)_{j=1}^n$ be sequences on $\mathbb{F}$, with $b_j \neq b_{j'}$. Then $\{a_i\} \subseteq \{b_j\}$ as sets (with multiples of values removed) if and only if there exists a sequence $(m_j)_{j=1}^m$ (the multiplicities of each a_i in (b_j)) such that*

$$\sum_{i=1}^{m} \frac{1}{X + a_i} = \sum_{j=1}^{n} \frac{m_j}{X + b_j}.$$

For ease of exposition, we will present the protocol as a public coin interactive argument, which can be compiled into a CP-SNARK using the Fiat-Shamir heuristic. The protocol closely follows the one of [CFF+24].

Key Generation. Recall that the commitment key is $\mathsf{ck} = \big(\{[\tau^i]_1\}_{i \in [0, m+1]}, [1, \tau]_2\}\big)$. The algorithm $\mathsf{KeyGen}_{\mathsf{PFR}}$ deterministically outputs the commitment key along with a

$\mathbf{H}_1(1^\lambda)$

$\mathsf{ck} \leftarrow \mathsf{CS.Setup}(1^\lambda)\,;\;\; \mathsf{srs} := (\mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(\mathsf{ck})\,;\;\; (\hat{\mathsf{x}} = (\boldsymbol{c}, x, y), st) \leftarrow \mathcal{A}_0(\hat{\mathcal{R}}_{\mathsf{FHFC}}, \mathsf{srs})$

$(\mathsf{tr}, b) \leftarrow \langle \mathcal{A}_1(st), \mathbb{V}(\mathsf{vk}, \hat{\mathsf{x}})\rangle\,;\;\; (\mathsf{R}, \boldsymbol{o}) \leftarrow \mathcal{E}_{\mathsf{PFR}}(\mathsf{srs})\,;\;\; b_E \leftarrow \hat{\mathcal{R}}_{\mathsf{FHFC}}((\boldsymbol{c}, \mathsf{x}, \mathsf{y}), (\mathsf{R}, \boldsymbol{o}))$

$\boxed{b^{\mathsf{PFR}} \leftarrow \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}, \boldsymbol{o}) \wedge \mathsf{R} \in \mathcal{D}_f\,;\;\; b_W \leftarrow b \wedge b^{\mathsf{PFR}}}$

$\mathbf{H}_2(1^\lambda)$

$\mathsf{ck} \leftarrow \mathsf{CS.Setup}(1^\lambda)\,;\;\; \mathsf{srs} := (\mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(\mathsf{ck})\,;\;\; (\hat{\mathsf{x}} = (\boldsymbol{c}, x, y), st) \leftarrow \mathcal{A}_0(\hat{\mathcal{R}}_{\mathsf{FHFC}}, \mathsf{srs})$

$(\mathsf{tr}, b) \leftarrow \langle \mathcal{A}_1(st), \mathbb{V}(\mathsf{vk}, \hat{\mathsf{x}})\rangle\,;\;\; (\mathsf{R}, \boldsymbol{o}) \leftarrow \mathcal{E}_{\mathsf{PFR}}(\mathsf{srs})\,;\;\; b_E \leftarrow \hat{\mathcal{R}}_{\mathsf{FHFC}}((\boldsymbol{c}, \mathsf{x}, \mathsf{y}), (\mathsf{R}, \boldsymbol{o}))$

$b^{\mathsf{PFR}} \leftarrow \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}, \boldsymbol{o}) \wedge \mathsf{R} \in \mathcal{D}_f\,;\;\; \boxed{(\mathsf{R}', \boldsymbol{o}', \boldsymbol{p}^*, \boldsymbol{o}^*, \mathsf{r}) \leftarrow \mathcal{E}_{\mathsf{link}}(\mathsf{srs})}$

$\boxed{b^{\mathsf{link}} \leftarrow \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}', \boldsymbol{o}') \wedge \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}_0, \boldsymbol{p}^*, \boldsymbol{o}^*) \wedge \mathcal{I}(\mathsf{R}', \mathsf{r}) = \boldsymbol{p}^*}$

$\boxed{b_W \leftarrow b \wedge b^{\mathsf{PFR}} \wedge b^{\mathsf{link}}}$

$\mathbf{H}_3(1^\lambda)$

$\mathsf{ck} \leftarrow \mathsf{CS.Setup}(1^\lambda)\,;\;\; \mathsf{srs} := (\mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(\mathsf{ck})\,;\;\; (\hat{\mathsf{x}} = (\boldsymbol{c}, x, y), st) \leftarrow \mathcal{A}_0(\hat{\mathcal{R}}_{\mathsf{FHFC}}, \mathsf{srs})$

$(\mathsf{tr}, b) \leftarrow \langle \mathcal{A}_1(st), \mathbb{V}(\mathsf{vk}, \hat{\mathsf{x}})\rangle\,;\;\; (\mathsf{R}, \boldsymbol{o}) \leftarrow \mathcal{E}_{\mathsf{PFR}}(\mathsf{srs})\,;\;\; b_E \leftarrow \hat{\mathcal{R}}_{\mathsf{FHFC}}((\boldsymbol{c}, \mathsf{x}, \mathsf{y}), (\mathsf{R}, \boldsymbol{o}))$

$b^{\mathsf{PFR}} \leftarrow \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}, \boldsymbol{o}) \wedge \mathsf{R} \in \mathcal{D}_f\,;\;\; (\mathsf{R}', \boldsymbol{o}', \boldsymbol{p}^*, \boldsymbol{o}^*, \mathsf{r}) \leftarrow \mathcal{E}_{\mathsf{link}}(\mathsf{srs})$

$b^{\mathsf{link}} \leftarrow \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}', \boldsymbol{o}') \wedge \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}_0, \boldsymbol{p}^*, \boldsymbol{o}^*) \wedge \mathcal{I}(\mathsf{R}', \mathsf{r}) = \boldsymbol{p}^*$

$\boxed{(p'_j, o'_j)_{j \in [\mathsf{n}^*]} \leftarrow \mathcal{E}_{\mathsf{php}}(\mathsf{srs})}$

$\boxed{\begin{aligned} b^{\mathsf{php}} \leftarrow & \textstyle\bigwedge_{k \in [\mathsf{n}_e]} (G_k(X, \{p'_j(v_{k,j}(X))\}_{j \in [\mathsf{n}^*]}, \{\pi_j\}_{j \in [m^*]}) \equiv 0) \\ & \wedge \textstyle\bigwedge_{j \in [\mathsf{n}^*]} (\deg(p'_j) \leq d_j) \wedge \bigwedge_{j \in [\mathsf{n}^*]} \mathsf{VerCom}(\mathsf{ck}, c_j, p'_j, o'_j) \end{aligned}}$

$\boxed{b_W \leftarrow b \wedge b^{\mathsf{PFR}} \wedge b^{\mathsf{link}} \wedge b^{\mathsf{php}}}$

$\mathbf{H}_4(1^\lambda)$

$\mathsf{ck} \leftarrow \mathsf{CS.Setup}(1^\lambda)\,;\;\; \mathsf{srs} := (\mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(\mathsf{ck})\,;\;\; (\hat{\mathsf{x}} = (\boldsymbol{c}, x, y), st) \leftarrow \mathcal{A}_0(\hat{\mathcal{R}}_{\mathsf{FHFC}}, \mathsf{srs})$

$(\mathsf{tr}, b) \leftarrow \langle \mathcal{A}_1(st), \mathbb{V}(\mathsf{vk}, \hat{\mathsf{x}})\rangle\,;\;\; (\mathsf{R}, \boldsymbol{o}) \leftarrow \mathcal{E}_{\mathsf{PFR}}(\mathsf{srs})\,;\;\; b_E \leftarrow \hat{\mathcal{R}}_{\mathsf{FHFC}}((\boldsymbol{c}, \mathsf{x}, \mathsf{y}), (\mathsf{R}, \boldsymbol{o}))$

$b^{\mathsf{PFR}} \leftarrow \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}, \boldsymbol{o}) \wedge \mathsf{R} \in \mathcal{D}_f\,;\;\; (\mathsf{R}', \boldsymbol{o}', \boldsymbol{p}^*, \boldsymbol{o}^*, \mathsf{r}) \leftarrow \mathcal{E}_{\mathsf{link}}(\mathsf{srs})$

$b^{\mathsf{link}} \leftarrow \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}', \boldsymbol{o}') \wedge \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}_0, \boldsymbol{p}^*, \boldsymbol{o}^*) \wedge \mathcal{I}(\mathsf{R}', \mathsf{r}) = \boldsymbol{p}^*$

$(p'_j, o'_j)_{j \in [\mathsf{n}^*]} \leftarrow \mathcal{E}_{\mathsf{php}}(\mathsf{srs})$

$\begin{aligned} b^{\mathsf{php}} \leftarrow & \textstyle\bigwedge_{k \in [\mathsf{n}_e]} (G_k(X, \{p'_j(v_{k,j}(X))\}_{j \in [\mathsf{n}^*]}, \{\pi_j\}_{j \in [m^*]}) \equiv 0) \\ & \wedge \textstyle\bigwedge_{j \in [\mathsf{n}^*]} (\deg(p'_j) \leq d_j) \wedge \bigwedge_{j \in [\mathsf{n}^*]} \mathsf{VerCom}(\mathsf{ck}, c_j, p'_j, o'_j) \end{aligned}$

$\boxed{\textbf{for } i \in [0, \mathsf{rnd}] : (p_{i,j}, o_{i,j})_{j \in [\mathsf{n}(i)]} \leftarrow \mathcal{E}_{\mathsf{opn},j}(\mathsf{srs})\,;\;\; b^i \leftarrow \textstyle\bigwedge_{j \in [\mathsf{n}(i)]} \mathsf{VerCom}(\mathsf{ck}, c_{i,j}, p_{i,j}, o_{i,j})}$

$\boxed{b_W \leftarrow b \wedge b^{\mathsf{PFR}} \wedge b^{\mathsf{link}} \wedge b^{\mathsf{php}} \wedge (\wedge_i b^i)}$

Figure 4: First 4 hybrids

$\mathbf{H}_5(1^\lambda)$

$\mathsf{ck} \leftarrow \mathsf{CS.Setup}(1^\lambda)$; $\mathsf{srs} := (\mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(\mathsf{ck})$; $(\hat{\mathsf{x}} = (\boldsymbol{c}, x, y), st) \leftarrow \mathcal{A}_0(\hat{\mathcal{R}}_{\mathsf{FHFC}}, \mathsf{srs})$

$(\mathsf{tr}, b) \leftarrow \langle \mathcal{A}_1(st), \mathbb{V}(\mathsf{vk}, \hat{\mathsf{x}}) \rangle$; $(\mathsf{R}, \boldsymbol{o}) \leftarrow \mathcal{E}_{\mathsf{PFR}}(\mathsf{srs})$; $b_E \leftarrow \hat{\mathcal{R}}_{\mathsf{FHFC}}((\boldsymbol{c}, \mathsf{x}, \mathsf{y}), (\mathsf{R}, \boldsymbol{o}))$

$b^{\mathsf{PFR}} \leftarrow \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}, \boldsymbol{o}) \wedge \mathsf{R} \in \mathcal{D}_f$; $(\mathsf{R}', \boldsymbol{o}', \boldsymbol{p}^*, \boldsymbol{o}^*, \mathsf{r}) \leftarrow \mathcal{E}_{\mathsf{link}}(\mathsf{srs})$

$b^{\mathsf{link}} \leftarrow \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}, \mathsf{R}', \boldsymbol{o}') \wedge \mathsf{VerCom}(\mathsf{ck}, \boldsymbol{c}_0, \boldsymbol{p}^*, \boldsymbol{o}^*) \wedge \mathcal{I}(\mathsf{R}', \mathsf{r}) = \boldsymbol{p}^*$

$(p'_j, o'_j)_{j \in [\mathsf{n}^*]} \leftarrow \mathcal{E}_{\mathsf{php}}(\mathsf{srs})$

$b^{\mathsf{php}} \leftarrow \bigwedge_{k \in [\mathsf{n}_e]} (G_k(X, \{p'_j(v_{k,j}(X))\}_{j \in [\mathsf{n}^*]}, \{\pi_j\}_{j \in [m^*]}) \equiv 0)$

$\quad \wedge \bigwedge_{j \in [\mathsf{n}^*]} (\deg(p'_j) \leq d_j) \wedge \bigwedge_{j \in [\mathsf{n}^*]} \mathsf{VerCom}(\mathsf{ck}, c_j, p'_j, o'_j)$

for $i \in [0, \mathsf{rnd}] : (p_{i,j}, o_{i,j})_{j \in [\mathsf{n}(i)]} \leftarrow \mathcal{E}_{\mathsf{opn},j}(\mathsf{srs})$; $b^i \leftarrow \bigwedge_{j \in [\mathsf{n}(i)]} \mathsf{VerCom}(\mathsf{ck}, c_{i,j}, p_{i,j}, o_{i,j})$

$\boxed{\textbf{let } (p_j)_{j \in [\mathsf{n}^*]} := (p_{i,j})_{i \in [0, \mathsf{rnd}], j \in [\mathsf{n}(i)]} \text{ ; } b_{\mathsf{CS}} \leftarrow \forall j \ (p_j = p'_j)}$

$\boxed{\textbf{let } \boldsymbol{p}_0 := (p_{0,j})_{j \in [\mathsf{n}(0)]} \text{ ; } b_{\mathsf{CS}} \leftarrow b_{\mathsf{CS}} \wedge (\boldsymbol{p}_0 = \boldsymbol{p}^*) \wedge (\mathsf{R} = \mathsf{R}')}$

$\boxed{b_W \leftarrow b \wedge b^{\mathsf{PFR}} \wedge b^{\mathsf{link}} \wedge b^{\mathsf{php}} \wedge (\wedge_i b^i) \wedge b_{\mathsf{CS}}}$

Prover $\mathcal{P}^*$:

$(\hat{\mathsf{x}} = (\boldsymbol{c}, x, y), st) \leftarrow \mathcal{A}_0(\hat{\mathcal{R}}_{\mathsf{FHFC}}, \mathsf{srs})$
if $i = 0$ **then :**
  $(\pi_{\mathsf{PFR}}, \boldsymbol{c}_0, \pi_{\mathsf{opn},0}, \pi_{\mathsf{link}}) \leftarrow \mathcal{A}_1(st)$
  if $\mathsf{Verify}_{\mathsf{opn}}(\mathsf{vk}_{\mathsf{opn}}, \boldsymbol{c}_0, \pi_{\mathsf{opn},0}) = 0$ **then return** $\perp$
  $(\mathsf{R}, \boldsymbol{o}) \leftarrow \mathcal{E}_{\mathsf{PFR}}(\mathsf{srs})$
  if $\mathsf{Verify}_{\mathsf{PFR}}(\mathsf{vk}_{\mathsf{PFR}}, \boldsymbol{c}, \pi_{\mathsf{PFR}}) = 0$ **then return** $\perp$
  $(\mathsf{R}', \boldsymbol{o}', \boldsymbol{p}^*, \boldsymbol{o}^*, \mathsf{r}) \leftarrow \mathcal{E}_{\mathsf{link}}(\mathsf{srs})$
  if $\mathsf{Verify}_{\mathsf{link}}(\mathsf{vk}_{\mathsf{link}}, \boldsymbol{c}, \boldsymbol{c}_0, \pi_{\mathsf{link}}) = 0$ **then return** $\perp$
  if $\mathsf{R} \neq \mathsf{R}'$ **then return** $\perp$
  $(\boldsymbol{p}_0, \boldsymbol{o}_0) \leftarrow \mathcal{E}_{\mathsf{opn},0}(\mathsf{srs})$
  if $\boldsymbol{p}_0 \neq \boldsymbol{p}^*$ **then return** $\perp$
if $1 \leq i \leq \mathsf{rnd}$ **then :**
  $((c_{i,j})_{j \in [\mathsf{n}(i)]}, \{\pi_{i,j}\}_{j \in [\mathsf{rnd}(i)]}, \pi_{\mathsf{opn},j}) \leftarrow \mathcal{A}_1(st, \rho_1, \ldots, \rho_i))$
  if $\mathsf{Verify}_{\mathsf{opn}}(\mathsf{vk}_{\mathsf{opn}}, \boldsymbol{c}_j, \pi_{\mathsf{opn},j}) = 0$ **then return** $\perp$
  $(p_{i,j}, o_{i,j})_{j \in [\mathsf{n}(i)]} \leftarrow \mathcal{E}_{\mathsf{opn},j}(\mathsf{srs})$
  if $\exists j : \mathsf{VerCom}(\mathsf{ck}, c_{i,j}, p_{i,j}, o_{i,j}) = 0$ **then return** $\perp$
  return $(p_{i,j})_{j \in [\mathsf{n}(i)]}, \{\pi_{i,j}\}_{j \in [\mathsf{rnd}(i)]}$
if $i = \mathsf{rnd} + 1$ **then :**
  $((c_{i,j})_{j \in [\mathsf{n}(i)]}, \{\pi_{i,j}\}_{j \in [\mathsf{rnd}(i)]}, \pi_{\mathsf{php}}) \leftarrow \mathcal{A}_1(st, \rho_1, \ldots, \rho_{\mathsf{rnd}})$
  $\hat{\mathsf{x}}_{\mathsf{PHP}} := ((i_j, d_j)_{j \in [\mathsf{n}_\mathsf{p}]}, (G'_j, \boldsymbol{v}_j)_{j \in [\mathsf{n}_\mathsf{e}]}, (c_j)_{j \in [\mathsf{n}^*]})$
  if $\mathsf{Verify}_{\mathsf{php}}(\mathsf{vk}_{\mathsf{php}}, \hat{\mathsf{x}}_{\mathsf{PHP}}, \pi) = 0$ **then return** $\perp$
  $(p'_k, o'_k)_{k \in [\mathsf{n}^*]} \leftarrow \mathcal{E}_{\mathsf{php}}(\mathsf{srs})$
  for $k \in [\mathsf{n}^*]$ **let** $p_{i,j} := p'_{ind^{-1}(k)}$
  if $\exists i, j : p'_{i,j} \neq p_{i,j}$ **then return** $\perp$

Figure 5: Final hybrid and malicious rPHP prover

commitment[8] $c_h = [h(\tau)]_1$ to the polynomial $h(X) \in \mathbb{F}^{\leq n-1}[X]$ such that $h(\omega^j) = \Delta^j$ for $j = 0, \ldots, n-1$.

Round 1. Let $r_i, c_i \in [0, n-1]$ such that $\mathsf{row}(\kappa^i) = \omega^{r_i}$, $\mathsf{col}(\kappa^i) = \omega^{c_i}$ for $i \in [m]$. The prover starts by sending the randomized low degree extensions of the polynomials that encode the square roots of $\mathsf{row}(\kappa^i), \mathsf{col}(\kappa^i)$. More concretely, prover will sample $\rho_R(X), \rho_C(X) \leftarrow \mathbb{F}^{\leq 1}[X]$ and define

$$R(X) = \sum_{i=1}^{m} \Delta^{r_i} \lambda_i^{\mathbb{K}}(X) + \rho_R(X) z_{\mathbb{K}}(X), \quad C(X) = \sum_{i=1}^{m} \Delta^{c_i} \lambda_i^{\mathbb{K}}(X) + \rho_C(X) z_{\mathbb{K}}(X).$$

As we want to use Lemma 5, prover will also will compute the vector of multiplicities $m = (m_1, \ldots, m_n) \in \mathbb{F}^n$ that satisfies

$$\sum_{i=1}^{m} \frac{1}{R(\kappa^i) + X} + \frac{1}{C(\kappa^i) + X} + \frac{1}{\frac{C(\kappa^i)}{\Delta R(\kappa^i)} + X} + \frac{1}{\frac{C(\kappa^i)}{\Delta^t} + X} = \sum_{j=1}^{n} \frac{m_j}{h(\omega^j) + X}. \tag{7}$$

Again, prover will sample $\rho_m \leftarrow \mathbb{F}$ to compute the randomized low degree extension

$$m(X) = \sum_{j=1}^{n} m_j \lambda_j^{\mathbb{H}}(X) + \rho_m z_{\mathbb{H}}(X).$$

Finally, it will also sample $R_S, \rho_S \leftarrow \mathbb{F}$ and compute $S(X) = R_S X + \rho_S z_{\mathbb{K}}(X)$ to blind the sumcheck in the following rounds.

Recall that prover also needs to show that $\mathsf{rowcol}(\kappa^i) = \mathsf{row}(\kappa^i)\mathsf{col}(\kappa^i)$, which can be proven by sending the quotient $q(X)$ such that $\mathsf{rowcol}(X) - \mathsf{row}(X)\mathsf{col}(X) = q(X) z_{\mathbb{K}}(X)$. However, verifying this equation would require sending an evaluation of row or col, which would break zero-knowledge. For this reason, prover will sample $\rho_{\widetilde{\mathsf{row}}}(X) \leftarrow \mathbb{F}^{\leq 1}[X]$ and send a commitment to an auxiliary polynomial

$$\widetilde{\mathsf{row}}(X) = \sum_{i=1}^{m} \mathsf{row}(\kappa^i) \lambda_i^{\mathbb{K}}(X) + \rho_{\widetilde{\mathsf{row}}}(X) z_{\mathbb{K}}(X)$$

to later prove that $\mathsf{row}(\kappa^i) = \widetilde{\mathsf{row}}(\kappa^i)$ and $\mathsf{rowcol}(\kappa^i) = \widetilde{\mathsf{row}}(\kappa^i)\mathsf{col}(\kappa^i)$.

To sum up, in the first round the prover sends $[R(\tau), C(\tau), m(\tau), S(\tau), \widetilde{\mathsf{row}}(\tau)]_1$.

Round 2. The verifier sends a random challenge β. Now, the goal of the prover is to convince the verifier that equation (7) holds when evaluated at $X = \beta$. We will transform the sum over the smaller $\mathbb{H}$ to a sum over $\mathbb{K}$[9] so that we only need to perform one sumcheck. Let $z_{\mathbb{K}\setminus\mathbb{H}}(X) = \frac{n}{m}\frac{X^m - 1}{X^n - 1}$, which vanishes over $\mathbb{K} \setminus \mathbb{H}$ and has been normalized so that $z_{\mathbb{K}\setminus\mathbb{H}}(\omega^j) = 1$ for $j = 1, \ldots, n$. Then, $\sum_{j=1}^{n} \frac{m(\omega^j)}{h(\omega^j)+X} = \sum_{i=1}^{m} \frac{m(\kappa^i) z_{\mathbb{K}\setminus\mathbb{H}}(\kappa^i)}{h(\kappa^i)+X}$, so equation (7) evaluated at $X = \beta$ can be written as

$$\sum_{i=1}^{m} \frac{1}{\beta + R(\kappa^i)} + \frac{1}{\beta + C(\kappa^i)} + \frac{1}{\beta + \frac{C(\kappa^i)}{\Delta R(\kappa^i)}} + \frac{1}{\beta + \frac{C(\kappa^i)}{\Delta^t}} - \frac{m(\kappa^i) z_{\mathbb{K}\setminus\mathbb{H}}(\kappa^i)}{\beta + h(\kappa^i)} = 0. \tag{8}$$

[8] It could be argued that the commitment to the polynomial $h(X)$ should not be output by the key generation algorithm and instead it should be the prover that shows it has been correctly computed. Notice that $h(X)$ does not depend the specific relation $\mathsf{R} \in \mathcal{D}_{\mathsf{func}}(n, m, t, s)$ because it only depends on the public parameter n. As it could be shared for different relations, we argue it makes more sense to give a separate proof to show that $h(X)$ has been correctly computed instead of doing it in the PFR.

[9] Since both are smooth subgroups, we have that $\mathbb{H} \subseteq \mathbb{K}$.

We transform this rational sumcheck into a regular sumcheck using standard techniques. Namely, prover will sample $\rho_1(X) \leftarrow \mathbb{F}^{\leq 1}[X]$ and define $F_1(X) = \sum_{i=1}^{m} \frac{1}{\beta + R(\kappa^i)} \lambda_i^{\mathbb{K}}(X) + \rho_1(X) z_{\mathbb{K}}(X)$. Define $F_j(X)$ for $j = 2, \ldots, 5$ analogously for the rest of the summands in equation (8).

To sum up, in the second round the prover sends $[F_1(\tau), F_2(\tau), F_3(\tau), F_4(\tau), F_5(\tau)]_1$.

Round 3. The verifier sends a random challenge $\eta \leftarrow \mathbb{F}$. The goal of the prover in this round is to prove that $R(X)$, $C(X)$, $\widetilde{\mathsf{row}}(X)$, $\mathsf{rowcol}(X)$ and the $F_j(X)$ are well formed (in the sense that they take the claimed values over $\mathbb{K}$), and that $\sum_{i=1}^{n} \sum_{j=1}^{5} F_j(\kappa^i) = 0$.

As this last equation leaks information about $A(X)$, we will instead perform the sumcheck on $\sum_{j=1}^{5} F_j(X) + \eta S(X)$, where $S(X)$ is the random polynomial committed in the first round. Therefore, the equation becomes

$$\sum_{j=1}^{5} F_j(X) + \eta S(X) = (q_F(X) + \eta \rho_S) z_{\mathbb{K}}(X) + (R_F(X) + \eta R_S) X, \tag{9}$$

with $\deg(R_F) \leq m-2$. We apply the same idea from [CFF+24] to obtain a free degree check on $R_F(X)$. Let $U(X) = X^3 - 1$ and ask the prover to send $R^*(X) = (R_F(X) + \eta R_S) U(X)$, which is a degree $m+1$ polynomial, the maximum degree supported by the protocol. To balance this, we multiply the rest of equation (9) by $U(X)$, obtaining

$$\left(\sum_{j=1}^{5} F_j(X) + \eta S(X) \right) U(X) = (q_F(X) + \eta \rho_S) z_{\mathbb{K}}(X) U(X) + R^*(X) X. \tag{10}$$

Finally, we batch the previous claims using the verifier's challenge η. Namely, we define

$$\begin{aligned} P(X) = \Bigg(& F_1(X)(\beta + R(X)) - 1 + \eta\big(F_2(X)(\beta + C(X)) - 1\big) \\ & + \eta^2\big(F_3(X)(\beta + C(X)) - \Delta R(X)\big) + \eta^3\big(F_4(X)(\beta + C(X)) - \Delta^t\big) \\ & - \eta^4\big(F_5(X)(\beta + h(X)) - m(X) z_{\mathbb{K}\setminus\mathbb{H}}(X)\big) + \eta^5\big(R^2(X) - \mathsf{row}(X)\big) \\ & + \eta^6\big(C^2(X) - \mathsf{col}(X)\big) + \eta^7\big(\mathsf{rowcol}(X) - \widetilde{\mathsf{row}}(X)\mathsf{col}(X)\big) \\ & + \eta^8\big(\widetilde{\mathsf{row}}(X) - \mathsf{row}(X)\big) + \eta^9\Big(\sum_{j=1}^{5} F_j(X)\Big) + \eta^{10} S(X) \Bigg) U(X) - \eta^9 X R^*(X). \end{aligned}$$

and make the prover send a commitment to the quotient $q(X) = \frac{P(X)}{z_{\mathbb{K}}(X) U(X)}$.

To sum up, in this round the prover sends commitments $[R^*(\tau), q(\tau)]_1$.

Round 4. The verifier sends a random challenge $\alpha \leftarrow \mathbb{F}$, with the purpose of checking the equation $P(X) = q(X) z_{\mathbb{K}}(X) U(X)$ at α. With standard linearization tricks, we only need the evaluations $h_\alpha = h(\alpha)$, $R_\alpha = R(\alpha)$, $C_\alpha = C(\alpha)$ and $\widetilde{\mathsf{row}}_\alpha = \widetilde{\mathsf{row}}(\alpha)$ Therefore, the prover sends those evaluations to the verifier.

Round 5. The verifier sends a random challenge $\delta \leftarrow \mathbb{F}$ to batch the evaluation claims from the last round. Define the linearized polynomial

$$\begin{aligned}\mathsf{Lin}(X) = \Bigg(& F_1(X)(\beta + R_\alpha) - 1 + \eta\big(F_2(X)(\beta + C_\alpha) - 1\big) \\ & + \eta^2\big(F_3(X)(\beta + C_\alpha) - \Delta R_\alpha\big) + \eta^3\big(F_4(X)(\beta + C_\alpha) - \Delta^t\big) \\ & - \eta^4\big(F_5(X)(\beta + h_\alpha) - m(X) z_{\mathbb{K}\setminus\mathbb{H}}(\alpha)\big) + \eta^5\big(R_\alpha^2 - \mathsf{row}(X)\big) \\ & + \eta^6\big(C_\alpha^2 - \mathsf{col}(X)\big) + \eta^7\big(\mathsf{rowcol}(X) - \widetilde{\mathsf{row}}_\alpha \mathsf{col}(X)\big) + \eta^8\big(\widetilde{\mathsf{row}}(X) - \mathsf{row}(X)\big) \\ & + \eta^9\big(\sum_{j=1}^{5} F_j(X)\big) + \eta^{10} S(X)\Bigg) U(\alpha) - \eta^9 \alpha R^*(X) - q(X) U(\alpha) z_{\mathbb{K}}(\alpha),\end{aligned}$$

which satisfies that $\mathsf{Lin}(\alpha) = 0$. Finally, we batch all of the evaluation claims at α. Let

$$Q(X) = \frac{(h(X) - h_\alpha) + \delta(R(X) - R_\alpha) + \delta^2(C(X) - C_\alpha) + \delta^3(\widetilde{\mathsf{row}}(X) - \widetilde{\mathsf{row}}_\alpha) + \delta^4 \mathsf{Lin}(X))}{X - \alpha},$$

which serves as the evaluation proof for $h_\alpha = h(\alpha), R_\alpha = R(\alpha), C_\alpha = C(\alpha)$, $\widetilde{\mathsf{row}}_\alpha = \widetilde{\mathsf{row}}(\alpha)$ $\mathsf{Lin}(\alpha) = 0$. Prover sends the commitment $[Q(\tau)]_1$.

Proof. In summary, the proof is

$$\pi_{\mathsf{PFR}} = \left(\begin{array}{c} \big[R(\tau), C(\tau), m(\tau), S(\tau), \widetilde{\mathsf{row}}(\tau), F_1(\tau), F_2(\tau), F_3(\tau), \\ F_4(\tau), F_5(\tau), R^*(\tau), q(\tau), Q(\tau)\big]_1, h_\alpha, R_\alpha, C_\alpha, \widetilde{\mathsf{row}}_\alpha \end{array} \right) \in \mathbb{G}^{13} \times \mathbb{F}^4.$$

Verification. From the proof π_{PFR} and the input commitments, the verifier derives a commitment $[\mathsf{Lin}(\tau)]_1$ to the linearized polynomial, computes $[y]_1 = [h(\tau)]_1 - [h_\alpha]_1 + \delta([R(\tau)]_1 - [R_\alpha]_1) + \delta^2([C(\tau)]_1 - [C_\alpha]_1) + \delta^3([\widetilde{\mathsf{row}}(\tau)]_1 - [\widetilde{\mathsf{row}}_\alpha]_1) + \delta^4[\mathsf{Lin}(\tau)]_1$, and checks if

$$e\left([y]_1, [1]_2\right) \stackrel{?}{=} e([Q(\tau)]_1, [\tau - \alpha]_2)$$

Efficiency: Proof consists of 13 elements in $\mathbb{G}_1$ and 4 elements in $\mathbb{F}$. The prover is quasilinear in n, while the verifier is logarithmic in n and needs to perform 2 pairings.

Theorem 7. *The protocol above is a zero-knowledge succinct interactive argument of knowledge for $\hat{\mathcal{R}}_{\mathsf{func}}$ in the AGM under the $(m+2, 2)$-DLOG assumption (see Definition 1).*

The proof is almost identical to the one from [CFF+24] and therefore omitted.